

台灣自編大學教科書

[@prof](#)

機器人學導論

機構、感知、控制與具身人工智慧

第 二 篇

空間數學與操作臂運動學

第 8 章 空間描述、姿態與齊次轉換

第 9 章 操作臂正向運動學

第 10 章 逆向運動學

第 11 章 雅可比矩陣、速度與靜力

第 12 章 奇異點、冗餘與多任務運動

編著：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

2026 年 9 月

第二篇 篇章導讀

第一篇讓你能把一台機器人**組起來**，這一篇要讓你把它**算清楚**。

五章的內容環環相扣，構成一個完整的邏輯鏈。第 8 章先建立描述空間的語言——座標系、旋轉矩陣、四元數與齊次變換。這一章不談機器人，只談幾何，但它是後面所有計算的字母表：沒有這套語言，連「手臂末端在哪裡」都無法精確表達。

接下來的四章，依序回答關於機械手臂的四個問題。第 9 章（正向運動學）回答**手在哪裡**：把每個關節的角度代入 DH 參數，連乘變換矩陣就得到末端位姿。這是唯一必定有解、且解唯一的問題。第 10 章（逆向運動學）反過來問**手該怎麼擺**：已知目標位置，各關節該轉幾度？這困難得多——可能無解、可能有八組解、也可能有無限多解。第 11 章（雅可比矩陣）進到速度層級，回答**手能怎麼動**：關節轉多快，末端就跑多快？而同一個矩陣的轉置，還意外地連結了力與扭矩。第 12 章（奇異點與冗餘）則回答**手何時動不了**，並說明多餘的自由度該怎麼用。

這一篇有一條貫穿全程的暗線：**退化**。第 8 章的萬向節鎖、第 9 章手臂完全伸直、第 10 章兩組逆解合而為一、第 11 章可操作度歸零——這四個看似不相干的現象，在第 12 章被證明是同一件事：雅可比矩陣失去滿秩。一個線性代數課上看似抽象的概念「矩陣的秩」，就這樣成為理解機器人運動極限的鑰匙。

這一篇要回答的六個問題

- 如何用數字精確描述「一個物體在空間中的位置與方向」？
- 為什麼描述旋轉需要四種不同的表示法，而每一種都會在某處背叛你？
- 已知所有關節角度，末端執行器究竟在哪裡？
- 已知目標位置，各關節該轉幾度？為什麼答案可能有八個、或一個都沒有？
- 關節速度與末端速度之間的「兌換率」為什麼會隨姿態改變？
- 為什麼有些姿態下，機器人再怎麼用力也往某個方向動不了？

讀完這一篇之後

你將擁有完整分析一支機械手臂運動能力的工具：能建立 DH 參數表並計算末端位姿、能求出封閉解或以數值迭代逼近逆解、能構造雅可比矩陣並由它推算速度與力、能診斷奇異點並提出處理策略、能運用冗餘自由度在完成任務的同時最佳化次要目標。

但這一篇的所有分析都是**運動學的**——只談位置、速度與幾何，完全沒有考慮質量、慣性與力的動態效應。這隱含了一個假設：關節可以瞬間達到任何我們指定的速度。這個假設在低速時尚可接受，但在高速運動、重負載或需要精確力控制時就會失效。

第三篇將處理這些問題：手臂有質量、會因重力下垂、加速時需要額外的力矩、旋轉時會產生科氏力。屆時你會發現，本篇建立的雅可比矩陣仍然無所不在——它將出現在慣性矩陣、科氏項與操作空間控制的每一個公式中。

換句話說：第二篇教你機器人**能不能到那裡**，第三篇教你**該用多大的力氣到那裡**。

本篇章節配置

章次	章名	核心內容	叫獸開講
第 8 章	空間描述、姿態與齊次轉換	座標系、旋轉矩陣、歐拉角、四元數、齊次變換、轉換樹	「杯子就在桌上，機器人為什麼還要問：它在哪一個座標系？」——阿波羅十三號的萬向節鎖
第 9 章	操作臂正向運動學	DH 參數、連桿變換、平面手臂、SCARA、六軸手臂、標定	「知道每一顆馬達轉幾度，怎麼算出機器手指尖的位置？」——調了三十次滑桿卻算不出來
第 10 章	逆向運動學	可達性、多解結構、解耦、Pieper 準則、數值迭代、DLS	「人看到杯子就伸手，機器人卻可能算出八種姿勢。」——無解、多解與無限多解
第 11 章	雅可比矩陣、速度與靜力	幾何構造、角速度、虛功原理、速度力對偶、可操作度	「為什麼機器手臂在某些姿勢突然變得很笨重？」——不斷改變的兌換率
第 12 章	奇異點、冗餘與多任務運動	秩與 SVD、三類奇異點、零空間、次要目標、分層逆運動學	「手臂伸得太直，反而最沒有力？」——四個現象，同一個原因

第二篇 空間數學與操作臂運動學

第 8 章

空間描述、姿態與齊次轉換

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】左半為阿波羅十三號登月艙內的慣性導航平台特寫——三個同心萬向環，其中兩環已接近對齊，旁邊放著任務手冊上標示紅色禁區的姿態球圖；右半為現代六軸機械手臂，其基座、法蘭、工具與工件上各疊印一組彩色座標軸（ X 紅、 Y 綠、 Z 藍），並以虛線箭頭標出彼此的變換關係。標題文字：從萬向節鎖到四元數——描述「朝向」的五十年。

叫獸開講 「杯子就在桌上，機器人為什麼還要問：它在哪一個座標系？」

一九七〇年四月十四日，阿波羅十三號的服務艙氧氣罐爆炸。太空人在漆黑的太空中，用僅存的登月艙電力維生，準備繞過月球背面返航。返航前必須執行一次精準的引擎點火，而這需要先讓太空船轉到正確的姿態。

問題來了：導航系統的慣性平台，是靠三個同心的機械環（萬向架）維持一個不隨太空船轉動的參考方向。任務手冊上有一條紅色的警告——某些姿態組合會導致「萬向節鎖」，三個環中的兩個會排列到同一平面上，這時整個平台會瞬間失去一個轉動自由度，導航資料全部作廢，必須用肉眼觀測星星重新校準。在氧氣與電力都所剩無幾的當下，這是他們絕對不能承受的損失。

地面控制中心花了數小時，替太空人算出一條「繞開禁區」的旋轉路徑。他們必須用特定的順序、特定的角度轉動太空船，才能既到達目標姿態、又不讓那三個環對齊。

叫獸講到這裡停下來，拿出一個玩具陀螺儀教具，讓三個環自由轉動。「你們看，」他把中間那個環轉了九十度，「現在最外圈和最內圈的軸，是不是變成同一個方向了？」學生們湊近看——確實，原本互相垂直的三個轉軸，有兩個重合了。「這時候不管我怎麼轉這兩個環，都只能得到同一種轉動。三個自由度變成兩個，有一個方向永遠到不了。」

「更麻煩的是，」他繼續轉動教具，「這不是機械故障，這是**數學本身的問題**。用三個角度描述空間旋轉，無論你怎麼選這三個軸，一定會存在這樣的奇異點。這是幾何學的定理，不是工程師偷懶。」

有學生問：「那現在的機器人怎麼處理？」叫獸在白板上寫下四個字：**四元數**。「今天的無人機、遊戲引擎、機器手臂，內部幾乎都不用歐拉角來儲存姿態，而是用一個四維的數學物件。它沒有奇異點、計算快、內插平滑——代價是完全不直觀，你看著四個數字，腦中浮現不出任何畫面。」

他把玩具放回桌上：「所以這一章我們要學四種描述姿態的方法。不是因為老師要為難你們，而是因為**每一種都有它擅長的場合**：跟人溝通用歐拉角、內部計算用四元數、做座標轉換用矩陣。工程師的專業，就在於知道什麼時候該用哪一種——以及知道每一種會在哪裡背叛你。」

本章為什麼重要

從本章開始，我們進入機器人學的數學核心。第一篇讓你能組出一台會動的機器人，第二篇

要讓你能精確描述並計算它的運動——而這一切的起點，是回答一個問題：**如何用數字描述「一個物體在空間中的位置與方向」**？

這個問題比想像中深刻。位置很簡單，三個座標就夠了；但「方向」（姿態）卻是機器人學中最微妙的概念之一。它有多種表示法，各有優劣與陷阱，而且旋轉不像位移那樣可以隨意交換順序——先轉 X 再轉 Y，與先轉 Y 再轉 X，結果完全不同。

本章建立的座標轉換工具，將是後續所有章节的共同語言：第 9 章的正向運動學是一連串齊次變換的連乘；第 10 章的逆運動學是求解這些變換的反問題；第 22 章的手眼校正是求相機座標與機器人座標的關係；第 25 章的 SLAM 是不斷估計機器人座標系與地圖座標系的變換。可以說，學不好本章，後面二十八章都會卡住。

學習目標

- 能定義座標系、位置向量與姿態，並說明右手座標系的慣例。
- 能寫出並解釋二維與三維旋轉矩陣，並驗證其正交性質。
- 能說明旋轉矩陣的三種幾何詮釋（座標軸、旋轉算子、座標變換）。
- 能使用歐拉角與滾轉—俯仰—偏航描述姿態，並解釋萬向節鎖的成因。
- 能說明軸角表示法與 Rodrigues 公式的意義。
- 能定義單位四元數，說明其與軸角的關係及無奇異點的優勢。
- 能比較四種姿態表示法的優缺點並依場合正確選用。
- 能建構齊次變換矩陣，並用它同時表達平移與旋轉。
- 能正確判斷變換的相乘順序，並區分「向前框架變換」與「向後座標變換」。
- 能計算齊次變換矩陣的反矩陣並說明其物理意義。
- 能說明基座、工具、工件與相機座標系的關係，並繪製轉換樹。
- 能以 Python 實作座標轉換並驗證計算結果。

先備知識

本章需要基本的線性代數：向量加減、內積與外積、矩陣乘法、反矩陣與行列式的概念。若這些概念已經生疏，請先閱讀附錄 D.1 的數學速成，或在遇到時隨時回查。第 2 章曾說明「即時

學習」的策略——本章正是實踐它的最佳時機：你會看到每一個數學工具都對應到一個具體的機器人問題。

三角函數（正弦、餘弦、反正切）在本章大量使用。特別提醒：程式中計算角度務必使用 $\text{atan2}(y, x)$ 而非 $\text{atan}(y/x)$ ，理由已在第 2 章 2.4.3 節說明。

核心名詞中英對照

表 8-1 本章核心名詞

中文	英文	說明
座標系 / 框架	Coordinate Frame	描述位置與方向所依據的參考系統
位姿	Pose	位置與姿態的合稱
姿態	Orientation / Attitude	物體在空間中的朝向
旋轉矩陣	Rotation Matrix	描述姿態的 3×3 正交矩陣
特殊正交群	Special Orthogonal Group, $SO(3)$	所有三維旋轉矩陣所構成的集合
正交矩陣	Orthogonal Matrix	行（列）向量彼此正交且為單位長度的矩陣
右手定則	Right-hand Rule	決定座標軸與旋轉正方向的慣例
歐拉角	Euler Angles	以三次連續旋轉描述姿態的三個角度
滾轉俯仰偏航	Roll-Pitch-Yaw, RPY	繞固定軸的三個旋轉角，航空慣例
萬向節鎖	Gimbal Lock	兩軸對齊導致失去一個自由度的奇異現象
軸角表示法	Axis-Angle Representation	以旋轉軸與旋轉角描述姿態
Rodrigues 公式	Rodrigues' Rotation Formula	由軸角求旋轉矩陣的公式
四元數	Quaternion	以四個數描述旋轉的數學物件
齊次座標	Homogeneous Coordinates	以增加一維的方式統一表示平移與旋轉
齊次變換矩陣	Homogeneous Transformation Matrix	同時描述平移與旋轉的 4×4 矩陣
特殊歐氏群	Special Euclidean Group, $SE(3)$	所有剛體變換所構成的集合
轉換樹	Transform Tree, TF	描述多座標系間關係的樹狀結構

8.1 座標系、位置與姿態

8.1.1 為什麼需要座標系

「杯子在桌上」這句話對人類足夠了，對機器人卻毫無意義。機器人需要知道的是：杯子相對於什麼、在哪個方向、距離多遠。這個「相對於什麼」，就是座標系（coordinate frame，又譯參考框架）。

一個座標系由一個原點與三個互相垂直的單位向量（座標軸）構成。在機器人學中，我們幾乎總是使用**右手座標系**：伸出右手，食指指向 X 軸、中指指向 Y 軸、拇指指向 Z 軸；三者互相垂直。這個慣例的價值在於統一——若團隊中有人用左手系，所有的外積計算與旋轉方向都會反過來，錯誤會非常隱晦難查。

同樣地，**旋轉的正方向**也由右手定則決定：右手拇指指向轉軸的正方向，四指彎曲的方向就是正的旋轉方向（逆時針，由軸的正方向往原點看）。

8.1.2 位置：一個向量

點 P 在座標系 {A} 中的位置，以一個三維向量表示，習慣寫作 ${}^A\mathbf{P}$ ，左上標表示「相對於座標系 A」：

$${}^A\mathbf{P} = [p_x \ p_y \ p_z]^T$$

左上標 A 表示此向量是在座標系 {A} 中量測的。這個上標慣例在本書中會嚴格遵守，因為它是避免混淆的關鍵。

同一個物理點，在不同座標系中會有完全不同的數值。這不是矛盾，而是必然——就像同一個地點，用經緯度描述與用「從火車站往北三百公尺」描述，數字完全不同但指的是同一處。機器人學中大量的錯誤，都源自於忘記標明「這個數字是在哪個座標系裡」。

8.1.3 姿態：三個座標軸的方向

描述姿態的直覺方法是：把物體上的座標系 {B} 的三個座標軸，用座標系 {A} 的語言表達出來。設 {B} 的三個單位軸向量在 {A} 中分別為 ${}^A\mathbf{x}_B$ 、 ${}^A\mathbf{y}_B$ 、 ${}^A\mathbf{z}_B$ ，把它們並排成一個矩陣：

$${}^A\mathbf{R}_B = [{}^A\mathbf{x}_B \ {}^A\mathbf{y}_B \ {}^A\mathbf{z}_B] \quad (3 \times 3 \text{ 矩陣, 三個行向量})$$

這就是旋轉矩陣。它的每一行，是 {B} 的一個座標軸在 {A} 中的方向。

這個定義極為重要，因為它讓旋轉矩陣不再是抽象的數字表格，而是有明確的幾何意義：看矩陣的第一行，就知道 B 的 X 軸指向哪裡。8.2 節會由此推導出旋轉矩陣的所有性質。

圖 8-1 兩個座標系與位置向量

建議配圖：三維透視圖。世界座標系 $\{A\}$ (原點在左下，三軸以粗箭頭標示 X_A 、 Y_A 、 Z_A) 與物體座標系 $\{B\}$ (原點偏右上方，三軸為 X_B 、 Y_B 、 Z_B 且相對 $\{A\}$ 有旋轉)。由 $\{A\}$ 原點指向 $\{B\}$ 原點的向量標為 ${}^A P_{B_{or}}^g$ ，並在 $\{B\}$ 上放一個機械手爪示意物體。旁註：位置由向量描述、姿態由三軸方向描述。

8.2 旋轉矩陣與 $SO(3)$

8.2.1 二維旋轉：從最簡單的開始

先看平面上的旋轉。若座標系 $\{B\}$ 相對 $\{A\}$ 逆時針轉了角度 θ ，則 $\{B\}$ 的 X 軸在 $\{A\}$ 中的方向是 $[\cos \theta, \sin \theta]^T$ ， Y 軸則是 $[-\sin \theta, \cos \theta]^T$ 。依 8.1.3 節的定義並排成矩陣：

$$R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

二維旋轉矩陣。注意負號的位置——它出現在右上角，這是逆時針為正的右手慣例。

驗算一個直覺：若把點 $[1, 0]^T$ (在 X 軸上) 用 $R(90^\circ)$ 旋轉，得到 $[0, 1]^T$ (跑到 Y 軸上)，符合逆時針轉 90 度的預期。

8.2.2 三維的三個基本旋轉

三維空間中，繞三個座標軸的旋轉矩陣分別為：

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c\theta & -s\theta \\ 0 & s\theta & c\theta \end{bmatrix}$$

繞 X 軸旋轉。 $c\theta$ 與 $s\theta$ 為 $\cos \theta$ 與 $\sin \theta$ 的簡寫。注意 X 那一列與那一行保持不變。

$$R_y(\theta) = \begin{bmatrix} c\theta & 0 & s\theta \\ 0 & 1 & 0 \\ -s\theta & 0 & c\theta \end{bmatrix}$$

繞 Y 軸旋轉。注意負號的位置與 X 、 Z 不同——這是初學者最常抄錯的一個。

$$R_z(\theta) = \begin{bmatrix} c\theta & -s\theta & 0 \\ s\theta & c\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

繞 Z 軸旋轉。 Z 那一列與那一行保持不變。

記憶技巧：繞哪一軸旋轉，該軸對應的那一行與那一列就是單位向量（保持不變），剩下的 2×2 子矩陣就是二維旋轉矩陣。唯一的例外是 R_y 的負號位置與另外兩個相反——原因是 $Z \rightarrow X$ 的循環順序與 $X \rightarrow Y$ 、 $Y \rightarrow Z$ 不同。

8.2.3 旋轉矩陣的性質

由 8.1.3 節的定義（三個行是互相垂直的單位向量），可以直接推出旋轉矩陣的核心性質：

$$\mathbf{R}^T \mathbf{R} = \mathbf{I} \Rightarrow \mathbf{R}^{-1} = \mathbf{R}^T$$

旋轉矩陣是正交矩陣，其反矩陣等於轉置。這是機器人學中最有價值的一條性質。

為什麼這條性質這麼重要？因為求一般 3×3 矩陣的反矩陣需要計算行列式與伴隨矩陣，運算量大且有數值誤差；而轉置只是把元素位置對調，幾乎不花任何運算。這意味著座標的反向轉換是免費的——這個特性在每秒要做上千次轉換的機器人系統中，價值極高。

此外，旋轉矩陣的行列式恆為 $+1$ ：

$$\det(\mathbf{R}) = +1$$

行列式為 $+1$ 代表這是「純旋轉」（保持右手系）。若 $\det(\mathbf{R}) = -1$ ，則包含了鏡射，會把右手系變成左手系，在物理上對應不到任何剛體運動。

所有滿足這兩個條件（正交且行列式為 $+1$ ）的 3×3 矩陣，構成一個集合，稱為**特殊正交群** $\mathbf{SO}(3)$ 。它是一個群（group），意思是：兩個旋轉矩陣相乘仍是旋轉矩陣（封閉性）、有單位元素（單位矩陣，代表不轉）、每個元素有反元素（反向旋轉）。這個結構保證了旋轉的組合永遠有意義。

8.2.4 旋轉矩陣的三種詮釋

同一個旋轉矩陣，在不同場合有三種讀法，能靈活切換是理解本章的關鍵。

詮釋一：座標軸的描述。 $\mathbf{AR}_B$ 的三個行，是 $\{B\}$ 的三個座標軸在 $\{A\}$ 中的方向（8.1.3 節的定義）。

詮釋二：座標的轉換。若已知點在 $\{B\}$ 中的座標 $\mathbf{BP}$ ，可求其在 $\{A\}$ 中的座標： $\mathbf{AP} = \mathbf{AR}_B \cdot \mathbf{BP}$ 。這是實務上最常用的一種。

詮釋三：旋轉的算子。 $\mathbf{R}$ 也可以視為一個「動作」：把一個向量在同一個座標系內轉動。例如 $\mathbf{R}_z(90^\circ)$ 作用在向量上，把它繞 Z 軸轉 90 度。

詮釋二與詮釋三在數學上是同一個式子，但物理意義相反：詮釋二是「物體不動，換個角度描述它」；詮釋三是「座標系不動，把物體轉動」。這個「主動旋轉 vs 被動旋轉」的區別是初學者混淆的主要來源，第 8.7 節會再從變換的角度深入。

◎ 一個檢查錯誤的好習慣

拿到一個聲稱是旋轉矩陣的數值矩陣時，做兩個快速檢查：一、每一行的長度是否為 1 （平方和為

1) ? 二、任兩行的內積是否為 0 (互相垂直) ? 若有偏差 (例如 0.998 而非 1.000) , 通常是數值誤差累積所致 , 可用正交化 (如 Gram-Schmidt 或 SVD) 修正。長期累積不修正 , 會導致機器人的姿態逐漸「歪掉」。

8.3 歐拉角與 RPY

8.3.1 用三個角度描述姿態

旋轉矩陣有九個數字 , 卻只有三個自由度 (因為有六個正交條件約束) , 顯得非常冗餘。歐拉角則用最少的三個數字來描述姿態 : 把姿態拆解成繞三個軸的連續旋轉。

關鍵在於 , 這三次旋轉可以有不同的**軸序**與**參考基準** , 因此歐拉角有多達二十四種慣例 , 這是實務上混亂的根源。兩個主要的分類軸是 :

內旋 (intrinsic) vs 外旋 (extrinsic) : 內旋是繞著「轉動後的新座標軸」旋轉 (動軸) , 外旋是繞著「固定不動的原座標軸」旋轉 (定軸) 。

軸序 : 常見的有 Z-Y-X、Z-X-Z、X-Y-Z 等。航空領域常用 Z-Y-X , 古典力學常用 Z-X-Z 。

8.3.2 滾轉—俯仰—偏航 (RPY)

機器人與航空領域最常用的是 RPY 慣例 : 繞**固定軸**依序旋轉 , 先繞 X 軸轉滾轉角 γ (roll) 、再繞 Y 軸轉俯仰角 β (pitch) 、最後繞 Z 軸轉偏航角 α (yaw) 。對應的旋轉矩陣為 :

$$R_{RPY}(\alpha, \beta, \gamma) = R_z(\alpha) \cdot R_y(\beta) \cdot R_x(\gamma)$$

注意相乘順序 : 先做的旋轉寫在右邊。這是繞固定軸旋轉的慣例 , 稱為外旋。

這個「先做的寫右邊」規則常令初學者困惑。可以這樣理解 : 矩陣作用在向量上時 , 是由右往左依序作用的 ($R_z \cdot R_y \cdot R_x \cdot P$, 最先碰到 P 的是 R_x) 。因此最先執行的旋轉必須寫在最右邊。

一個有用的定理是 : 繞**固定軸**依序 **X-Y-Z** 旋轉 , 等價於繞**動軸**依序 **Z-Y-X** 旋轉。這解釋了為什麼有些文獻說 RPY 是「Z-Y-X 歐拉角」、有些說是「X-Y-Z 固定角」——它們指的是同一件事。

對飛行器與移動機器人而言 , RPY 的直覺意義是 : 滾轉是機身左右傾斜、俯仰是機頭上下、偏航是機頭左右轉向。這個直覺性正是它在人機介面上不可取代的原因——你可以對操作員說

「機頭抬高十度」，但不能說「四元數的第二個分量增加 0.087」。

8.3.3 萬向節鎖：歐拉角的致命缺陷

回到開頭的阿波羅十三號。萬向節鎖發生在**中間那次旋轉達到 $\pm 90^\circ$ 度時**。以 Z-Y-X 慣例為例，當俯仰角 $\beta = 90^\circ$ 時，把它代入矩陣運算會發現：第一次的 Z 軸旋轉與第三次的 X 軸旋轉，在效果上變成了繞同一個軸的旋轉。

當 $\beta = 90^\circ$ 時， α 與 γ 只以 $(\alpha - \gamma)$ 的組合出現

兩個自由度退化成一個。此時給定一個姿態， α 與 γ 有無限多組解，數值計算會失去唯一性且極不穩定。

這造成三個實際問題。第一，**失去一個自由度**：無論怎麼調整 α 與 γ ，都有一個方向的旋轉到不了。第二，**數值不穩定**：在接近奇異點時，微小的姿態變化會導致角度值劇烈跳動（例如從 179° 跳到 -179° ）。第三，**內插失敗**：若在兩個姿態之間對三個角度做線性內插，路徑會扭曲甚至發生翻轉。

必須強調：這不是某種慣例的缺陷，而是數學的必然。**任何以三個數描述三維旋轉的方法，都必然存在奇異點**。要避開它，只能增加維度——這就是四元數（四個數）與旋轉矩陣（九個數）存在的理由。

圖 8-2 萬向節鎖的幾何解釋

建議配圖：三格連續圖。左格：三個同心環（外環繞 Z、中環繞 Y、內環繞 X）互相垂直，三個轉軸指向不同方向，標註「三個獨立自由度」。中格：中環轉了 90° 度，此時外環軸與內環軸變成平行（重合），以紅色虛線標出兩軸重合。右格：無論轉動外環或內環，物體都只繞同一軸轉動，標註「自由度由 3 降為 2」。下方附阿波羅任務手冊上禁區示意。

8.4 軸角表示法與 Rodrigues 公式

8.4.1 歐拉旋轉定理

有一個優美的定理（歐拉旋轉定理）指出：**任何三維旋轉，都可以表示為繞某一個固定軸旋轉某一個角度**。無論姿態多複雜，都不需要三次旋轉，一次就夠——只要選對軸。

因此可以用一個單位向量 $\mathbf{k}$ （旋轉軸）與一個純量 θ （旋轉角）來描述姿態，這稱為軸角表示法。它常被寫成一個三維向量：

$$\mathbf{r} = \theta \cdot \mathbf{k} \quad (\text{旋轉向量 / Rotation Vector})$$

向量的方向是旋轉軸，長度是旋轉角（弧度）。這是最精簡的表示法，只需三個數且具有明確的幾何意義。

8.4.2 Rodrigues 公式

由軸角求旋轉矩陣的公式稱為 Rodrigues 公式：

$$R = I + \sin \theta \cdot [k] \times + (1 - \cos \theta) \cdot [k] \times^2$$

其中 $[k] \times$ 是 k 的反對稱矩陣 (skew-symmetric matrix)，定義於下式。

$$[k] \times = \begin{bmatrix} 0 & -k_z & k_y \\ k_z & 0 & -k_x \\ -k_y & k_x & 0 \end{bmatrix}$$

反對稱矩陣，滿足 $[k] \times \cdot v = k \times v$ (外積)。它是連接向量外積與矩陣運算的橋樑，在第 11 章推導雅可比矩陣時會大量使用。

反對稱矩陣是一個值得熟悉的工具。它的核心性質是：把外積運算轉換成矩陣乘法。在機器人學中，角速度與線速度的關係、力矩與力的關係，都會用到這個轉換。

8.4.3 優點與陷阱

軸角表示法的優點是幾何意義明確、只需三個數、且在小角度時有優雅的近似 (當 θ 很小， $R \approx I + [r] \times$ ，這是第 11 章微分運動學的基礎)。

但它仍有奇異點：當 $\theta = 0$ 時，旋轉軸 k 完全沒有定義 (不轉的話，繞哪個軸都一樣)；當 θ 接近 180° 時， k 與 $-k$ 對應同一個旋轉，數值計算會不穩定。此外，兩個旋轉向量無法直接相加來合成旋轉——這是初學者常犯的錯誤 (旋轉不可交換，所以不能像位移那樣疊加)。

8.5 四元數

8.5.1 定義與幾何意義

四元數是十九世紀數學家 Hamilton 發明的數系，可視為複數的三維推廣。一個四元數由四個實數組成：

$$q = q_0 + q_1 i + q_2 j + q_3 k \quad \text{或寫作} \quad q = (q_0, \bar{q}), \quad \text{其中 } \bar{q} = [q_1, q_2, q_3]^T$$

q_0 稱為純量部 (實部)， $\bar{q}$ 稱為向量部 (虛部)。

用於描述旋轉的是單位四元數 (長度為 1)。它與軸角表示法有一個極為優雅的對應關係：若旋轉軸為單位向量 k 、旋轉角為 θ ，則

$$q_0 = \cos(\theta/2), \quad \bar{q} = \sin(\theta/2) \cdot k$$

注意是半角。這個 $\theta/2$ 是四元數所有神奇性質的來源，也是它最反直覺的地方。

為什麼是半角？直觀的理解是：四元數作用在向量上時，需要「左乘一次、右乘一次」($q \cdot v \cdot q^{-1}$)，因此每次貢獻一半的角度。這也導致一個重要現象： q 與 $-q$ 代表**完全相同的旋轉**（因為 θ 與 $\theta+360^\circ$ 是同一個姿態，但半角差了 180° ，使四個分量全部變號）。這稱為**雙重覆蓋**（double cover），在實作中必須注意——比較兩個四元數是否相同時，要同時檢查 q 與 $-q$ 。

8.5.2 三大優勢

優勢一：無奇異點。四元數在整個旋轉空間中都表現良好，不存在萬向節鎖。這是它成為無人機飛控與遊戲引擎標準的首要原因。

優勢二：計算效率高。兩個四元數相乘（合成兩個旋轉）需要 16 次乘法與 12 次加法；兩個 3×3 旋轉矩陣相乘則需要 27 次乘法與 18 次加法。且四元數只需儲存 4 個數而非 9 個。在每秒數千次姿態更新的系統中，這個差距很可觀。

優勢三：內插平滑。在兩個姿態之間平滑過渡時，四元數可用球面線性內插（SLERP），得到的是繞固定軸的等速旋轉——這是最自然的路徑。相較之下，對歐拉角做線性內插會產生扭曲的路徑，對旋轉矩陣的元素做內插甚至會產生非旋轉矩陣。這正是第 15 章姿態軌跡規劃必用四元數的原因。

8.5.3 基本運算

旋轉的合成（相當於矩陣相乘）用四元數乘法（Hamilton 積）：

$$q \cdot p = (q_0 p_0 - \mathbf{q} \cdot \mathbf{p}, \quad q_0 \mathbf{p} + p_0 \mathbf{q} + \mathbf{q} \times \mathbf{p})$$

注意四元數乘法不可交換（ $q \cdot p \neq p \cdot q$ ），這正確反映了旋轉不可交換的事實。

反向旋轉（相當於矩陣轉置）用共軛：

$$q^* = (q_0, -\mathbf{q}) \quad (\text{對單位四元數, } q^{-1} = q^*)$$

只需把向量部變號，運算成本極低——與旋轉矩陣的轉置一樣便宜。

旋轉一個向量 v ：

$$v' = q \cdot (0, v) \cdot q^*$$

把向量寫成純量部為 0 的四元數，左乘 q 、右乘 q^* 。實作上通常會展開成更高效的形式，或先轉成旋轉矩陣再作用。

8.5.4 最大的缺點：不直觀

四元數的唯一但致命的缺點是：人類完全無法直觀理解。給你一組數值 (0.707, 0, 0.707, 0)，你的腦中浮現不出任何畫面（它其實是繞 Y 軸轉 90 度）。因此實務上的標準做法是：內部用四元數儲存與計算，介面上轉成歐拉角顯示給人看。

這正是本章開頭叫獸強調的重點——不同表示法各有其舞台。無人機的飛控軟體內部用四元數運算，但地面站的螢幕上顯示的是「俯仰 12 度、滾轉 -3 度」；機器手臂的控制器內部用旋轉矩陣，但示教器上讓操作員輸入的是 RPY 角。

表 8-2 四種姿態表示法的比較

表示法	參數數目	奇異點	合成運算	內插	直觀性	典型用途
旋轉矩陣	9 (3 自由度)	無	矩陣相乘 (27 乘法)	困難 (需正交化)	中	座標轉換、運動學計算
歐拉角 / RPY	3	有 (萬向節鎖)	需先轉成矩陣	會扭曲	高	人機介面、規格標示
軸角 / 旋轉向量	3	有 ($\theta=0$ 與 180°)	不可直接相加	尚可	中高	小角度近似、微分運動學
單位四元數	4 (3 自由度)	無	四元數乘法 (16 乘法)	優 (SLERP)	極低	飛控、遊戲引擎、姿態濾波

◎ 選用指南

與人溝通、顯示與輸入 → 歐拉角 / RPY。座標轉換與運動學推導 → 旋轉矩陣（可與平移合併為齊次變換）。內部儲存、姿態更新與內插 → 四元數。微小旋轉的線性化分析 → 軸角 / 旋轉向量。四者之間可以自由轉換，實務上經常在同一個系統中並存——重點是每次轉換都要清楚知道自己在哪一種表示法中。

8.6 齊次座標與齊次變換矩陣

8.6.1 把平移與旋轉合而為一

到目前為止，旋轉用矩陣描述、平移用向量描述。一個完整的剛體位姿變換要同時包含兩者：

$$\mathbf{AP} = \mathbf{AR}_B \cdot \mathbf{BP} + \mathbf{AP}_{B_{or}^g}$$

點在 {B} 中的座標先經旋轉，再加上 {B} 原點在 {A} 中的位置。這是最直接的寫法，但有一個缺點——它是「乘法加上加法」的混合形式。

為什麼混合形式不好？因為當我們要串接多次變換時（第 9 章的運動鏈有六、七次），這個式子會層層嵌套成一個難以整理的長串。若能把它變成純粹的矩陣乘法，就可以直接連乘，形式極為簡潔。

解法是一個巧妙的技巧：**升高一個維度**。把三維座標寫成四維，最後一個元素固定為 1：

$$\tilde{\mathbf{P}} = [p_x \ p_y \ p_z \ 1]^T$$

這稱為齊次座標 (*homogeneous coordinates*)。多出來的那個 1，是讓平移能被「乘」進去的關鍵。

8.6.2 齊次變換矩陣

有了齊次座標，就可以把旋轉與平移打包成一個 4×4 矩陣：

$${}^A\mathbf{T}_B = \begin{bmatrix} {}^A\mathbf{R}_B & {}^A\mathbf{p}_{B_o.r^g} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & 1 \end{bmatrix}$$

左上 3×3 是旋轉、右上 3×1 是平移、左下三個 0 與右下的 1 是為了維持矩陣運算的一致性。

驗證它確實做對了事：

$${}^A\tilde{\mathbf{P}} = {}^A\mathbf{T}_B \cdot {}^B\tilde{\mathbf{P}} \Rightarrow \begin{bmatrix} {}^A\mathbf{p} \\ 1 \end{bmatrix} = \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ \mathbf{0} & 1 \end{bmatrix} \begin{bmatrix} {}^B\mathbf{p} \\ 1 \end{bmatrix} = \begin{bmatrix} \mathbf{R} \cdot {}^B\mathbf{p} + \mathbf{p} \\ 1 \end{bmatrix}$$

展開後的上三列正是 8.6.1 節的式子，而最後一列給出 $1 = 1$ （恆成立）。齊次變換確實等價於「旋轉後再平移」。

所有這種形式的 4×4 矩陣構成的集合，稱為**特殊歐氏群 SE(3)**，代表所有可能的剛體運動。它同樣是一個群：兩個剛體變換的組合仍是剛體變換、有單位元素、每個元素有反元素。

8.6.3 為什麼要那么多餘的一列

學生常問：最後一列 $[0 \ 0 \ 0 \ 1]$ 看起來毫無作用，為什麼要加？答案有三個。

第一，**維持矩陣可乘性**。只有方陣才能連乘任意多次並保持形式一致，這正是第 9 章運動學連乘的需求。

第二，**保證反矩陣存在**。 4×4 的齊次變換矩陣行列式為 1（非零），必定可逆。

第三，**可擴充性**。在電腦圖學中，把最後一列改成非零值可以表示透視投影；把右下角的 1 改成其他值可以表示縮放。機器人學中不用這些功能，但這正是相機成像模型（第 22 章）使用同一套框架的原因。

8.7 變換的組合與相乘順序

8.7.1 連鎖法則

齊次變換最強大的地方在於它們可以直接串接。若已知 {A} 到 {B} 的變換、以及 {B} 到 {C} 的變換，則 {A} 到 {C} 的變換為：

$${}^A T_C = {}^A T_B \cdot {}^B T_C$$

上下標的「消去律」：右邊相鄰的 B 與 B 互相抵消，剩下 A 到 C 。這是檢查式子有沒有寫錯的最快方法。

這條消去律極其實用。例如第 9 章的六軸手臂正向運動學：

$${}^0 T_6 = {}^0 T_1 \cdot {}^1 T_2 \cdot {}^2 T_3 \cdot {}^3 T_4 \cdot {}^4 T_5 \cdot {}^5 T_6$$

從基座 (0) 到末端 (6) 的變換，就是把每一節連桿的變換依序相乘。相鄰的上下標兩兩抵消，剩下 0 到 6。

看到這裡，第 4 章數過的自由度、第 5 章選過的關節，就在這一行式子裡被統一起來了——每一個 ${}^{i-1}T_i$ 都包含一個關節變數。這就是機器人運動學的核心方程式。

8.7.2 順序至關重要

矩陣乘法不可交換，因此 $T_1 \cdot T_2 \neq T_2 \cdot T_1$ 。這在物理上完全合理：先向前走再左轉，跟先左轉再向前走，到達的位置完全不同。

親手做一個實驗最能建立直覺：拿一本書平放在桌上，先繞書的長邊翻轉 90 度、再繞短邊翻轉 90 度，記住最終姿態；把書放回原位，改成先繞短邊、再繞長邊，你會得到完全不同的姿態。這個「不可交換性」是三維旋轉最重要也最違反直覺的性質。

8.7.3 左乘與右乘：定軸與動軸

既然順序重要，就必須弄清楚：新的變換應該乘在左邊還是右邊？這對應到兩種不同的物理操作。

右乘 = 相對於當前的自身座標系 (動軸)。 $T_{\text{new}} = T_{\text{current}} \cdot T_{\text{relative}}$ 。例如機器手臂「沿著自己夾爪的方向前進 5 公分」，這個運動是相對於夾爪自身的座標系，因此右乘。

左乘 = 相對於固定的參考座標系 (定軸)。 $T_{\text{new}} = T_{\text{relative}} \cdot T_{\text{current}}$ 。例如「把整個物體繞世界座標系的 Z 軸旋轉 30 度」，這是相對於外部固定座標系，因此左乘。

這個規則可以用 8.7.1 節的消去律驗證： ${}^A T_C = {}^A T_B \cdot {}^B T_C$ 中，若 {B} 是當前位姿、{C} 是相對於 {B} 的新變換，則新變換寫在右邊——正是右乘的形式。

圖 8-3 左乘與右乘的差異

建議配圖：兩列對照圖。上列（右乘／動軸）：機械手臂末端沿著自身 Z 軸方向前進，箭頭隨夾爪姿態改變方向；下列（左乘／定軸）：整個手臂繞世界座標系的 Z 軸旋轉，箭頭固定指向世界的垂直方向。各列標出對應的矩陣式子與適用情境。

8.7.4 「向前框架變換」與「向後座標變換」

這是一個常令人困惑但極為重要的觀念。同一個矩陣 A^T_B 有兩種讀法，兩者方向相反：

框架的變換（向前）： A^T_B 描述了如何從框架 {A} 移動到框架 {B}——先平移、再旋轉，A 走到 B。

座標的變換（向後）： A^T_B 把在 {B} 中的座標轉換成 {A} 中的座標，即 $A^P = A^T_B \cdot B^P$ 。方向是 B 到 A。

兩者方向相反並非矛盾，而是數學上的必然：框架與座標是**對偶**的關係。想像你把座標系往右移動 3 公尺，那麼原本靜止不動的物體，在新座標系中的座標就會往左變 3 公尺。框架向右，座標向左——這就是「框架變換協變、座標變換逆變」的直觀說法。

實務上的建議是：永遠使用上下標記號（ A^T_B ），並嚴格遵守消去律。只要式子的上下標能正確消去，方向就不會錯，不需要每次都在腦中重新推理。

8.8 齊次變換的反矩陣

8.8.1 不要用一般的反矩陣公式

要求 B^T_A （{A} 相對於 {B}），可以對 A^T_B 求反矩陣。但千萬不要用一般的 4×4 矩陣求逆演算法——那需要大量運算且有數值誤差。利用齊次變換的結構，可以寫出封閉形式的解：

$$A^T_B^{-1} = B^T_A = \begin{bmatrix} {}^A R_B^T & -{}^A R_B^T \cdot {}^A P_{B_o.r}^g \\ 0 & 1 \end{bmatrix}$$

旋轉部分取轉置（利用正交性），平移部分則是「先反向旋轉、再取負號」。

8.8.2 為什麼平移部分不是直接取負

初學者常誤以為反矩陣的平移部分就是 $-p$ 。這是錯的，正確答案是 $-R^T \cdot p$ 。可以這樣理解：

原本的變換是「先旋轉、再平移」。要反過來，必須「先反平移、再反旋轉」——順序也要顛倒。而在新（旋轉後的）座標系中表達那個反向平移，就必須先用 R^T 把它轉換過來，因此得到 $-R^T \cdot p$ 。

這個公式的運算成本極低：一次轉置（免費）加上一次 3×3 矩陣與向量的乘法（9 次乘法）。相較於一般 4×4 求逆的數十次運算，效率高出許多，且不會有數值不穩定的問題。這也是機器人系統中座標轉換能夠即時執行的原因之一。

8.9 機器人系統中的座標系

8.9.1 五個標準座標系

一台典型的工業機器人工作站中，至少有五個必須明確定義的座標系。

世界座標系 {W}：整個工作站的絕對參考，通常固定在地面或工作台的某個角落。多台機器人協同作業時，這是它們共同的語言。

基座座標系 {B}：固定在機器人底座上。它與世界座標系的關係（ ${}^W T_B$ ）在安裝完成後就固定了，由現場測量或校正得到。

手腕 / 法蘭座標系 {F}：位於機器人第六軸的法蘭面上。從 {B} 到 {F} 的變換，正是正向運動學要計算的（第 9 章），它隨關節角度而變。

工具座標系 {T}：定義在末端執行器的作用點上（例如夾爪的中心、焊槍的槍尖）。從 {F} 到 {T} 的變換由工具的幾何決定，稱為工具中心點（TCP）設定。這是實務上最常需要重新設定的參數——換一支夾具就要重設一次。

工件座標系 {O}：定義在工件或治具上。程式中的路徑通常以工件座標系描述，這樣當工件位置移動時，只需更新一個變換矩陣，所有路徑點自動跟著移動，不必逐點重教。這是離線編程（第 30 章）的基礎。

$${}^W T_T = {}^W T_B \cdot {}^B T_F(q) \cdot {}^F T_T$$

工具在世界座標系中的位姿。中間那一項是關節角度 q 的函數（正向運動學），其餘兩項在標定後固定。

8.9.2 轉換樹與 ROS 2 的 TF

當座標系數量增加（加上相機、光達、多個工件），手動管理所有變換會變得極為繁瑣。ROS 2 的 TF 系統提供了系統化的解法：把所有座標系組織成一棵**樹**，每個節點是一個座標系，每條邊是一個變換。

樹狀結構的關鍵性質是：任兩個座標系之間，只有唯一一條路徑。因此系統可以自動計算任

意兩個座標系之間的變換——只要沿著樹走一趟，把路上的變換連乘（往上走時取反矩陣）即可。使用者只需要問「相機看到的這個點，在機器人基座座標系中是哪裡？」系統就會自動找出路徑並完成計算。

TF 系統還處理了一個實務上極重要的問題：**時間**。感測器資料有延遲，相機在時刻 t 拍到的影像，對應的機器人姿態是 t 時刻的而非現在的。TF 會儲存變換的歷史紀錄，讓你能查詢「在時刻 t 時，這兩個座標系的關係是什麼」。這對視覺伺服與 SLAM 至關重要，第 29 章會詳細說明。

圖 8-4 機器人工作站的轉換樹

建議配圖：樹狀圖。根節點為 *world*；其下分支到 *robot_base*（固定變換，安裝時標定）與 *table*（工作台）。*robot_base* 下依序串接 *link1* ~ *link6*（各由關節角度決定，標註「隨 q 變化」），*link6* 下接 *tool0* 與 *tcp*（工具中心點，標註「換夾具需重設」）。*table* 下接 *workpiece* 與 *camera_link*，*camera_link* 下接 *camera_optical_frame*。每條邊標註變換名稱與是否為固定變換。

8.10 表示法之間的轉換

8.10.1 旋轉矩陣轉歐拉角

實務上經常需要在表示法之間轉換。由旋轉矩陣求 RPY 角（Z-Y-X 慣例）的公式為：

$$\begin{aligned}\beta &= \text{atan2}(-r_{31}, \sqrt{r_{11}^2 + r_{21}^2}) \\ \alpha &= \text{atan2}(r_{21}/\cos \beta, r_{11}/\cos \beta) \\ \gamma &= \text{atan2}(r_{32}/\cos \beta, r_{33}/\cos \beta)\end{aligned}$$

r_{ij} 表示旋轉矩陣的第 i 列第 j 行元素。務必使用 *atan2* 以正確處理象限。

注意這組公式在 $\cos \beta = 0$ （即 $\beta = \pm 90^\circ$ ）時會除以零——這正是萬向節鎖在數值上的表現。程式必須特別處理這個情形：通常的作法是偵測到 $|r_{31}|$ 接近 1 時，把 γ 設為 0，並將剩餘的旋轉全部歸給 α 。

8.10.2 旋轉矩陣轉四元數

由旋轉矩陣求四元數的基本關係為：

$$\begin{aligned}q_0 &= \frac{1}{2}\sqrt{1 + r_{11} + r_{22} + r_{33}} \\ q_1 &= (r_{32} - r_{23})/(4q_0), \quad q_2 = (r_{13} - r_{31})/(4q_0), \quad q_3 = (r_{21} - r_{12})/(4q_0)\end{aligned}$$

當 q_0 接近 0（旋轉角接近 180° ）時，此式會數值不穩定。

穩健的實作（Shepperd 方法）會先比較矩陣的跡（trace）與三個對角元素，選出四個分量中絕對值最大的那一個先計算，再由它推出其餘三個。這避免了除以極小數的問題。實務上直接使用成熟的函式庫（如 SciPy 的 Rotation 類別）即可，不需自行實作。

8.10.3 旋轉矩陣轉軸角

由旋轉矩陣求旋轉角與旋轉軸：

$$\theta = \arccos((\text{tr}(\mathbf{R}) - 1)/2), \quad \text{其中 } \text{tr}(\mathbf{R}) = r_{11} + r_{22} + r_{33}$$

旋轉角由矩陣的跡直接求得。

$$\mathbf{k} = 1/(2 \sin \theta) \cdot [r_{32} - r_{23}, \quad r_{13} - r_{31}, \quad r_{21} - r_{12}]^T$$

旋轉軸。當 $\sin \theta = 0$ ($\theta = 0$ 或 180°) 時失效，需特別處理。

這組公式有一個實用的副產品：它提供了衡量兩個姿態差異的方法。若要知道姿態 $\mathbf{R}_1$ 與 $\mathbf{R}_2$ 差多少，可計算 $\mathbf{R}_{\text{error}} = \mathbf{R}_1^T \cdot \mathbf{R}_2$ ，再取其旋轉角 θ 作為「姿態誤差的大小」。這個純量在第 10 章的逆運動學迭代與第 15 章的軌跡誤差評估中都會用到。

表 8-3 表示法轉換的注意事項

轉換方向	關鍵公式	數值陷阱	建議作法
RPY $\rightarrow$ 矩陣	$\mathbf{R}_z \cdot \mathbf{R}_y \cdot \mathbf{R}_x$ 連乘	無	直接計算
矩陣 $\rightarrow$ RPY	atan2 組合	$\beta = \pm 90^\circ$ 時除以零	偵測奇異點並特別處理
軸角 $\rightarrow$ 矩陣	Rodrigues 公式	$\theta = 0$ 時軸無定義	θ 極小時回傳單位矩陣
矩陣 $\rightarrow$ 軸角	由 $\text{tr}(\mathbf{R})$ 求 θ	$\theta = 0$ 或 180° 時軸不穩定	θ 接近 180° 時改用四元數路徑
四元數 $\rightarrow$ 矩陣	二次式展開	無 (最穩健)	直接計算
矩陣 $\rightarrow$ 四元數	由 $\text{tr}(\mathbf{R})$ 求 q_0	q_0 接近 0 時不穩定	採 Shepperd 方法選最大分量

8.11 常見錯誤與除錯技巧

8.11.1 三個最常見的錯誤來源

錯誤一：慣例不一致。這是實務上最大的陷阱。不同的軟體、廠商與文獻，可能使用不同的歐拉角軸序、不同的四元數分量順序（有些把純量部放第一位 $[w, x, y, z]$ ，有些放最後 $[x, y, z, w]$ ）、甚至不同的角度單位（度或弧度）。整合兩個系統時，若沒有明確確認慣例，會得到看似合理卻完全錯誤的結果。**建議：**任何介面文件的第一頁就寫清楚慣例。

錯誤二：上下標搞反。把 $\mathbf{A}_T \mathbf{B}$ 誤用為 $\mathbf{B}_T \mathbf{A}$ ，是初學者最常見的計算錯誤。症狀是機器人往完全相反或垂直的方向移動。**建議：**嚴格使用上下標記號並檢查消去律。

錯誤三：相乘順序顛倒。把左乘寫成右乘，或把 $\mathbf{R}_z \cdot \mathbf{R}_y \cdot \mathbf{R}_x$ 寫成 $\mathbf{R}_x \cdot \mathbf{R}_y \cdot \mathbf{R}_z$ 。症狀是小角

度時看起來正確（因為小角度旋轉近似可交換），大角度時完全錯亂——這使得它特別難以發現。
建議：用大角度（如 90 度）測試，錯誤會立刻放大顯現。

8.11.2 四個實用的除錯技巧

技巧一：用單位測試驗證恆等式。任何座標轉換程式碼寫完後，立刻驗證三件事： $T \cdot T^{-1} = I$ 、旋轉矩陣的正交性、以及「轉過去再轉回來」是否回到原點。這三個測試能抓出絕大多數的錯誤。

技巧二：用極端值測試。輸入 0 度應得到單位矩陣、輸入 90 度應得到座標軸的置換、輸入 180 度應得到明顯的翻轉。這些結果可以用心算驗證。

技巧三：視覺化。用 RViz、Matplotlib 或任何 3D 繪圖工具，把座標系畫出來。人眼對「座標軸指向奇怪的方向」極為敏感，一眼就能看出矩陣錯誤——這比盯著九個數字有效得多。

技巧四：檢查行列式。若某個「旋轉矩陣」的行列式是 -1 ，代表它包含鏡射，通常是因為某一軸的方向弄反了（例如誤用了左手座標系）。

◎ 一個真實的教訓

某團隊整合視覺系統與機器手臂時，發現夾取位置總是在 Y 方向偏移。檢查了兩週的相機校正參數都找不出問題，最後發現：相機驅動輸出的四元數是 $[x, y, z, w]$ 順序，而機器人 API 期待的是 $[w, x, y, z]$ 。四個數字順序錯了一位，造成的旋轉錯誤剛好在某些姿態下不明顯。這個故事的教訓是：先驗證慣例，再檢查數值。

8.12 由座標系邁向運動學

8.12.1 本章建立的工具

回顧本章，我們建立了一整套描述空間的語言：座標系與位置向量描述「在哪裡」；旋轉矩陣、歐拉角、軸角與四元數描述「朝向哪裡」；齊次變換矩陣把兩者合而為一；而變換的連乘與反矩陣，讓我們能在任意多個座標系之間自由換算。

這套工具的威力在於它的**通用性**。同一個 4×4 矩陣，可以描述機器手臂的一節連桿、相機與機器人的相對安裝、工件在輸送帶上的位置、甚至是無人機在世界地圖中的位姿。整本書後續的所有幾何運算，都建立在這個基礎之上。

8.12.2 接下來：讓變換動起來

本章的所有變換都是靜態的——描述某一瞬間的空間關係。但機器人的本質是運動，因此接下來三章要讓這些變換動起來。

第 9 章（正向運動學）：把每一個關節角度對應到一個變換矩陣，將它們連乘起來，就能算出「給定所有關節角度，末端執行器在空間中的哪裡」。第 8.7.1 節那條 ${}^0T_6 = {}^0T_1 \cdot {}^1T_2 \cdots {}^5T_6$ 的式子，將在第 9 章被完整展開。

第 10 章（逆向運動學）：反過來問「要讓末端到達指定姿，各關節該轉幾度」。這是機器人實際作業時真正需要回答的問題，也困難得多——可能無解、可能有多組解。

第 11 章（雅可比矩陣）：由位置關係推進到速度關係。當關節以某個角速度轉動時，末端的線速度與角速度是多少？答案是一個矩陣，而它也將揭示奇異點的數學本質（第 12 章）。

換句話說：第 8 章教你描述空間，第 9 至 12 章教你計算運動。

完整計算例題

例題 8-1 書本旋轉問題

題目：一本書平放於桌面，其座標系 {B} 與世界座標系 {W} 完全重合。現將書向右平移 1 單位，接著繞原點（世界座標系的 Z 軸）逆時針旋轉 45 度。（a）求 WT_B 。（b）書上有一點在書的座標系中為 ${}^BP = [1, 1, 0]^T$ ，求其在世界座標系中的座標。（c）若世界座標系中有一點 ${}^WQ = [0, 1, 0]^T$ ，求其在書的座標系中的座標。

解（a）：注意題目說「繞世界座標系的 Z 軸旋轉」——這是相對於固定座標系，因此左乘（8.7.3 節）。先平移的變換為 T_{trans} （平移 $[1,0,0]$ ），再繞世界 Z 軸旋轉的變換為 T_{rot} ：

$${}^WT_B = T_{rot} \cdot T_{trans}$$

左乘：後執行且相對於固定座標系的變換寫在左邊。

以 $\cos 45^\circ = \sin 45^\circ = 0.7071$ 代入計算：

$${}^WT_B = \begin{bmatrix} 0.7071 & -0.7071 & 0 & 0.7071 \\ 0.7071 & 0.7071 & 0 & 0.7071 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

旋轉部分為繞 Z 軸 45 度；平移部分為原平移向量 $[1,0,0]$ 經旋轉後變成 $[0.7071, 0.7071, 0]$ 。

解（b）：直接代入 ${}^WP = {}^WT_B \cdot {}^BP$ 。以齊次座標 ${}^BP = [1,1,0,1]^T$ ：

$$\begin{aligned} {}^W P &= [0.7071(1) - 0.7071(1) + 0.7071, \quad 0.7071(1) + 0.7071(1) + 0.7071, \quad 0]^T \\ &= [0.7071, \quad 2.1213, \quad 0]^T \end{aligned}$$

書上該點在世界座標系中的位置。

解 (c)：需要反向轉換，用 8.8.1 節的公式求 ${}^B T_W = {}^W T_B^{-1}$ 。旋轉部分取轉置（即繞 Z 軸 -45 度），平移部分為 $-R^T \cdot p$ ：

$$-R^T \cdot p = -\begin{bmatrix} 0.7071 & 0.7071 \\ -0.7071 & 0.7071 \end{bmatrix} \begin{bmatrix} 0.7071 \\ 0 \end{bmatrix} = -\begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

平移部分為 $[-1, 0, 0]^T$ ，符合直覺：從書的角度看，世界原點在它的左後方。

代入 ${}^B Q = {}^B T_W \cdot {}^W Q$ ，其中 ${}^W Q = [0, 1, 0, 1]^T$ ：

$$\begin{aligned} {}^B Q &= [0.7071(0) + 0.7071(1) - 1, \quad -0.7071(0) + 0.7071(1) + 0, \quad 0]^T \\ &= [-0.2929, \quad 0.7071, \quad 0]^T \end{aligned}$$

世界座標中的該點，在書的座標系中的位置。

驗算技巧：把 (b) 的答案 ${}^W P$ 代回 ${}^B T_W$ 計算，應該要得回 $[1, 1, 0]^T$ 。這種「來回轉換」的驗算能抓出絕大多數的計算錯誤。

例題 8-2 萬向節鎖的數值驗證

題目：以 Z-Y-X 的 RPY 慣例，已知 α （偏航）= 30° 、 β （俯仰）= 90° 、 γ （滾轉）= 20° 。

(a) 求旋轉矩陣。(b) 證明此時 α 與 γ 只以其差值影響結果。(c) 說明由此矩陣反求 RPY 角時會遇到什麼問題。

解 (a)：代入 $R = R_z(30^\circ) \cdot R_y(90^\circ) \cdot R_x(20^\circ)$ 。由於 $\beta = 90^\circ$ ， $\cos \beta = 0$ 、 $\sin \beta = 1$ ，計算後可得矩陣的第一行為 $[0, 0, -1]^T$ 。展開其餘元素會發現，所有元素都只包含 $\sin(\alpha - \gamma)$ 與 $\cos(\alpha - \gamma)$ 的組合：

$$\text{當 } \beta = 90^\circ : R = \begin{bmatrix} 0 & \sin(\alpha - \gamma) & \cos(\alpha - \gamma) \\ 0 & \cos(\alpha - \gamma) & -\sin(\alpha - \gamma) \\ -1 & 0 & 0 \end{bmatrix}$$

α 與 γ 僅以差值 $(\alpha - \gamma)$ 出現，兩個自由度退化為一個。

解 (b)：本例中 $\alpha - \gamma = 30^\circ - 20^\circ = 10^\circ$ 。若改為 $\alpha = 50^\circ$ 、 $\gamma = 40^\circ$ （差值同為 10° ），會得到完全相同的矩陣。這證明了無限多組 (α, γ) 對應到同一個姿態。

解 (c)：由此矩陣反求 RPY 時，8.10.1 節的公式中 $\cos \beta = 0$ ， α 與 γ 的計算都會除以零。程式必須偵測此情形：當 $|r_{31}|$ 接近 1 時，慣例上把 γ 設為 0，並將全部的旋轉歸給 α （此例中 $\alpha = 10^\circ$ 、 $\gamma = 0^\circ$ ）。雖然數值不同，但代表的姿態完全相同。

工程啟示：這正是無人機、太空船與人形機器人的姿態控制堅持使用四元數的原因。任何可能經歷大角度姿態變化的系統，都不應在內部使用歐拉角儲存姿態。

例題 8-3 工具中心點與工件座標系

題目：某機器人基座在世界座標系中的位置為 $[2, 1, 0]^T$ ，且繞 Z 軸旋轉了 90° 。經正向運動學計算，法蘭面相對於基座的變換 ${}^B T_F$ 之平移部分為 $[0.5, 0, 0.8]^T$ 、旋轉為單位矩陣。所裝夾具的 TCP 相對於法蘭面沿其 Z 軸前伸 0.15 公尺（無旋轉）。（a）求 TCP 在世界座標系中的位置。（b）若工件座標系原點在世界座標的 $[3, 2, 0.5]^T$ 且無旋轉，求 TCP 在工件座標系中的位置。

解（a）：依 8.9.1 節的連鎖式 ${}^W T_T = {}^W T_B \cdot {}^B T_F \cdot {}^F T_T$ 。

先求 ${}^B T_T = {}^B T_F \cdot {}^F T_T$ 。由於 ${}^B T_F$ 的旋轉為單位矩陣，TCP 相對基座的平移即為 $[0.5, 0, 0.8]^T + [0, 0, 0.15]^T = [0.5, 0, 0.95]^T$ 。

再左乘 ${}^W T_B$ 。基座繞 Z 軸轉 90° ，其旋轉矩陣把 $[x, y, z]$ 映射為 $[-y, x, z]$ ，因此：

$$\begin{aligned} {}^W P_T &= R_z(90^\circ) \cdot [0.5, 0, 0.95]^T + [2, 1, 0]^T \\ &= [0, 0.5, 0.95]^T + [2, 1, 0]^T = [2, 1.5, 0.95]^T \end{aligned}$$

TCP 在世界座標系中的位置。注意基座的旋轉使得「機器人自己的前方 0.5 公尺」變成了世界座標的 Y 方向。

解（b）：工件座標系無旋轉，因此 ${}^O T_W$ 的旋轉為單位矩陣、平移為 $-[3, 2, 0.5]^T$ ：

$${}^O P_T = {}^W P_T - {}^W P_{O_{or}^g} = [2, 1.5, 0.95]^T - [3, 2, 0.5]^T = [-1, -0.5, 0.45]^T$$

TCP 在工件座標系中的位置。

實務意義：程式若以工件座標系撰寫，當工件被移動到新位置時，只需更新 ${}^W T_O$ 一個矩陣，所有路徑點自動跟隨，不必逐點重教。這正是離線編程與快速換線的基礎（第 30 章）。

程式範例：座標轉換工具箱

以下 Python 程式實作本章的核心運算：基本旋轉矩陣、RPY 與矩陣互轉、齊次變換的建構、快速求逆、以及變換的連乘。程式最後以例題 8-1 與 8-2 驗證，並包含 8.11.2 節建議的單位測試。

```
import numpy as np

# ----- 基本旋轉矩陣 -----
def rot_x(t):
    c, s = np.cos(t), np.sin(t)
```

```

    return np.array([[1, 0, 0], [0, c, -s], [0, s, c]])

def rot_y(t):
    c, s = np.cos(t), np.sin(t)
    return np.array([[c, 0, s], [0, 1, 0], [-s, 0, c]])

def rot_z(t):
    c, s = np.cos(t), np.sin(t)
    return np.array([[c, -s, 0], [s, c, 0], [0, 0, 1]])

# ----- RPY ( Z-Y-X 固定軸 ) 與矩陣互轉 -----
def rpy_to_rot(roll, pitch, yaw):
    """先繞 X(roll)、再 Y(pitch)、最後 Z(yaw)；先做的寫右邊"""
    return rot_z(yaw) @ rot_y(pitch) @ rot_x(roll)

def rot_to_rpy(R):
    """回傳 (roll, pitch, yaw) · 含萬向節鎖處理"""
    if abs(R[2, 0]) > 0.99999:
        # 萬向節鎖：pitch = ±90°
        pitch = -np.sign(R[2, 0]) * np.pi / 2
        roll = 0.0
        yaw = np.arctan2(-R[0, 1], R[1, 1])
        print(" [警告] 偵測到萬向節鎖 · roll 已設為 0")
        return roll, pitch, yaw
    pitch = np.arctan2(-R[2, 0], np.hypot(R[0, 0], R[1, 0]))
    yaw = np.arctan2(R[1, 0], R[0, 0])
    roll = np.arctan2(R[2, 1], R[2, 2])
    return roll, pitch, yaw

# ----- 齊次變換 -----
def make_T(R=np.eye(3), p=np.zeros(3)):
    T = np.eye(4)
    T[:3, :3] = R
    T[:3, 3] = p
    return T

def inv_T(T):
    """利用結構快速求逆：R 取轉置、平移為 -R^T·p ( 8.8.1 節 ) """
    R, p = T[:3, :3], T[:3, 3]
    return make_T(R.T, -R.T @ p)

def apply_T(T, point):
    """把 3D 點以齊次座標作轉換"""
    return (T @ np.append(point, 1.0))[:3]

# ----- 驗證一：例題 8-1 -----
print("=== 例題 8-1：書本旋轉 ===")
T_trans = make_T(p=np.array([1, 0, 0]))
T_rot = make_T(R=rot_z(np.deg2rad(45)))
T_WB = T_rot @ T_trans

```

```

P_B = np.array([1, 1, 0])
print(f"(b) 書上點在世界座標: {apply_T(T_WB, P_B).round(4)}")

Q_W = np.array([0, 1, 0])
T_BW = inv_T(T_WB)
print(f"(c) 世界點在書座標: {apply_T(T_BW, Q_W).round(4)}")
print(f"    來回驗算 (應回到 [1,1,0]): "
      f"{apply_T(T_BW, apply_T(T_WB, P_B)).round(4)}")

# ----- 驗證二：例題 8-2 萬向節鎖 -----
print("\n=== 例題 8-2：萬向節鎖 ===")
R1 = rpy_to_rot(np.deg2rad(20), np.deg2rad(90), np.deg2rad(30))
R2 = rpy_to_rot(np.deg2rad(40), np.deg2rad(90), np.deg2rad(50))
print(f"(30°, 90°, 20°) 與 (50°, 90°, 40°) 是否相同矩陣: "
      f"{np.allclose(R1, R2)} ← 差值同為 10°")
r, p_, y = rot_to_rpy(R1)
print(f"反求 RPY: roll={np.rad2deg(r):.1f}° "
      f"pitch={np.rad2deg(p_):.1f}° yaw={np.rad2deg(y):.1f}°")

# ----- 單位測試 (8.11.2 節建議) -----
print("\n=== 單位測試 ===")
R = rpy_to_rot(0.3, -0.5, 1.2)
print(f"正交性  $R^T \cdot R = I$  : {np.allclose(R.T @ R, np.eye(3))}")
print(f"行列式  $\det(R) = 1$ : {np.isclose(np.linalg.det(R), 1.0)}")
T = make_T(R, np.array([1.5, -2.0, 0.7]))
print(f"求逆  $T \cdot T^{-1} = I$  : {np.allclose(T @ inv_T(T), np.eye(4))}")
print(f"RPY 來回轉換一致: "
      f"{np.allclose(rpy_to_rot(*rot_to_rpy(R)), R)}")

```

請試著修改：(一) 把例題 8-1 的左乘改成右乘 ($T_{\text{trans}} @ T_{\text{rot}}$)，觀察結果如何改變，並解釋其物理意義的差異。(二) 把 pitch 從 90° 改成 89.9° ，觀察 rot_to_rpy 的輸出是否劇烈變化——這就是接近奇異點時的數值不穩定。

實作：座標系視覺化與手動驗證

本章的實作分為紙上與電腦兩部分，目的是建立空間直覺——這是純看公式無法獲得的。

第一部分：紙上實驗 (不需電腦)

1. 取一本書，在封面上用便利貼標出三個軸：長邊為 X、短邊為 Y、垂直書面向上為 Z (右手系)。

2. 把書平放，此時書的座標系與桌面座標系重合。先繞 X 軸翻轉 90° 、再繞 Y 軸翻轉 90° ，記錄最終三軸的指向。
3. 把書放回原位，改為先繞 Y 軸、再繞 X 軸，記錄結果。比較兩者——親身驗證旋轉的不可交換性。
4. 手算這兩種順序的旋轉矩陣 ($R_y \cdot R_x$ 與 $R_x \cdot R_y$)，確認計算結果與實物觀察一致。

第二部分：電腦視覺化

1. 使用 Matplotlib 的 3D 繪圖，撰寫一個函式 `draw_frame(T, label)`，能將任一齊次變換矩陣所代表的座標系以三支彩色箭頭 (X 紅、Y 綠、Z 藍) 畫出。
2. 畫出世界座標系與例題 8-3 中的基座、法蘭、TCP、工件四個座標系，確認它們的相對位置符合直覺。
3. 製作動畫：讓某個座標系繞世界 Z 軸連續旋轉，同時另一個座標系繞自身 Z 軸旋轉，直觀比較左乘與右乘的差異。
4. (進階) 實作 SLERP 四元數內插與歐拉角線性內插，各自產生 20 個中間姿態並畫出，比較兩者路徑的平滑程度——你會清楚看到歐拉角內插的扭曲。
5. (若已安裝 ROS 2) 執行 `rviz2` 並發布幾個 TF 座標系，觀察 TF 樹如何自動計算任兩個座標系之間的關係。

國中小也看得懂：轉的順序不一樣，結果就不一樣

玩過「機器人指令遊戲」嗎？你對同學說「向前走三步，然後右轉」，跟說「右轉，然後向前走三步」，他會走到完全不同的地方對不對？這就是機器人數學裡最重要的一件事：**順序很重要！**

再試一個更好玩的實驗。拿一本課本平放在桌上，記住封面朝上、書名朝著你。第一次：先把書往前翻 90° (讓封面朝向你)，再往右翻 90° 。記住現在書長什麼樣子。第二次：把書放回原位，這次先往右翻 90° ，再往前翻 90° 。是不是完全不一樣？明明轉的角度都一樣，順序換了，結果就不同——這件事連大人都常常搞錯喔！

那機器人怎麼知道東西在哪裡呢？它需要一個「參考點」。就像你說「橡皮擦在鉛筆盒裡」，機器人要說「橡皮擦在我的攝影機前面 30 公分、右邊 10 公分」。如果換一個位置看，同樣一塊

橡皮擦，數字就完全不一樣了。所以機器人身上會有好多組「參考點」——底座一組、手臂一組、攝影機一組——它必須不停地把這些數字換來換去，才知道手該伸到哪裡。這種換算，就是這一章大哥哥大姐姐們在學的東西！

叫獸提醒與常見錯誤

- 誤區一：以為旋轉可以像位移一樣交換順序。 $R_1 \cdot R_2 \neq R_2 \cdot R_1$ ，先轉 X 再轉 Y 與先轉 Y 再轉 X 結果完全不同。
- 誤區二：不標明座標系。同一個點在不同座標系中數值完全不同，所有向量都應標註上標（如 ${}^A P$ ）。
- 誤區三：上下標搞反，把 ${}^A T_B$ 誤用為 ${}^B T_A$ 。用消去律檢查：相鄰的上下標應能兩兩抵消。
- 誤區四：左乘右乘混淆。相對於自身座標系的變換用右乘（動軸），相對於固定座標系的用左乘（定軸）。
- 誤區五：以為 R_y 的負號位置與 R_x 、 R_z 相同。 R_y 的負號在左下角，這是最常抄錯的一個公式。
- 誤區六：在內部儲存中使用歐拉角。任何可能經歷大角度變化的系統都應改用四元數，避免萬向節鎖。
- 誤區七：用一般矩陣求逆算齊次變換的反矩陣。應使用 8.8.1 節的封閉解，運算量低且數值穩定。
- 誤區八：反矩陣的平移部分直接取負。正確為 $-R^T \cdot p$ ，不是 $-p$ 。
- 誤區九：四元數分量順序不一致。有些系統用 $[w, x, y, z]$ 、有些用 $[x, y, z, w]$ ，整合時必須確認。
- 誤區十：對旋轉矩陣的元素做線性內插。結果不再是旋轉矩陣；姿態內插應使用四元數 SLERP。
- 誤區十一：忽略數值誤差累積。連續多次矩陣相乘後正交性會逐漸喪失，應定期正交化修正。
- 誤區十二：程式中用 atan 而非 atan2 。前者無法判斷象限，會導致角度計算在特定區域錯誤。

課堂討論

1. 阿波羅十三號的地面控制中心花數小時算出「繞開萬向節鎖」的旋轉路徑。若當時已有四元數技術並應用於導航平台，這個問題還會存在嗎？請區分「數學表示法的奇異點」與「機械結構的奇異點」。
2. 旋轉矩陣用九個數描述三個自由度，冗餘度很高；四元數用四個數；歐拉角只用三個。為什麼工程上不總是選最精簡的？請從奇異點、計算效率與直觀性三方面討論。
3. 為什麼「任何用三個數描述三維旋轉的方法都必然有奇異點」？試以「地球儀上的經緯度在南北極會發生什麼」作類比說明。
4. 工件座標系的設計讓「工件移動時不必重教路徑」。請設計一個實際情境說明這個好處，並討論若工件不只平移還有旋轉時，需要如何處理。
5. TF 系統儲存變換的「歷史紀錄」以處理感測器延遲。若沒有這個機制，視覺伺服會發生什麼問題？請以「相機在 t 時刻拍照、影像處理耗時 100 毫秒、手臂持續運動」的情境具體說明。

章末摘要

- 座標系由原點與三個互相垂直的單位軸構成；機器人學統一使用右手座標系與右手定則。
- 旋轉矩陣 A_R_B 的每一行，是 $\{B\}$ 的座標軸在 $\{A\}$ 中的方向；它有三種詮釋：座標軸描述、座標轉換、旋轉算子。
- 旋轉矩陣為正交矩陣 ($R^{-1} = R^T$) 且 $\det(R) = +1$ ，構成 $SO(3)$ 群；轉置即求逆的特性使反向轉換幾乎免費。
- 歐拉角 / RPY 用三個角度描述姿態，直觀但存在萬向節鎖——中間旋轉達 $\pm 90^\circ$ 時失去一個自由度。
- 任何以三個數描述三維旋轉的方法都必然有奇異點，這是數學的必然而非工程的缺陷。
- 軸角表示法源自歐拉旋轉定理（任何旋轉可化為繞單一軸的一次旋轉），由 Rodrigues 公式轉為矩陣。
- 單位四元數以半角形式編碼軸角，無奇異點、計算效率高、可用 SLERP 平滑內插，但完全不直觀。
- 實務策略：內部用四元數儲存計算、介面用歐拉角顯示、幾何推導用旋轉矩陣。

- 齊次座標把三維點升為四維（末位補 1），使平移能以矩陣乘法表達；齊次變換矩陣 4×4 統合旋轉與平移，構成 $SE(3)$ 。
- 變換連乘遵守消去律 $AT_C = AT_B \cdot BT_C$ ，這是正向運動學 ${}^0T_6 = {}^0T_1 \dots {}^5T_6$ 的基礎。
- 右乘代表相對自身座標系（動軸）、左乘代表相對固定座標系（定軸）；矩陣乘法不可交換，順序至關重要。
- 同一矩陣有「框架向前變換」與「座標向後變換」兩種對偶讀法；嚴格使用上下標記號可避免混淆。
- 齊次變換的反矩陣有封閉解：旋轉取轉置、平移為 $-R^T \cdot p$ ，切勿使用一般矩陣求逆。
- 機器人工作站的五個標準座標系：世界、基座、法蘭、工具（TCP）、工件；ROS 2 的 TF 以樹狀結構自動管理並處理時間戳。

觀念題與計算題

觀念題

1. 說明右手座標系與右手定則的內容，並解釋為何團隊必須統一慣例。
2. 旋轉矩陣的每一行代表什麼幾何意義？請由此說明為何它必為正交矩陣。
3. 為什麼 $R^{-1} = R^T$ 這條性質在工程上特別有價值？
4. 若某矩陣的行列式為 -1 ，代表什麼？在物理上可能對應到什麼錯誤？
5. 說明旋轉矩陣的三種詮釋，並區分「主動旋轉」與「被動旋轉」。
6. 解釋內旋與外旋的差異，並說明「繞固定軸 X-Y-Z」為何等價於「繞動軸 Z-Y-X」。
7. 萬向節鎖如何發生？為何說它是數學的必然而非工程缺陷？
8. 敘述歐拉旋轉定理，並說明軸角表示法的兩個奇異點各在何處。
9. 四元數為何使用半角？由此導致的「雙重覆蓋」現象在實作上需注意什麼？
10. 列舉四元數的三大優勢與一項致命缺點，並說明實務上的因應策略。
11. 齊次座標為何要在末位補 1？多出來的那一列有哪三個作用？
12. 說明變換連乘的消去律，並解釋為何它是檢查式子正確性的最快方法。
13. 區分左乘與右乘的物理意義，各舉一個機器人操作的實例。
14. 為何齊次變換反矩陣的平移部分是 $-R^T \cdot p$ 而非 $-p$ ？請說明其物理推理。

15. 列出機器人工作站的五個標準座標系及其用途，並說明以工件座標系撰寫程式的好處。

計算題

- 座標系 {B} 相對於 {A} 先繞 A 的 Z 軸旋轉 90° ，再沿 A 的 X 軸平移 3 單位。(a) 求 A_T_B 。(b) 點 $BP = [1, 2, 0]^T$ 在 {A} 中的座標為何？(c) 求 B_T_A 並驗證 $A_T_B \cdot B_T_A = I$ 。
- 已知 RPY 角為 $roll = 30^\circ$ 、 $pitch = 45^\circ$ 、 $yaw = 60^\circ$ (Z-Y-X 固定軸慣例)。(a) 寫出對應的旋轉矩陣 (可保留至小數點後三位)。(b) 求其行列式與驗證正交性。(c) 由此矩陣反求 RPY 角，確認與原值一致。
- 某旋轉繞單位軸 $k = [0, 0, 1]^T$ 轉 60° 。(a) 以 Rodrigues 公式求旋轉矩陣。(b) 求對應的單位四元數。(c) 驗證此四元數的長度為 1。(d) 求此四元數的共軛，並說明其代表的旋轉。
- 機器人基座在世界座標的位置為 $[1, 2, 0]^T$ 且繞 Z 軸轉 -90° 。正向運動學求得法蘭相對基座的變換：平移 $[0.6, 0.2, 0.9]^T$ 、旋轉為繞基座 Y 軸 90° 。工具相對法蘭沿其 Z 軸前伸 0.2 公尺。(a) 求 W_T_B 、 B_T_F 與 F_T_T 。(b) 求 TCP 在世界座標系中的位姿 (位置與旋轉矩陣)。(c) 若工件座標系位於世界座標 $[2, 3, 0.4]^T$ 且繞 Z 軸轉 45° ，求 TCP 在工件座標系中的位置。

動手做與專題延伸

本章實作：座標系關係表

為你手邊的機器人 (或附錄中的任一台) 建立完整的座標系清單與轉換關係表。這份表格是後續所有幾何計算的基礎文件。

表 8-4 座標系關係表 (前兩列為範例)

座標系	原點位置	軸向定義	父座標系	變換是否固定	取得方式
world	工作台左後角	X 向右、Y 向前、Z 向上	(根)	—	人為定義
base_link	機器人底座中心	X 為機器人正前方	world	固定	安裝後量測或標定
(自行填寫：各連桿、末端工具、相機、工件等)					

專題延伸

- 撰寫一個完整的座標轉換 Python 模組，包含四種表示法之間的所有轉換函式，並為每一個函式撰寫單元測試（驗證恆等式、極端值與來回轉換一致性）。這個模組將在第 9~11 章持續使用。
- 以 Matplotlib 製作互動式的萬向節鎖示範程式：以三個滑桿控制 RPY 角，即時繪出三個旋轉環與物體姿態。當 pitch 接近 90° 時，用紅色標示出兩軸重合的狀態，並顯示「自由度由 3 降為 2」的警告。
- 進階：實作一個簡易的 TF 系統。以字典儲存各座標系的父子關係與變換矩陣，並撰寫 `lookup_transform(from_frame, to_frame)` 函式，能自動在樹中尋找路徑（往上走時取反矩陣）並連乘出結果。以例題 8-3 的四個座標系測試，並與手算結果比對。

參考文獻與延伸閱讀

- Craig, J. J., Introduction to Robotics: Mechanics and Control, 4th ed., Pearson (第 2 章：空間描述與變換，本章的主要參考)。
- Lynch, K. M. and Park, F. C., Modern Robotics: Mechanics, Planning, and Control, Cambridge University Press (第 3 章：剛體運動，含李群觀點)。
- Siciliano, B., Sciavicco, L., Villani, L. and Oriolo, G., Robotics: Modelling, Planning and Control, Springer (第 2 章：運動學基礎)。
- Murray, R. M., Li, Z. and Sastry, S. S., A Mathematical Introduction to Robotic Manipulation, CRC Press ($SO(3)$ 與 $SE(3)$ 的嚴謹數學處理)。
- Kuipers, J. B., Quaternions and Rotation Sequences, Princeton University Press (四元數的經典入門專書)。
- Shoemake, K., "Animating Rotation with Quaternion Curves," SIGGRAPH 1985 (SLERP 內插的原始論文)。
- Shepperd, S. W., "Quaternion from Rotation Matrix," Journal of Guidance and Control, 1978 (穩健的矩陣轉四元數演算法)。
- Diebel, J., "Representing Attitude: Euler Angles, Unit Quaternions, and Rotation Vectors," Stanford University, 2006 (各種姿態表示法的完整整理與轉換公式)。

- Toussaint, M., "3D Geometry Notes," TU Berlin 課程講義 (本書系參考的德式圖解講義) 。
- ROS 2 官方 TF2 教學文件 (docs.ros.org) : 轉換樹的概念、時間戳查詢與實作。

第 9 章

操作臂正向運動學

編著：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

【章首情境圖建議】一支六軸工業機器人的「透視爆炸圖」：機構以半透明呈現，每一個關節處疊印一組彩色連桿座標系 (X 紅、 Y 綠、 Z 藍)，六組座標系之間以發光的箭頭串連成一條由基座通往夾爪的鏈；每個箭頭旁標註對應的變換矩陣符號 0T_1 、 1T_2 5T_6 。畫面下緣以一行公式收尾： ${}^0T_6 = {}^0T_1 \cdot {}^1T_2 \cdot {}^2T_3 \cdot {}^3T_4 \cdot {}^4T_5 \cdot {}^5T_6$ 。標題文字：六次接力，從基座算到指尖。

叫獸開講 「知道每一顆馬達轉幾度，怎麼算出機器手指尖的位置？」

實驗室裡有一支學生自製的三軸機械手臂。叫獸請小翰示範：「請你讓夾爪停在距離基座正前方 35 公分、高度 20 公分的位置。」

小翰坐到電腦前，開始調整三個滑桿。第一軸轉了 30 度，手臂歪向一邊；再調第二軸 45 度，夾爪抬得太高；調第三軸補償，又太靠近了。他來回試了兩分鐘，滑桿調了三十幾次，夾爪終於停在大約對的位置——用尺一量，差了 2.8 公分。

「很好，」叫獸說，「現在請你告訴我，剛才那三個角度分別是多少？」小翰看了看螢幕：「28.4 度、52.1 度、負 71.6 度。」

「那麼，如果我現在把這三個數字報給你，你能不能在不看手臂的情況下，用紙筆算出夾爪會停在哪裡？」小翰愣住了。他能靠試誤把手臂移到想要的位置，卻算不出來為什麼是這三個數字。

叫獸拿起白板筆：「這就是正向運動學要解決的問題——**已知所有關節角度，求末端執行器在空間中的位姿**。聽起來好像沒什麼用，畢竟實務上我們通常是先知道目標位置、再反推角度。但如果你連正向都算不出來，逆向根本無從談起。」

他在白板上畫了一條由三段線構成的鏈：「而且，正向運動學遠不只是『算個位置』。它是機器人的**數位分身**：有了它，你可以在電腦裡模擬手臂的每一個動作而不用真的動它；你可以檢查某個姿態會不會撞到桌子；你可以計算工作空間的邊界；你可以把編碼器讀到的角度，即時換算成螢幕上顯示的座標。沒有正向運動學，機器人就只是一堆各轉各的馬達。」

「更關鍵的是，」叫獸在鏈條的每一段旁邊寫下一個矩陣符號，「上一章我們學了齊次變換，還記得那條消去律嗎？ $AT_C = AT_B \cdot BT_C$ 。今天我們要做的事情，就是把它用六次——把六個關節的變換串起來，從基座一路乘到夾爪。整個正向運動學，其實就是一連串的**矩陣連乘**。」

他轉過身：「難的不是乘法，是**怎麼替每一節連桿建立座標系**。同一支手臂，一百個工程師可能畫出一百種座標系配置，算出來的矩陣長得完全不一樣——但夾爪的位置必須是同一個。所以五十年前有兩位學者提出了一套規則，讓全世界的工程師畫出同一組座標系。這套規則，叫做 DH 參數法。」

本章為什麼重要

正向運動學是機器人學的第一個完整的計算成果。它把第 4 章的機構、第 6 章的編碼器讀值、第 8 章的座標變換全部串聯起來，回答一個核心問題：**這台機器人現在的手，究竟在哪裡？**

它的用途遠比表面看起來廣。在模擬與離線編程中，它讓你能在電腦裡預演整套動作；在碰撞檢測中，它提供每一節連桿的即時位置；在工作空間分析中，它讓你掃描出機器人能到達的範圍；在示教與監控中，它把關節角度即時轉換成操作員看得懂的笛卡兒座標。而最重要的是——第 10 章的逆向運動學、第 11 章的雅可比矩陣、第 13 章的動力學，全部建立在正向運動學的基礎上。

本章也是全書第一次要求讀者親手完成完整的矩陣推導。這個過程可能會有點辛苦，但走過一遍之後，你對機器人的理解會產生質變——從「知道它會動」變成「知道它為什麼動到那裡」。

學習目標

- 能定義正向運動學問題，並說明其在模擬、監控與規劃中的用途。
- 能說明運動鏈的連桿與關節編號慣例。
- 能列出四個 DH 參數並說明各自的幾何意義。
- 能依標準 DH 規則為一支機械手臂建立連桿座標系並填寫參數表。
- 能寫出單一連桿的齊次變換矩陣並解釋其四個組成旋轉與平移。
- 能區分標準 DH 與修正式 DH 的差異，並說明混用的後果。
- 能推導平面 2R 與 3R 手臂的正向運動學封閉解。
- 能建立 SCARA 與六軸手臂的 DH 參數表並計算末端位姿。
- 能以 Python 實作任意串聯手臂的正向運動學。
- 能說明運動學模型的誤差來源與標定的必要性。

先備知識

第 8 章的全部內容，特別是齊次變換矩陣、變換的連乘與消去律、以及基本旋轉矩陣。第 4 章的連桿、關節與運動鏈概念。需要熟練的矩陣乘法與三角函數運算。

本章的推導量較大，建議準備紙筆邊讀邊算。若某個步驟卡住，多半是第 8 章的某個觀念還

不夠熟——回頭複習比硬撐下去有效率得多。

核心名詞中英對照

表 9-1 本章核心名詞

中文	英文	說明
正向運動學	Forward Kinematics, FK	由關節變數求末端位姿的問題
關節變數	Joint Variable	旋轉關節的角度或移動關節的位移
關節空間	Joint Space	以關節變數為座標的空間
任務空間 / 笛卡兒空間	Task Space / Cartesian Space	以末端位姿描述的空間
連桿座標系	Link Frame	固定在每一節連桿上的座標系
Denavit–Hartenberg 參數	DH Parameters	描述相鄰連桿關係的四個參數
連桿長度	Link Length, a	兩相鄰關節軸的公垂線長度
連桿扭轉角	Link Twist, α	兩相鄰關節軸之間的夾角
連桿偏移	Link Offset, d	沿關節軸方向的偏移量
關節角	Joint Angle, θ	繞關節軸的旋轉角
公垂線	Common Normal	與兩條空間直線都垂直的線段
標準 DH	Standard DH	座標系置於連桿遠端的慣例
修正式 DH	Modified DH	座標系置於連桿近端的慣例 (Craig 版本)
工具中心點	Tool Center Point, TCP	末端執行器的作用點
運動學標定	Kinematic Calibration	以量測修正運動學模型參數的程序

9.1 正向運動學問題

9.1.1 問題定義

正向運動學的問題陳述極為簡潔：給定一組關節變數，求末端執行器相對於基座的位姿。

已知： $\mathbf{q} = [q_1, q_2, \dots, q_n]^T$ 求： 0T_n (末端相對基座的齊次變換)

$\mathbf{q}$ 是關節變數向量，旋轉關節用角度、移動關節用位移。 0T_n 是一個 4×4 齊次變換矩陣，同時包含位置與姿態。

這個從關節空間到任務空間的映射，通常寫成一個函數：

$$x = f(q) \quad \text{其中 } x \in \mathbb{R}^6 \text{ (位置 3 + 姿態 3)}, q \in \mathbb{R}^n$$

第 8 章末提到的「運動圖」(kinematic map)就是這個 f 。它是一個非線性函數(因為含三角函數)，但重要的是——它總是有唯一解。

9.1.2 為什麼正向比逆向簡單

正向運動學有一個極為友善的性質：**給定關節角度，末端位姿必定存在且唯一**。這在物理上顯而易見——把每個馬達轉到指定角度，手臂只會擺出一種姿態，不可能同時出現在兩個地方。

這與第 10 章的逆向運動學形成鮮明對比。逆向問題可能無解(目標超出工作空間)、可能有多組解(手肘朝上或朝下)、甚至可能有無限多解(冗餘手臂)。正向運動學的唯一性，使它成為整個運動學體系中最穩固的基石——也是為什麼所有逆運動學的數值方法，內部都要反覆呼叫正向運動學。

9.1.3 五個實際用途

用途一：即時狀態顯示。編碼器每毫秒回報關節角度，經正向運動學換算成笛卡兒座標，讓操作員在示教器上看到「TCP 現在在 $X=520, Y=180, Z=340$ 」。

用途二：模擬與離線編程。在電腦中預演整套動作，驗證軌跡是否合理、節拍是否達標，不需佔用實體設備(第 30 章)。

用途三：碰撞檢測。正向運動學不只算末端，還能算出每一節連桿的位姿。有了它，才能判斷手臂會不會撞到自己、撞到治具或撞到人。

用途四：工作空間分析。掃描所有可能的關節組合，計算對應的末端位置，就能繪出第 4 章討論的工作空間形狀與邊界。

用途五：逆向運動學的基礎。數值型逆運動學(第 10 章)的每一次迭代，都需要用正向運動學計算「目前走到哪裡」，才能算出還差多遠。

圖 9-1 正向與逆向運動學的關係

建議配圖：中央為機器手臂圖示。左側方框「關節空間 $q = [\theta_1, \theta_2, \dots, \theta_n]$ 」，右側方框「任務空間 $x = (\text{位置}, \text{姿態})$ 」。由左至右的粗箭頭標註「正向運動學 $f(q)$ ：唯一解、直接計算」；由右至左的虛線箭頭標註「逆向運動學 $f^{-1}(x)$ ：可能無解 / 多解 / 無限多解」。下方註明：正向是連乘、逆向是解方程。

9.2 運動鏈與編號慣例

9.2.1 連桿與關節的編號

要把手臂拆解成一連串的變換，第一步是建立統一的編號。機器人學的慣例是：

連桿編號：固定不動的基座是連桿 0，往末端依序為連桿 1、2、.....、n。因此一支 n 軸手臂共有 n+1 根連桿（含基座）。

關節編號：連接連桿 i-1 與連桿 i 的關節，編號為關節 i。因此關節 1 連接基座與第一根連桿，關節 n 連接連桿 n-1 與連桿 n。

關節變數：關節 i 的變數記為 q_i 。若為旋轉關節， $q_i = \theta_i$ （角度）；若為移動關節， $q_i = d_i$ （位移）。

這個編號方式的好處是：關節 i 的運動，直接對應到連桿 i 的移動。從基座往末端走，每經過一個關節就前進一個編號。

9.2.2 連桿座標系的目標

接下來要在每一根連桿上「釘」一個座標系 $\{i\}$ ，使得連桿 i 相對於連桿 i-1 的變換 ${}^{i-1}T_i$ 可以只用少數幾個參數描述。理想上，我們希望：

- 每個變換只包含一個變數（該關節的 q_i ），其餘都是機構製造時就固定的常數。
- 參數數目盡可能少，方便建立模型與標定。
- 全世界的工程師依同一套規則，會得到同一組參數，方便交流與交叉驗證。

直覺上，描述兩個空間中的座標系關係需要六個參數（三平移三旋轉）。但 Denavit 與 Hartenberg 在 1955 年提出的巧妙作法，透過對座標系軸向的特殊選擇，把這六個參數壓縮到只需要四個。這就是 DH 參數法。

9.3 Denavit–Hartenberg 參數

9.3.1 核心洞見：沿著公垂線建立座標系

DH 法的關鍵洞見是：空間中任兩條直線（兩個關節軸），必定存在一條**公垂線**（同時垂直於兩者的線段），而且除了兩軸平行的特殊情形外，這條公垂線是唯一的。

若把座標系的軸向與這條公垂線對齊，就能消去兩個自由度，使六個參數降為四個。具體的規則是：

- **Z 軸沿著關節軸**： Z_i 軸與關節 $i+1$ 的轉軸重合（標準 DH）。
- **X 軸沿著公垂線**： X_i 軸沿著 Z_{i-1} 與 Z_i 的公垂線，方向由 Z_{i-1} 指向 Z_i 。
- **Y 軸由右手定則決定**： $Y_i = Z_i \times X_i$ 。

正因為 X 軸被強制與公垂線對齊，才使得「沿 Y 軸的平移」與「繞 Y 軸的旋轉」這兩個自由度自動歸零，四個參數就足夠了。

9.3.2 四個參數的幾何意義

依照上述規則建立座標系後，連桿 i 與 $i-1$ 的關係由四個參數完全決定：

表 9-2 DH 四參數的定義與意義

參數	符號	定義	幾何意義	何時為變數
連桿長度	a_{i-1}	沿 X_{i-1} 由 Z_{i-1} 量到 Z_i 的距離	公垂線的長度，即連桿實際長度	永遠是常數
連桿扭轉角	α_{i-1}	繞 X_{i-1} 由 Z_{i-1} 轉到 Z_i 的角度	相鄰兩關節軸的夾角	永遠是常數
連桿偏移	d_i	沿 Z_i 由 X_{i-1} 量到 X_i 的距離	沿關節軸方向的錯開量	移動關節時為變數
關節角	θ_i	繞 Z_i 由 X_{i-1} 轉到 X_i 的角度	關節的轉動角度	旋轉關節時為變數

記憶方法：**a** 與 **α** 描述「連桿本身」的幾何（兩個關節軸之間的關係，出廠就固定）；**d** 與 **θ** 描述「關節」的狀態（其中一個是變數）。這也是為什麼一個關節只貢獻一個自由度——四個參數中恰好只有一個會變。

圖 9-2 DH 四參數的幾何定義

建議配圖：三維透視圖。畫出兩條不平行、不相交的空間直線代表關節軸 Z_{i-1} 與 Z_i ，以及它們之間的公垂線（標為 X_{i-1} ）。標註四個參數： a_{i-1} （公垂線長度，沿 X_{i-1} ）、 α_{i-1} （兩軸夾角，繞 X_{i-1} ）、 d_i （沿 Z_i 的偏移，由 X_{i-1} 量到 X_i ）、 θ_i （繞 Z_i 的轉角）。用不同顏色區分「常數參數」（ a 、 α ，藍色）與「可變參數」（ d 、 θ ，紅色）。

9.3.3 建立座標系的六個步驟

實務上為一支手臂建立 DH 參數表，可依下列步驟進行：

1. **畫出機構簡圖**：以線段代表連桿、圓柱代表旋轉關節、方塊代表移動關節，標出所有關節軸的方向。

2. 標定所有 Z 軸： Z_i 沿關節 $i+1$ 的軸線（標準 DH）。方向可任選，但一旦選定就不能改。
3. 建立基座座標系 $\{0\}$ ： Z_0 沿關節 1 的軸線；原點與 X_0 方向可自由選擇，通常選在基座底面中心並讓 X_0 指向機器人正前方以簡化參數。
4. 逐一建立各連桿座標系：對 $i = 1$ 到 $n-1$ ，找出 Z_{i-1} 與 Z_i 的公垂線，原點置於公垂線與 Z_i 的交點， X_i 沿公垂線由 Z_{i-1} 指向 Z_i 。
5. 處理末端座標系 $\{n\}$ ： Z_n 通常沿末端執行器的接近方向（approach direction）， X_n 的選擇以使參數簡潔為原則。
6. 填寫參數表並驗算：對每一行讀出四個參數。驗算方法是把所有關節設為零位，用手（或模型）比對計算結果與實際幾何是否吻合。

◎ 三種特殊情形的處理

一、兩軸平行：公垂線不唯一，可自由選擇位置，通常選能讓 $d_i = 0$ 的那一條以簡化參數。二、兩軸相交：公垂線長度 $a = 0$ ， X 軸取兩軸外積的方向（ $X_i = \pm Z_{i-1} \times Z_i$ ）。三、兩軸重合：這是移動關節常見的情形， $a = 0$ 且 $\alpha = 0$ 。這三種情形在實際機器人上非常普遍——正因為工程師刻意讓軸平行或相交，才使機構容易製造且逆運動學有封閉解（第 10 章）。

9.4 連桿變換矩陣

9.4.1 由四個參數組成變換

有了四個 DH 參數，相鄰連桿之間的變換可以拆解成四個基本動作的連乘。以修正式 DH（Craig 慣例）為例，從座標系 $\{i-1\}$ 到 $\{i\}$ 的變換為：

$${}^{i-1}T_i = R_x(\alpha_{i-1}) \cdot D_x(a_{i-1}) \cdot R_z(\theta_i) \cdot D_z(d_i)$$

四個動作依序為：繞 X 轉 α 、沿 X 移 a 、繞 Z 轉 θ 、沿 Z 移 d 。前兩者跨越連桿、後兩者繞著關節。

這個順序有清楚的物理意義：先「走過連桿」（由前一個關節軸移動到這一個關節軸），再「轉動關節」。把四個矩陣乘開，得到標準的連桿變換矩陣：

$${}^{i-1}T_i = \begin{bmatrix} c\theta_i & -s\theta_i & 0 & a_{i-1} \\ s\theta_i c\alpha_{i-1} & c\theta_i c\alpha_{i-1} & -s\alpha_{i-1} & -s\alpha_{i-1}d_i \\ s\theta_i s\alpha_{i-1} & c\theta_i s\alpha_{i-1} & c\alpha_{i-1} & c\alpha_{i-1}d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

修正式 DH 的連桿變換矩陣。 c 與 s 分別代表 $\cos$ 與 $\sin$ 。這是本章最重要的一個公式，後續所有計算都由它出發。

9.4.2 讀懂這個矩陣

這個矩陣看似複雜，但結構清晰。右上角的平移欄 $[a_{i-1}, -s\alpha_{i-1}d_i, c\alpha_{i-1}d_i]^T$ 告訴我們：座標系原點的位移由 a 沿 X 、 d 沿 Z （再被 α 旋轉）所構成。左上角的 3×3 旋轉部分，則是 $R_x(\alpha) \cdot R_z(\theta)$ 的乘積。

特別注意第一列 $[c\theta, -s\theta, 0, a]$ ：它完全不含 α ，這是因為 X 軸的方向不受繞 X 軸旋轉的影響。這類結構性的觀察，是驗算矩陣有沒有抄錯的好方法。

9.4.3 串起整條運動鏈

有了每一節的變換，第 8 章的消去律就派上用場了：

$${}^0T_n = {}^0T_1 \cdot {}^1T_2 \cdot {}^2T_3 \cdot \dots \cdot {}^{n-1}T_n$$

正向運動學的核心方程式。相鄰的上下標兩兩抵消，剩下由基座 0 到末端 n 。

再加上末端工具的固定變換，就得到 TCP 在基座座標系中的位姿：

$${}^0T_{TCP} = {}^0T_n \cdot {}^nT_{TCP}$$

${}^nT_{TCP}$ 由工具幾何決定，換夾具時需重新設定（第 8.9.1 節）。

至此，正向運動學的完整解答已經呈現：把每個關節角度代入各自的連桿變換矩陣，然後全部乘起來。剩下的工作，就是把它算對——這正是接下來三節要示範的。

9.4.4 由矩陣讀出位置與姿態

計算出 0T_n 之後，如何取出我們要的資訊？依第 8.6.2 節的結構：

$${}^0T_n = \begin{bmatrix} \hat{n} & \hat{o} & \hat{a} & p \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

右上 3×1 的 p 就是末端位置；左上 3×3 的三個行向量 $\hat{n}$ 、 $\hat{o}$ 、 $\hat{a}$ 分別是末端座標系的 X 、 Y 、 Z 軸方向。

在機器人學傳統上，這三個行向量有專門的名稱： $\hat{a}$ (approach) 是接近向量，通常沿夾爪的伸入方向； $\hat{o}$ (orientation) 是姿態向量，沿夾爪開合的方向； $\hat{n}$ (normal) 是法線向量，由 $\hat{o} \times \hat{a}$ 決定。工業機器人的示教器上顯示的姿態，往往就是由這三個向量換算出的 RPY 角（第 8.10.1 節）。

9.5 標準 DH 與修正式 DH

9.5.1 兩種慣例的差異

DH 參數法有兩種常見的慣例，兩者的差別在於座標系放在連桿的哪一端，以及四個基本動

作的順序。這是機器人學中一個惱人但必須弄清楚的分歧。

標準 DH (Denavit-Hartenberg 原始版本)：座標系 $\{i\}$ 放在連桿 i 的遠端 (靠近下一個關節)， Z_i 軸沿關節 $i+1$ 的軸線。變換順序為：

$${}^{i-1}T_i = R_z(\theta_i) \cdot D_z(d_i) \cdot D_x(a_i) \cdot R_x(\alpha_i)$$

標準 DH：先轉關節、再走連桿。注意參數的下標都是 i 。

修正式 DH (Craig 版本)：座標系 $\{i\}$ 放在連桿 i 的近端 (靠近前一個關節)， Z_i 軸沿關節 i 的軸線。變換順序為：

$${}^{i-1}T_i = R_x(\alpha_{i-1}) \cdot D_x(a_{i-1}) \cdot R_z(\theta_i) \cdot D_z(d_i)$$

修正式 DH：先走連桿、再轉關節。注意 a 與 α 的下標是 $i-1$ 。

9.5.2 該用哪一種

兩種慣例在數學上完全等價，都能得到正確的末端位姿，但參數值不同、座標系位置不同。實務上的選擇準則是：

修正式 DH 的優點：座標系 $\{i\}$ 的 Z 軸就是關節 i 的轉軸，這使得「第 i 個關節的變數對應到第 i 個座標系」的關係更直觀。此外，在動力學推導中 (第 14 章的 Newton-Euler 遞迴)，把座標系放在連桿近端更為自然，因為連桿的質心與慣量都在該座標系中描述。本書與 Craig 教科書一致，採用修正式 DH。

標準 DH 的優點：歷史悠久，許多經典文獻與商用軟體使用此慣例。若讀者需要與既有的參數表對接，必須確認對方使用哪一種。

◎ 最重要的一條警告

絕對不可混用兩種慣例。用標準 DH 的參數表配上修正式 DH 的變換矩陣公式，會得到一個看似合理、實則完全錯誤的結果——而且在零位附近誤差很小，不易察覺，直到手臂大幅運動時才會暴露。實務上的建議是：任何 DH 參數表都必須在標題明確註明「標準」或「修正式」，並在文件中寫出所使用的變換矩陣公式。這是機器人專案交接時最常出問題的環節之一。

9.5.3 DH 法的限制

DH 參數法雖然優雅，但有三個限制值得知道。

限制一：平行軸的奇異性。當相鄰兩關節軸接近平行時，公垂線的位置變得極為敏感——

軸向的微小偏差會導致公垂線位置大幅跳動。這使得 DH 參數在標定時對這類機構的數值穩定性較差。實務上會改用其他參數化方式（如 Hayati 模型）來處理。

限制二：只適用於串聯機構。 DH 法建立在「一根連桿接一根連桿」的鏈狀結構上。對第 4 章的並聯機器人（Delta、Stewart 平台），必須改用其他方法（通常是對每條支鏈分別建模，再加上閉環約束方程）。

限制三：不夠直觀。現代的機器人軟體（如 ROS 2 的 URDF）多半直接以「父連桿到子連桿的完整 6 參數變換」來描述，不再要求座標系必須沿公垂線放置。這樣參數雖多了兩個，但工程師可以把座標系放在任何方便的位置（例如 CAD 模型的自然原點），大幅降低建模門檻。DH 法在今日主要的價值，是在**理論推導與教學**中提供最精簡的表達。

9.6 平面 2R 與 3R 手臂

9.6.1 平面 2R 手臂的正向運動學

最簡單也最重要的例子是平面兩連桿手臂：兩個旋轉關節、軸線互相平行且垂直於運動平面，連桿長度分別為 L_1 與 L_2 。

這個機構的正向運動學可以用純幾何推導。第一節連桿末端（即第二個關節）的位置為：

$$x_1 = L_1 \cos \theta_1, \quad y_1 = L_1 \sin \theta_1$$

第一節末端位置。

第二節連桿相對於水平線的角度是 $\theta_1 + \theta_2$ （因為 θ_2 是相對於第一節的角度），因此末端位置為：

$$\begin{aligned} x &= L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2) \\ y &= L_1 \sin \theta_1 + L_2 \sin(\theta_1 + \theta_2) \end{aligned}$$

平面 2R 手臂的正向運動學。這是全機器人學最常被引用的一組方程式，請務必熟記。

末端的姿態（連桿 2 相對於基座的角度）則為 $\varphi = \theta_1 + \theta_2$ 。因此這個機構有三個輸出量（ x 、 y 、 φ ）但只有兩個自由度——這說明它無法獨立控制位置與姿態，這一點在第 10 章討論逆運動學時會再回來。

9.6.2 以 DH 法驗證

同樣的結果也可用 DH 法得到。建立修正式 DH 參數表：

表 9-3 平面 2R 手臂的修正式 DH 參數

i	α_{i-1}	a_{i-1}	d_i	θ_i
1	0	0	0	θ_1 (變數)
2	0	L_1	0	θ_2 (變數)
3 (末端)	0	L_2	0	0

代入 9.4.1 節的連桿變換矩陣 (因所有 $\alpha = 0$ ，矩陣大幅簡化)，連乘三個矩陣後可得：

$${}^0T_3 = \begin{bmatrix} c_{12} & -s_{12} & 0 & L_1 c_1 + L_2 c_{12} \\ s_{12} & c_{12} & 0 & L_1 s_1 + L_2 s_{12} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

其中 $c_1 = \cos \theta_1$ 、 $c_{12} = \cos(\theta_1 + \theta_2)$ ， s 同理。右上角的平移欄正是幾何推導的結果，兩種方法完全一致。

這個驗證很有價值：它證明了 DH 法這套看似繁瑣的規則，得到的答案與直覺的幾何推導完全相同。對於簡單機構，幾何法更快；但對於六軸手臂這種空間機構，幾何推導幾乎不可能，DH 法的系統性就變得不可或缺。

9.6.3 平面 3R 手臂

再加一節連桿，就成為平面 3R 手臂。依同樣的邏輯推展：

$$\begin{aligned} x &= L_1 c_1 + L_2 c_{12} + L_3 c_{123} \\ y &= L_1 s_1 + L_2 s_{12} + L_3 s_{123} \\ \phi &= \theta_1 + \theta_2 + \theta_3 \end{aligned}$$

其中 $c_{123} = \cos(\theta_1 + \theta_2 + \theta_3)$ 。三個自由度對應三個輸出 (x, y, ϕ)，恰好可獨立控制平面位置與姿態。

平面 3R 手臂是一個重要的教學範例，因為它是最小的「完整」機器人——在平面世界中，剛體有三個自由度 (x, y, ϕ)，而它恰好有三個關節。這使它成為研究逆運動學多解問題 (第 10 章) 與奇異點 (第 12 章) 的標準範例。

圖 9-3 平面 2R 與 3R 手臂的幾何關係

建議配圖：左圖為平面 2R 手臂——基座在原點，第一節長 L_1 與 X 軸夾角 θ_1 ，第二節長 L_2 與第一節夾角 θ_2 ，標出末端位置 (x, y) 及其分解的四個投影分量 ($L_1 \cos \theta_1$ 、 $L_2 \cos(\theta_1 + \theta_2)$ 等)。右圖為 3R 手臂，另標出末端姿態角 $\phi = \theta_1 + \theta_2 + \theta_3$ 。兩圖下方各附正向運動學方程式。

9.7 SCARA 機器人

9.7.1 構型回顧與座標系建立

如第 4.8.1 節所述，SCARA 是 RRPR 構型：兩個垂直軸的旋轉關節提供水平面運動、一個垂

直移動關節提供高度、末端再一個旋轉關節控制方向。

由於所有旋轉軸都平行（皆為垂直方向），移動軸也與之平行，這使得所有的 α 參數皆為 0（或 180° ），DH 參數表格外簡潔。設第一臂長 a_1 、第二臂長 a_2 、基座高度 d_1 ：

表 9-4 SCARA 機器人的修正式 DH 參數

i	α_{i-1}	a_{i-1}	d_i	θ_i	關節類型
1	0	0	d_1	θ_1 (變數)	旋轉
2	0	a_1	0	θ_2 (變數)	旋轉
3	0	a_2	d_3 (變數)	0	移動
4	180°	0	d_4	θ_4 (變數)	旋轉

第 4 行的 $\alpha = 180^\circ$ 是因為 SCARA 的第四軸（末端旋轉）習慣上讓 Z 軸朝下（工具向下作業），因此相對於前一個座標系翻轉了 180° 。

9.7.2 正向運動學結果

連乘四個變換矩陣後，可得末端位姿：

$$\begin{aligned}x &= a_1 c_1 + a_2 c_{12} \\y &= a_1 s_1 + a_2 s_{12} \\z &= d_1 - d_3 \\\phi &= \theta_1 + \theta_2 - \theta_4\end{aligned}$$

SCARA 的正向運動學。注意 x 、 y 與平面 $2R$ 完全相同（因為水平運動確實就是 $2R$ ）， z 由移動關節單獨決定， ϕ 為末端繞垂直軸的方向。

這組方程式展現了 SCARA 構型的優雅之處：**運動被完全解耦**。水平位置只由 θ_1 、 θ_2 決定；高度只由 d_3 決定；末端方向只由三者的組合決定。這種解耦使得逆向運動學有簡潔的封閉解，控制也更為直觀——這正是它在電子組裝線上如此普及的數學原因。

◎ 解耦的工程價值

當運動學方程式能夠解耦時，好處是全面的：逆運動學有封閉解（計算快、無迭代、必定收斂）、控制器可以分軸獨立設計、故障診斷更容易定位、而且操作員的直覺與機器行為一致（「我要它高一點」就只調 Z 軸）。第 4 章說 SCARA 是「剛剛好就好」的最佳範例，本節提供了數學上的證據。

9.8 六軸工業機器人

9.8.1 典型構型與球形手腕

絕大多數的六軸工業機器人採用相似的構型：前三軸（腰、肩、肘）決定末端**位置**，後三軸（手腕的三個旋轉軸）決定末端**姿態**。而後三軸的軸線通常**交會於同一點**，稱為**球形手腕**（spherical wrist）。

球形手腕的設計不是為了機構方便，而是為了**數學**。當後三軸交於一點時，逆向運動學可以被拆解成兩個獨立的子問題：先由手腕中心點的位置解出前三軸、再由姿態解出後三軸。這稱為**運動學解耦**（kinematic decoupling），它使得六軸手臂的逆運動學存在封閉解。若沒有球形手腕，逆運動學就只能用數值迭代求解（第 10 章）。

這是機構設計服務於演算法的經典案例：工程師刻意犧牲一些機構自由度（讓三軸交於一點會增加設計難度），換取計算上的巨大便利。

9.8.2 PUMA 型手臂的 DH 參數

以經典的 PUMA 560 構型為例（許多現代六軸手臂沿用此架構），其修正式 DH 參數表為：

表 9-5 六軸手臂（PUMA 型）的修正式 DH 參數

i	α_{i-1}	a_{i-1}	d_i	θ_i	說明
1	0	0	0	θ_1	腰部旋轉（繞垂直軸）
2	-90°	0	0	θ_2	肩部俯仰
3	0	a_2	d_3	θ_3	肘部俯仰
4	-90°	a_3	d_4	θ_4	手腕旋轉 1
5	90°	0	0	θ_5	手腕俯仰
6	-90°	0	0	θ_6	手腕旋轉 2

觀察這張表可以學到幾件事。第一， α 只出現 0、 $\pm 90^\circ$ 三種值——這不是巧合，而是因為工業機器人的關節軸總是互相平行或垂直，這樣才容易製造且容易分析。第二， a_2 是最大的參數（通常數百公釐），它就是上臂的長度，決定了機器人的工作半徑。第三，後三軸的 a 與 d 大多為 0，這正是球形手腕「三軸交於一點」的數學表現。

9.8.3 連乘的實務作法

六個 4×4 矩陣連乘，手算約需一小時且極易出錯。實務上有三種作法：

作法一：符號運算軟體。用 SymPy、Mathematica 或 MATLAB 的 Symbolic Toolbox，可以得到含 $\theta_1 \sim \theta_6$ 的解析表達式。這對理論分析與推導逆運動學公式很有用，但表達式會非常龐大（展開後可能數百項）。

作法二：數值計算。給定具體的關節角度，直接用 NumPy 做矩陣連乘，毫秒內完成。這是實務控制中的標準作法，本章的程式範例即採此法。

作法三：現成函式庫。使用 Robotics Toolbox、Pinocchio、KDL 等函式庫，只需輸入 DH 參數或 URDF 檔案即可。這是工程專案的最佳選擇——這些函式庫經過大量驗證，且效能經過最佳化。

值得強調的是：**理解推導過程與使用現成工具並不衝突**。你需要理解原理才能除錯（當結果不對時，知道該檢查哪個環節）、才能判斷工具的輸出是否合理、也才能在遇到非標準機構時自行建模。

圖 9-4 六軸工業機器人的座標系配置

建議配圖：六軸手臂側視圖與立體圖並列。在每一個關節處畫出該連桿座標系的三軸（ X 紅、 Y 綠、 Z 藍），並標註各軸的 DH 參數（ a_2 、 a_3 、 d_3 、 d_4 等實際尺寸）。特別以虛線圓圈標出後三軸交會的「手腕中心點」，並註明「球形手腕使逆運動學可解耦」。

9.9 運動學誤差與標定

9.9.1 理論與現實的落差

正向運動學給出的是**理論位置**。把它與實際位置比對，往往會發現數公釐甚至數公分的差距。這個落差來自五個誤差源：

誤差一：製造公差。連桿的實際長度與圖面標稱值有差異，通常在 ± 0.1 mm 等級。對 $a_2 = 500$ mm 的上臂，0.1 mm 的誤差在末端就是 0.1 mm——看似不大，但六個關節的誤差會累積。

誤差二：裝配誤差。關節軸線的實際方向與理論方向有夾角偏差。這是最嚴重的一類：0.1 度的軸線偏斜，在 500 mm 外會造成約 0.87 mm 的位置誤差（ $500 \times \tan 0.1^\circ \approx 0.87$ ）。這正是第 3.1.3 節提到的「誤差沿運動鏈累積放大」。

誤差三：關節零位偏移。編碼器的零點與理論零位不完全一致。這通常是最大的單一誤差源，但也最容易修正（只需在軟體中加一個偏移量）。

誤差四：彈性變形。第 4.1.2 節的剛體假設在此失效。手臂在重力與負載下會彎曲，末端下垂可達數公釐；且變形量隨姿態與負載而變，難以用固定參數補償。

誤差五：溫度效應。鋁的線膨脹係數約 $23 \times 10^{-6} / ^\circ\text{C}$ ，一支 1 公尺長的手臂在溫度變化 10°C 時會伸長約 0.23 mm。這是精密應用中不可忽略的因素，也是為什麼高精度工作站需要恆溫環境。

9.9.2 重複精度與絕對精度

這些誤差解釋了第 4.11.1 節提到的現象：**重複精度遠優於絕對精度**。

重複精度 (repeatability) 是「回到同一組關節角度時，末端位置的一致程度」。由於上述誤差都是系統性的 (同樣的角度總是產生同樣的誤差)，它們在重複定位時會互相抵消，因此重複精度可達 $\pm 0.02 \text{ mm}$ 。

絕對精度 (absolute accuracy) 是「到達程式指定的笛卡兒座標時，實際位置與指令值的差距」。這時所有的模型誤差都會直接顯現，典型值在 ± 0.5 到 $\pm 2 \text{ mm}$ ——比重複精度差了一到兩個數量級。

這個差距決定了兩種截然不同的使用方式。**示教編程**只依賴重複精度：操作員手動把機器人移到正確位置並記錄關節角度，之後重播即可，模型誤差完全不影響。**離線編程與視覺導引**則依賴絕對精度：程式給出的是笛卡兒座標，機器人必須靠運動學模型換算成關節角度——模型有多準，定位就有多準。

9.9.3 運動學標定

運動學標定的目的，是用實際量測資料反推出**真實的 DH 參數**，取代圖面上的理論值。標準流程為：

1. **量測**：用雷射追蹤儀、球桿儀或相機系統，量測數十到數百個不同姿態下的實際末端位置。
2. **建模**：建立誤差模型，把 DH 參數的偏差視為未知數，寫出「實際位置 = f(關節角, DH 參數 + 參數偏差)」。
3. **辨識**：以最小平方或非線性最佳化，求出使預測值與量測值差距最小的參數偏差。這在數學上與第 10 章的數值逆運動學屬於同一類問題。
4. **驗證**：用另一組未參與辨識的姿態驗證標定結果，避免過度擬合。

5. 寫入：把修正後的參數寫入控制器，取代原有的理論參數。

經過標定的機器人，絕對精度可由 $\pm 1 \text{ mm}$ 提升到 $\pm 0.2 \text{ mm}$ 以內，這對離線編程與視覺導引作業是決定性的改善。近年許多廠商已將標定納入出廠程序，並提供現場再標定的工具。

9.10 由正向邁向逆向

9.10.1 本章的成果

走完本章，你已經掌握了機器人學的第一項核心計算能力。給定任何一支串聯機械手臂與一組關節角度，你能夠：建立連桿座標系、填寫 DH 參數表、寫出各節的變換矩陣、連乘求得末端位姿、並由結果矩陣讀出位置與姿態。

更重要的是，你理解了一件事：**機器人的運動學，本質上是一連串座標系的接力**。從基座出發，每經過一個關節就轉換一次座標系，一路傳遞到夾爪。第 8 章的齊次變換不再是抽象的數學，而是描述真實機構的工具。

9.10.2 接下來的三個挑戰

第 10 章：逆向運動學。現在我們能由關節角求位置，但實務上需要的是反過來——「我要夾爪到那個位置，各關節該轉幾度？」這個問題困難得多：可能無解、可能有八組解、也可能有無限多解。我們將學習幾何解法、代數解法與數值迭代法，並理解為什麼球形手腕如此重要。

第 11 章：雅可比矩陣。本章處理的是靜態的位置關係。當關節開始以某個速度轉動時，末端會以多快的速度移動？答案是一個矩陣——雅可比矩陣。它不只連結速度，也連結力與力矩，是控制與力覺的橋樑。

第 12 章：奇異點與冗餘。當雅可比矩陣失去滿秩時，機器人會失去某個方向的運動能力。本章 9.8.1 節提到的手臂完全伸直、以及第 8 章的萬向節鎖，都將在那裡得到統一的數學解釋。

換句話說：第 9 章讓你算出手在哪裡，第 10 章讓你決定手該怎麼擺，第 11、12 章讓你知道手能怎麼動。

完整計算例題

例題 9-1 平面 2R 手臂的正向運動學

題目：某平面 2R 手臂， $L_1 = 300 \text{ mm}$ 、 $L_2 = 200 \text{ mm}$ 。(a) 當 $\theta_1 = 30^\circ$ 、 $\theta_2 = 45^\circ$ 時，

求末端位置與姿態。(b) 當 $\theta_1 = 0^\circ$ 、 $\theta_2 = 0^\circ$ 時，求末端位置，並說明此姿態的特殊意義。(c) 求此手臂末端可達的最大與最小距離。

解 (a)：直接代入 9.6.1 節的公式。先算 $\theta_1 + \theta_2 = 75^\circ$ 。

$$x = 300 \cos 30^\circ + 200 \cos 75^\circ = 300(0.8660) + 200(0.2588) = 259.81 + 51.76 = 311.57 \text{ mm}$$

末端 X 座標。

$$y = 300 \sin 30^\circ + 200 \sin 75^\circ = 300(0.5) + 200(0.9659) = 150.00 + 193.19 = 343.19 \text{ mm}$$

末端 Y 座標。

末端姿態 $\varphi = \theta_1 + \theta_2 = 75^\circ$ 。因此末端位姿為 $(311.57, 343.19, 75^\circ)$ ，距離基座 $\sqrt{(311.57^2 + 343.19^2)} \approx 463.5 \text{ mm}$ 。

解 (b)： $\theta_1 = \theta_2 = 0^\circ$ 時， $x = 300 + 200 = 500 \text{ mm}$ 、 $y = 0$ 。此時手臂**完全伸直**，末端位於工作空間的外邊界。這個姿態極為特殊：它是第 4.10.3 節提到的邊界奇異點，末端在徑向已無法再前進，且逆向運動學在此處會失去唯一性——這一點將在第 12 章詳細分析。

解 (c)：最大距離發生在完全伸直時： $r_{\max} = L_1 + L_2 = 500 \text{ mm}$ 。最小距離發生在完全折疊時 ($\theta_2 = 180^\circ$)：

$$r_{\min} = |L_1 - L_2| = |300 - 200| = 100 \text{ mm}$$

最小可達距離。工作空間是一個內徑 100 mm、外徑 500 mm 的圓環（不考慮關節極限）。

工程觀察：若兩連桿等長 ($L_1 = L_2$)，則 $r_{\min} = 0$ ，工作空間是一個完整的圓，中間沒有「洞」。這正是許多教學型手臂採用等長連桿的原因——工作空間最完整。但代價是完全折疊的姿態在實體上會發生自我碰撞，因此實際仍有內部限制。

例題 9-2 SCARA 的正向運動學與工作空間

題目：某 SCARA 機器人 $a_1 = 400 \text{ mm}$ 、 $a_2 = 300 \text{ mm}$ 、 $d_1 = 500 \text{ mm}$ ，Z 軸行程 d_3 範圍為 0~150 mm。(a) 當 $\theta_1 = 60^\circ$ 、 $\theta_2 = -30^\circ$ 、 $d_3 = 80 \text{ mm}$ 、 $\theta_4 = 20^\circ$ 時，求 TCP 位姿。(b) 求工作空間的水平環狀範圍與垂直範圍。(c) 若要在半徑 750 mm 處作業，是否可行？

解 (a)：代入 9.7.2 節的公式。 $\theta_1 + \theta_2 = 30^\circ$ 。

$$x = 400 \cos 60^\circ + 300 \cos 30^\circ = 400(0.5) + 300(0.8660) = 200.00 + 259.81 = 459.81 \text{ mm}$$

$$y = 400 \sin 60^\circ + 300 \sin 30^\circ = 400(0.8660) + 300(0.5) = 346.41 + 150.00 = 496.41 \text{ mm}$$

水平位置。

$$z = d_1 - d_3 = 500 - 80 = 420 \text{ mm}$$

$$\phi = \theta_1 + \theta_2 - \theta_4 = 60^\circ + (-30^\circ) - 20^\circ = 10^\circ$$

高度與末端方向。

TCP 位姿為 (459.81, 496.41, 420, $\phi=10^\circ$)，水平距離基座 $\sqrt{(459.81^2 + 496.41^2)} \approx 676.5$ mm。

解 (b)：水平方向的可達範圍為內徑 $|400 - 300| = 100$ mm、外徑 $400 + 300 = 700$ mm 的環狀區域。垂直方向為 z 由 $500 - 150 = 350$ mm 到 $500 - 0 = 500$ mm，行程 150 mm。整體工作空間是一個扁平的中空圓柱體。

解 (c)：750 mm 超過最大可達半徑 700 mm，不可行。且即使是 700 mm 也是完全伸直的邊界奇異點，實務上應保留至少 10% 餘裕（第 4.10.3 節），因此建議的最遠作業半徑約為 630 mm。若必須在 750 mm 處作業，解決方案有三：更換更長臂展的機型、將機器人加裝在第七軸行走軸上、或重新配置工作站佈局讓工件更靠近。

例題 9-3 三軸空間手臂的 DH 建模與計算

題目：某三軸手臂構型為：第一軸繞垂直 Z 軸旋轉（腰）、第二軸繞水平軸旋轉（肩）、第三軸繞平行於第二軸的水平軸旋轉（肘）。上臂長 $a_2 = 400$ mm、前臂長 $a_3 = 350$ mm，基座高 $d_1 = 300$ mm。（a）建立修正式 DH 參數表。（b）當 $\theta_1 = 45^\circ$ 、 $\theta_2 = 30^\circ$ 、 $\theta_3 = -60^\circ$ 時，求末端位置。

解 (a)：依 9.3.3 節步驟。 Z_0 沿第一軸（垂直）， Z_1 沿第二軸（水平），兩軸垂直相交於基座頂端，故 $\alpha_0 = \dots$ 需注意：由 Z_0 轉到 Z_1 需繞 X_0 轉 -90° ，且兩軸相交故 $a_0 = 0$ 。第二、三軸平行，故 $\alpha_1 = 0$ 、 $a_1 = a_2$ （上臂長）。

表 9-6 三軸手臂的修正式 DH 參數

i	α_{i-1}	a_{i-1}	d_i	θ_i
1	0	0	$d_1 = 300$	θ_1
2	-90°	0	0	θ_2
3	0	$a_2 = 400$	0	θ_3

i	α_{i-1}	a_{i-1}	d_i	θ_i
末端	0	$a_3 = 350$	0	0

解 (b)：這類「腰 + 平面二連桿」的構型，可用一個技巧簡化計算：先在垂直平面內當作平面 2R 手臂計算，再繞 Z 軸旋轉 θ_1 。

在垂直平面內，令 r 為水平投影距離、 z 為高度：

$$r = a_2 \cos \theta_2 + a_3 \cos(\theta_2 + \theta_3) = 400 \cos 30^\circ + 350 \cos(-30^\circ) \\ = 400(0.8660) + 350(0.8660) = 346.41 + 303.11 = 649.52 \text{ mm}$$

水平投影距離。注意 $\theta_2 + \theta_3 = 30^\circ + (-60^\circ) = -30^\circ$ ，而 $\cos(-30^\circ) = \cos 30^\circ$ 。

$$z = d_1 + a_2 \sin \theta_2 + a_3 \sin(\theta_2 + \theta_3) = 300 + 400(0.5) + 350(-0.5) \\ = 300 + 200 - 175 = 325 \text{ mm}$$

高度。

再繞 Z 軸旋轉 $\theta_1 = 45^\circ$ ，將水平投影分解到 X、Y：

$$x = r \cos \theta_1 = 649.52 \cos 45^\circ = 459.28 \text{ mm} \\ y = r \sin \theta_1 = 649.52 \sin 45^\circ = 459.28 \text{ mm} \\ z = 325 \text{ mm}$$

末端位置。

驗算技巧：末端到基座軸心的水平距離應為 $\sqrt{(459.28^2 + 459.28^2)} = 649.5 \text{ mm}$ ，與 r 相符。此外，由於 $\theta_2 + \theta_3 = -30^\circ$ 與 $\theta_2 = 30^\circ$ 對稱，前臂與上臂相對於水平線的角度大小相同、方向相反，這種對稱性使 r 的兩項相等——這正是計算結果中 346.41 與 303.11 之比恰為 400:350 的原因。

程式範例：通用正向運動學求解器

以下 Python 程式實作通用的串聯手臂正向運動學：輸入 DH 參數表與關節角度，輸出各連桿座標系與末端位姿。程式以本章三個例題驗證，並示範工作空間掃描。

```
import numpy as np

def dh_transform(alpha, a, d, theta):
    """修正式 DH (Craig 慣例) 的連桿變換矩陣 (9.4.1 節)"""
    ca, sa = np.cos(alpha), np.sin(alpha)
    ct, st = np.cos(theta), np.sin(theta)
    return np.array([
        [ ct, -st, 0, a ],
        [ st*ca, ct*ca, -sa, -sa*d ],
        [ st*sa, ct*sa, ca, ca*d ],
        [ 0, 0, 0, 1 ]
    ])

```



```

])

def forward_kinematics(dh_table, q):
    """
    dh_table: 每列為 (alpha, a, d, theta, joint_type)
               joint_type: 'R' 旋轉 (theta 為變數) 、'P' 移動 (d 為變數) 、
                           'F' 固定 (無變數)

    q: 關節變數陣列
    回傳: (末端 4x4 變換, 各連桿座標系原點位置串列)
    """
    T = np.eye(4)
    origins = [T[:3, 3].copy()]
    qi = 0
    for alpha, a, d, theta, jtype in dh_table:
        if jtype == 'R':
            theta = theta + q[qi]; qi += 1
        elif jtype == 'P':
            d = d + q[qi]; qi += 1
        T = T @ dh_transform(alpha, a, d, theta)
        origins.append(T[:3, 3].copy())
    return T, origins

def pose_from_T(T):
    """由變換矩陣取出位置與 RPY 姿態 (第 8.10.1 節)"""
    p = T[:3, 3]
    R = T[:3, :3]
    if abs(R[2, 0]) > 0.99999: # 萬向節鎖
        pitch = -np.sign(R[2, 0]) * np.pi/2
        roll, yaw = 0.0, np.arctan2(-R[0, 1], R[1, 1])
    else:
        pitch = np.arctan2(-R[2, 0], np.hypot(R[0, 0], R[1, 0]))
        yaw = np.arctan2(R[1, 0], R[0, 0])
        roll = np.arctan2(R[2, 1], R[2, 2])
    return p, np.rad2deg([roll, pitch, yaw])

d = np.deg2rad

# ----- 驗證一：例題 9-1 平面 2R -----
print("=== 例題 9-1：平面 2R 手臂 ===")
planar_2r = [
    (0, 0, 0, 0, 'R'), # 關節 1
    (0, 300, 0, 0, 'R'), # 關節 2 (a = L1)
    (0, 200, 0, 0, 'F'), # 末端 (a = L2)
]
T, _ = forward_kinematics(planar_2r, [d(30), d(45)])
p, rpy = pose_from_T(T)
print(f"末端位置: x={p[0]:.2f}, y={p[1]:.2f} mm")
print(f"末端姿態: yaw={rpy[2]:.1f}° (應為 75°)")

# ----- 驗證二：例題 9-2 SCARA -----
print("\n=== 例題 9-2：SCARA ===")
scara = [

```

```

    (0,      0,    500, 0, 'R'),    # theta1, d1=500
    (0,     400,     0, 0, 'R'),    # a1=400
    (0,     300,     0, 0, 'P'),    # a2=300, d3 為變數
    (np.pi, 0,     0, 0, 'R'),    # alpha=180°, theta4
]
T, _ = forward_kinematics(scara, [d(60), d(-30), -80, d(20)])
p, _ = pose_from_T(T)
print(f"TCP 位置: x={p[0]:.2f}, y={p[1]:.2f}, z={p[2]:.2f} mm")
print(f"水平距離: {np.hypot(p[0], p[1]):.1f} mm (應約 676.5)")

# ----- 驗證三：例題 9-3 三軸空間手臂 -----
print("\n=== 例題 9-3：三軸空間手臂 ===")
arm3 = [
    (0,      0, 300, 0, 'R'),
    (-np.pi/2, 0, 0, 0, 'R'),
    (0,     400, 0, 0, 'R'),
    (0,     350, 0, 0, 'F'),
]
T, origins = forward_kinematics(arm3, [d(45), d(30), d(-60)])
p, _ = pose_from_T(T)
print(f"末端位置: x={p[0]:.2f}, y={p[1]:.2f}, z={p[2]:.2f} mm")
print(f"各關節位置: {[np.round(o, 1).tolist() for o in origins]}")

# ----- 應用：工作空間掃描 (9.1.3 節用途四) -----
print("\n=== 平面 2R 工作空間掃描 ===")
pts = []
for t1 in np.linspace(-np.pi, np.pi, 60):
    for t2 in np.linspace(-np.pi, np.pi, 60):
        T, _ = forward_kinematics(planar_2r, [t1, t2])
        pts.append(T[:2, 3])
pts = np.array(pts)
r = np.linalg.norm(pts, axis=1)
print(f"可達半徑範圍: {r.min():.1f} ~ {r.max():.1f} mm")
print(f"理論值 (|L1-L2| ~ L1+L2): 100.0 ~ 500.0 mm")

```

請試著修改：（一）把 SCARA 的 α_4 由 180° 改成 0° ，觀察末端姿態如何改變。（二）為例題 9-3 的手臂增加一個手腕軸，觀察 DH 表如何擴充。（三）將工作空間掃描的結果以 Matplotlib 畫成散布圖，你會看到第 4.10.2 節所描述的環狀工作空間。

實作：為實體手臂建立運動學模型

本章的實作是為一支真實的機械手臂（建議使用附錄 E 的六軸人形手臂、或任何三軸以上的教學型手臂）建立完整的運動學模型，並驗證其準確度。

1. **繪製機構簡圖**：測量並記錄每一節連桿的長度、各關節軸的方向與相對位置。以線段與圓柱符號繪出機構簡圖，標出所有關節軸線。

2. **建立座標系**：依 9.3.3 節的六步驟，逐一標定 Z 軸、找出公垂線、建立各連桿座標系。將座標系直接畫在機構簡圖上。
3. **填寫 DH 參數表**：逐行讀出四個參數，注意標明使用的是修正式 DH。
4. **程式實作**：將參數表輸入本章的程式範例，計算數組不同關節角度下的末端位置。
5. **零位驗證**：將所有關節設為零位，比對程式計算結果與實際手臂的幾何量測值。若不符，通常是座標系建立或參數符號有誤。
6. **多點驗證**：選取至少五組不同的關節角度組合，實際擺出該姿態並用尺（或方格紙、雷射測距）量測末端位置，記錄理論值與實測值。
7. **誤差分析**：計算每一組的誤差量與方向，填入下表。分析誤差是系統性偏移（每次都往同一方向偏，可用零位補償修正）還是隨機分散（可能是背隙或關節鬆動）。
8. **零位標定**：若發現系統性誤差，嘗試調整各關節的零位偏移量，使誤差最小化。這是 9.9.3 節標定流程的簡化版。

表 9-7 運動學驗證記錄表（第一列為範例）

組別	關節角度 ($\theta_1, \theta_2, \theta_3$)	理論位置 (x, y, z)	實測位置	誤差量	誤差方向
1	(0°, 0°, 0°)	(750, 0, 300)	(742, 3, 296)	9.4 mm	偏內側、偏低
2~5	(自行填寫)				

國中小也看得懂：一節一節接力算出手在哪裡

伸出你的手臂，先把肩膀往前抬 45 度，再把手肘彎 90 度。現在你的手掌在哪裡？

你可能會說「在我胸前」——但如果要用數字說清楚呢？比如說「在我肩膀前面 20 公分、往上 10 公分」。這就是機器人每一毫秒都在做的計算。它的每個關節裡都有一個「角度感應器」，隨時知道自己轉了幾度；然後電腦要靠這些角度，算出手到底伸到哪裡去了。

為什麼要算？想想看：如果機器人不知道自己的手在哪裡，它怎麼知道會不會撞到桌子？怎麼知道有沒有碰到要抓的東西？就像你閉著眼睛走路會撞到門，機器人不會算，就什麼事都做不好。

這個計算其實有個小訣竅：**一節一節接力**。先算出第一節手臂的末端在哪裡，把那裡當成新的起點，再算第二節，再把那裡當起點算第三節……一直算到手掌。就像玩接力賽跑，每一棒交

給下一棒。你在附錄 E 做的六軸人形機器人，裡面的程式就是這樣一節一節算出手腳位置的喔！

叫獸提醒與常見錯誤

- 誤區一：混用標準 DH 與修正式 DH。參數表與變換矩陣公式必須來自同一種慣例，混用會得到「看似合理但完全錯誤」的結果。
- 誤區二：忘記標明慣例。任何 DH 參數表都應在標題註明是標準還是修正式，並附上所用的變換矩陣公式。
- 誤區三：Z 軸方向前後不一致。Z 軸方向可自由選擇，但一旦選定就必須貫徹到底；中途改變會使 α 與 θ 的符號全錯。
- 誤區四：把 a 當成連桿的實體長度。 a 是兩關節軸公垂線的長度，若軸線有偏移，它未必等於連桿外觀的長度。
- 誤區五：忽略 d 參數。當兩關節軸有沿軸向的錯開時（如 PUMA 的 d_3 、 d_4 ），漏掉 d 會造成數公分的誤差。
- 誤區六：兩軸平行時隨意選公垂線卻未保持一致。平行軸的公垂線不唯一，應選定使 $d = 0$ 的那一條並貫徹。
- 誤區七：程式中角度單位混用。DH 參數表常以度為單位書寫，但三角函數需要弧度，轉換遺漏是最常見的低級錯誤。
- 誤區八：忘記末端工具的變換。 0T_n 只到法蘭面，加上夾具後必須再乘 ${}^nT_{TCP}$ ，否則會差一個工具長度。
- 誤區九：以為理論計算就等於實際位置。製造、裝配、零位、變形與溫度誤差合計可達數公釐，精密應用必須標定。
- 誤區十：用重複精度的規格去承諾絕對精度的任務。離線編程與視覺導引需要的是絕對精度，兩者可差一到兩個數量級。

課堂討論

1. 開頭故事中，小翰花兩分鐘試誤才把夾爪移到目標位置，誤差 2.8 公分。若他有正向運動學程式但沒有逆向運動學，能否加速這個過程？請設計一個只用正向運動學的搜尋策略，並討論其效率。

2. 為什麼六軸工業機器人幾乎都採用「後三軸交於一點」的球形手腕？這個設計犧牲了什麼、換來了什麼？請從機構製造與演算法兩方面討論。
3. 現代的 URDF 用六個參數描述連桿關係，比 DH 的四個參數多。既然如此，為什麼教科書仍然要教 DH 法？請討論兩者各自的適用場合。
4. 某機器人的重複精度為 $\pm 0.02 \text{ mm}$ ，絕對精度為 $\pm 1.5 \text{ mm}$ 。請各設計一個「只需重複精度」與「必須絕對精度」的實際應用，並說明理由。
5. 若一支手臂的第二軸與第三軸並非完全平行（裝配偏差 0.2° ），對正向運動學的計算結果會有多大影響？請估算在 500 mm 臂展下的末端誤差，並討論該用什麼方法補救。

章末摘要

- 正向運動學由關節變數求末端位姿，其解必定存在且唯一，是逆運動學、雅可比與動力學的共同基礎。
- 五大用途：即時狀態顯示、模擬與離線編程、碰撞檢測、工作空間分析、以及逆運動學的迭代基礎。
- 編號慣例：基座為連桿 0、末端為連桿 n ；關節 i 連接連桿 $i-1$ 與 i ； n 軸手臂共有 $n+1$ 根連桿。
- DH 法的核心洞見是沿公垂線建立座標系（ Z 沿關節軸、 X 沿公垂線），使描述兩座標系關係的六個參數壓縮為四個。
- 四個 DH 參數： a 與 α 描述連桿本身（恆為常數）、 d 與 θ 描述關節狀態（其一為變數）。
- 修正式 DH 的連桿變換為 $R_x(\alpha) \cdot D_x(a) \cdot R_z(\theta) \cdot D_z(d)$ ，先走連桿再轉關節；本書採此慣例。
- 標準 DH 與修正式 DH 數學等價但參數值與座標系位置不同，絕對不可混用，且必須明確標註。
- 正向運動學即連桿變換的連乘： ${}^0T_n = {}^0T_1 \cdot {}^1T_2 \cdots {}^{n-1}T_n$ ，再乘 ${}^nT_{TCP}$ 得工具位姿。
- 由結果矩陣可讀出：右上 3×1 為位置，左上 3×3 的三行為末端座標系的 $\hat{n}$ 、 $\hat{o}$ 、 $\hat{a}$ 三軸方向。
- 平面 2R 手臂： $x = L_1 C_1 + L_2 C_{12}$ 、 $y = L_1 S_1 + L_2 S_{12}$ 、 $\varphi = \theta_1 + \theta_2$ ；工作空間為內徑 $|L_1 - L_2|$ 、外徑 $L_1 + L_2$ 的圓環。

- SCARA 的運動學完全解耦（水平由前兩軸、高度由移動軸、方向由三者組合），這是其逆運動學有簡潔封閉解的原因。
- 六軸手臂的球形手腕（後三軸交於一點）使逆運動學得以解耦成位置與姿態兩個子問題，是封閉解存在的關鍵。
- 運動學誤差有五源：製造公差、裝配誤差、零位偏移、彈性變形、溫度效應；其中裝配誤差因沿鏈累積而最為嚴重。
- 重複精度（ $\pm 0.02\text{ mm}$ ）遠優於絕對精度（ $\pm 0.5 \sim 2\text{ mm}$ ）；示教編程只需前者，離線編程與視覺導引需要後者。
- 運動學標定以實測資料反推真實 DH 參數，可將絕對精度提升至 $\pm 0.2\text{ mm}$ 以內。

觀念題與計算題

觀念題

1. 定義正向運動學問題，並說明為何其解必定存在且唯一。
2. 列舉正向運動學的五個實際用途，並各舉一個具體情境。
3. 說明連桿與關節的編號慣例，並指出 n 軸手臂共有幾根連桿。
4. DH 法如何把描述兩座標系關係所需的六個參數壓縮為四個？關鍵的幾何洞見是什麼？
5. 說明四個 DH 參數的定義與幾何意義，並指出哪兩個恆為常數。
6. 寫出修正式 DH 的四個基本動作及其順序，並說明其物理意義。
7. 比較標準 DH 與修正式 DH 的差異，並說明混用會造成什麼後果。
8. 當相鄰兩關節軸平行、相交、重合時，DH 參數各有什麼特徵？應如何處理公垂線的選擇？
9. 說明由 0T_n 矩陣中如何讀出末端的位置與姿態，並解釋 $\hat{n}$ 、 $\hat{o}$ 、 $\hat{a}$ 的意義。
10. 為何 SCARA 的正向運動學能夠完全解耦？這帶來哪四項工程好處？
11. 什麼是球形手腕？它對逆向運動學有何決定性的影響？
12. 列舉運動學誤差的五個來源，並指出何者因沿鏈累積而影響最大。
13. 區分重複精度與絕對精度，並各舉一個依賴該精度的應用。
14. 說明運動學標定的五個步驟，以及為何需要保留一組未參與辨識的資料做驗證。

計算題

1. 某平面 2R 手臂 $L_1 = 250 \text{ mm}$ 、 $L_2 = 250 \text{ mm}$ 。(a) 當 $\theta_1 = 45^\circ$ 、 $\theta_2 = 90^\circ$ 時求末端位置與姿態。(b) 求工作空間的內外半徑，並說明等長連桿的特殊性。(c) 若末端須到達 (300, 200)，請驗證此點是否落在工作空間內 (提示：計算其距原點的距離)。
2. 某平面 3R 手臂 $L_1 = 300$ 、 $L_2 = 250$ 、 $L_3 = 150 \text{ mm}$ 。當 $\theta_1 = 20^\circ$ 、 $\theta_2 = 40^\circ$ 、 $\theta_3 = -30^\circ$ 時，(a) 求末端位置與姿態角 φ 。(b) 求第二關節與第三關節的位置。(c) 若要維持末端位置不變但改變姿態角，說明是否可行及原因。
3. 某 SCARA 機器人 $a_1 = 350 \text{ mm}$ 、 $a_2 = 250 \text{ mm}$ 、 $d_1 = 450 \text{ mm}$ 、Z 軸行程 $0 \sim 200 \text{ mm}$ 。(a) 當 $\theta_1 = -40^\circ$ 、 $\theta_2 = 70^\circ$ 、 $d_3 = 120 \text{ mm}$ 、 $\theta_4 = 15^\circ$ 時求 TCP 位姿。(b) 求工作空間的水平環狀範圍與垂直範圍。(c) 若在此工作空間中要放置一個直徑 400 mm 的圓形治具，其中心應距基座多遠才能讓整個治具都在可達範圍內？
4. 某三軸手臂 $d_1 = 350 \text{ mm}$ 、 $a_2 = 450 \text{ mm}$ 、 $a_3 = 400 \text{ mm}$ ，構型同例題 9-3。(a) 建立修正式 DH 參數表。(b) 當 $\theta_1 = 60^\circ$ 、 $\theta_2 = 45^\circ$ 、 $\theta_3 = -90^\circ$ 時求末端位置。(c) 求此手臂在垂直平面內的最大水平可達距離，以及達到該距離時的關節角度組合。(d) 若第二軸的零位有 $+1^\circ$ 的偏移未被修正，估算在 (b) 的姿態下造成的末端位置誤差。

動手做與專題延伸

本章實作：DH 參數表建立練習

為下列三種機構分別建立修正式 DH 參數表，並以本章程式驗證。這是掌握 DH 法最有效的練習方式——建議按難度順序完成。

- **入門**：三軸桌上型手臂 (腰 + 肩 + 肘)，如附錄 C 的吃錢幣機器人手臂或任何三軸教學套件。
- **進階**：五軸手臂 (三軸位置 + 兩軸手腕)，如常見的教學型機械手臂套件。注意手腕兩軸的軸線關係。
- **挑戰**：六軸協作機器人。可由網路取得公開的 URDF 檔案 (如 UR5、Franka Emika)，由其中讀出連桿尺寸，反推 DH 參數表，再與文獻上公布的參數表比對。

專題延伸

- **運動學視覺化工具**：延續第 8 章的 `draw_frame` 函式，撰寫一個能繪出整支手臂的程式——輸入 DH 參數與關節角度，畫出所有連桿 (連接各座標系原點的線段) 與各連桿座標系。加

上滑桿控制各關節角度，即時更新圖形，做出一個互動式的運動學模擬器。

- **工作空間視覺化**：以蒙特卡羅法隨機取樣數萬組關節角度，計算對應的末端位置並繪成三維散布圖或點雲。加入關節極限的限制，比較有無限制時工作空間的差異——你會直觀看到第 4.10 節所說的「有效工作空間遠小於可達工作空間」。
- **進階：誤差敏感度分析**。為六軸手臂的每一個 DH 參數加上微小擾動（如 a 加 0.5 mm、 α 加 0.1° ），計算末端位置的變化量。由此找出「最敏感的參數」——這正是運動學標定時應優先辨識的參數，也是製造時應優先管控的公差項目。將結果整理成敏感度排序表。

參考文獻與延伸閱讀

- Craig, J. J., Introduction to Robotics: Mechanics and Control, 4th ed., Pearson (第 3 章：操作臂運動學，本章的主要參考，修正式 DH 的來源)。
- Denavit, J. and Hartenberg, R. S., "A Kinematic Notation for Lower-Pair Mechanisms Based on Matrices," ASME Journal of Applied Mechanics, 1955 (DH 參數法的原始論文)。
- Lynch, K. M. and Park, F. C., Modern Robotics: Mechanics, Planning, and Control, Cambridge University Press (第 4 章：正向運動學，含指數積公式的替代方法)。
- Siciliano, B., Sciavicco, L., Villani, L. and Oriolo, G., Robotics: Modelling, Planning and Control, Springer (第 2 章：運動學，採用標準 DH)。
- Spong, M. W., Hutchinson, S. and Vidyasagar, M., Robot Modeling and Control, 2nd ed., Wiley (DH 法與運動鏈建模)。
- Hayati, S. and Mirmirani, M., "Improving the Absolute Positioning Accuracy of Robot Manipulators," Journal of Robotic Systems, 1985 (平行軸問題與改良的參數化方法)。
- Roth, Z. S., Mooring, B. W. and Ravani, B., "An Overview of Robot Calibration," IEEE Journal on Robotics and Automation, 1987 (運動學標定的經典回顧)。
- Corke, P., Robotics, Vision and Control, 3rd ed., Springer (含 Robotics Toolbox 的實作範例)。
- ROS 2 URDF 官方文件與 Pinocchio、KDL 等運動學函式庫的技術文件。
- 各機器人廠商技術手冊中的 DH 參數表與運動學規格（多數廠商會在技術文件中公開）。

第 10 章

逆向運動學

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】同一個馬克杯、同一個手掌位置，卻是八張疊合的手臂剪影——每一張的肩、肘、腕呈現不同的構型（肩前／肩後、肘上／肘下、腕翻轉／不翻轉），以不同深淺的顏色區分。畫面右側另有一隻伸長卻構不到杯子的手臂，旁註「無解」；左下角一隻手掌固定而手肘上下擺動的殘影，旁註「無限多解」。標題文字：一個目標，八種答案——或者，沒有答案。

叫獸開講 「人看到杯子就伸手，機器人卻可能算出八種姿勢。」

叫獸在桌上放了一個馬克杯，請小好伸手去拿。她毫不猶豫地伸手、握住、拿起，全程不到一秒。

「請問，」叫獸說，「你剛才的肩關節轉了幾度？手肘呢？手腕呢？」小好笑了：「我怎麼可能知道。」

「但你的大腦知道，而且是在零點三秒內算出來的。」叫獸在白板上寫下一個方程式： $f(q) = x$ 。「上一章我們學的是已知 q 求 x ，這叫正向運動學，代進去乘一乘就有答案。今天我們要做的是反過來——已知你要的 x ，求 q 。這叫逆向運動學，而它困難得多。」

他請小好再拿一次杯子，但這次要求：「手肘必須抬高、貼著耳朵。」小好照做了，動作警扭但確實拿到了杯子。「注意，」叫獸對全班說，「同一個杯子、同一個手掌位置，她用了**兩種完全不同的姿勢**。這就是逆向運動學的第一個麻煩：**答案不只一個**。」

接著他把杯子往前推到桌子最遠處，超出小好坐著能伸到的範圍。小好伸長了手臂，指尖離杯子還有十公分。「這是第二個麻煩：**可能無解**。目標超出工作空間時，方程式沒有實數解——但機器人不會像人一樣停下來說『我構不到』，它可能會算出一個複數，然後控制器把它當成一個很大的角度指令送給馬達。」他停頓了一下，「這就是為什麼很多學生的第一支機械手臂，是在執行第一個逆運動學指令時撞爛的。」

最後他讓小好把手掌固定在杯子上不准動，然後說：「現在請你上下擺動手肘。」小好照做——手掌紋絲不動，手肘卻能上下擺動一大段。「第三個麻煩：**可能有無限多解**。這就是第 4 章講的冗餘。」

叫獸在白板上把三個麻煩並排寫下：**無解、多解、無限多解**。「正向運動學是代數，逆向運動學是解方程式——而且是一組非線性的三角方程式。有些機構運氣好，能解出漂亮的封閉公式；有些機構解不出來，只能用數值方法一步一步逼近。今天這堂課，我們兩種都要學。」

下課前他補了一句：「還有一件事。人類拿杯子時，從來不需要『算』角度——我們是靠反覆練習長出來的直覺。這件事在第 31 章會再出現：現代的機器人學習，某種程度上就是想讓機器也長出這種直覺，跳過解方程式這一步。但在那之前，我們得先學會老老實實地解它。」

本章為什麼重要

逆向運動學是機器人**實際作業時真正需要回答的問題**。沒有人會對機器人說「請把第三軸轉到 47.2 度」，人們說的是「把那個零件放到這個孔裡」。從笛卡兒座標到關節角度的翻譯，就是逆向運動學。

它也是機器人學中第一個「困難」的問題。正向運動學只需要把數字代進公式；逆向運動學則要解一組非線性方程，而且必須面對無解、多解與數值不穩定等現實。理解這些困難的來源，你才能診斷「為什麼機器人到不了這個點」「為什麼它突然用一個奇怪的姿勢過去」這類實務問題。

本章的內容也是後續章節的關鍵基礎：第 11 章的雅可比矩陣正是數值逆運動學的核心工具；第 12 章的奇異點解釋了為何某些位置的逆解會爆炸；第 15 章的軌跡生成需要對路徑上每一點求逆解；第 26 章的路徑規劃則要在關節空間與任務空間之間反覆換算。

學習目標

- 能定義逆向運動學問題，並說明其與正向運動學在本質上的三個差異。
- 能判斷一個目標位姿是否落在工作空間內，並說明無解時的處置方式。
- 能分析平面 2R 手臂的多解結構，並用幾何法求出封閉解。
- 能運用餘弦定理與 atan2 求解平面手臂的逆運動學。
- 能說明六軸手臂的八組解結構及其對應的機構構型。
- 能說明運動學解耦原理，並解釋球形手腕為何使封閉解成為可能。
- 能推導數值逆運動學的迭代公式，並說明雅可比反矩陣的角色。
- 能比較 Jacobian 轉置法、偽逆法與阻尼最小平方方法的特性。
- 能說明阻尼最小平方方法 (DLS) 如何處理奇異點附近的數值爆炸。
- 能在逆運動學中納入關節極限與避障等額外約束。
- 能以 Python 實作封閉解與數值解，並比較兩者的效能與適用場合。

先備知識

第 8 章的齊次變換與姿態表示、第 9 章的正向運動學與 DH 參數。本章需要三角函數的和差

角公式、餘弦定理，以及矩陣的反矩陣概念。數值方法部分會用到偏導數的概念（但完整的雅可比推導留待第 11 章）。

本章是全書目前為止數學最密集的一章。建議的讀法是：先確實掌握 10.3 節的平面 2R 幾何解（這是所有直覺的來源），再進入數值方法。若某段推導卡住，先跳過細節、抓住結論，之後再回頭補。

核心名詞中英對照

表 10-1 本章核心名詞

中文	英文	說明
逆向運動學	Inverse Kinematics, IK	由末端位姿求關節變數的問題
封閉解 / 解析解	Closed-form Solution	以有限次代數運算直接求得的解
數值解	Numerical Solution	以迭代逼近求得的解
多解	Multiple Solutions	同一位姿對應多組關節角度
手肘向上 / 向下	Elbow-up / Elbow-down	平面二連桿的兩種構型
構型	Configuration	逆解中對應特定機構姿勢的解分支
運動學解耦	Kinematic Decoupling	將位置與姿態拆為兩個子問題求解
球形手腕	Spherical Wrist	後三軸交於一點的手腕構型
Pieper 準則	Pieper's Criterion	六軸手臂存在封閉解的充分條件
雅可比矩陣	Jacobian Matrix	關節速度到末端速度的線性映射
偽逆	Pseudoinverse	非方陣的廣義反矩陣
阻尼最小平方法	Damped Least Squares, DLS	以阻尼項抑制奇異點附近數值爆炸的方法
零空間	Null Space	不影響末端運動的關節速度所張的空間
收斂	Convergence	迭代結果趨近於解的性質
殘差	Residual	目前位姿與目標位姿的差距

10.1 逆向運動學問題

10.1.1 問題定義

逆向運動學的問題陳述是正向運動學的反轉：

已知： ${}^0T_{\text{目標}}$ （末端的期望位姿） 求： $\mathbf{q} = [q_1, q_2, \dots, q_n]^T$

求出使 $f(q) = x$ 目標成立的所有關節變數組合。

這是一組非線性方程式。以六軸手臂為例， 0T_6 有 12 個有意義的元素（ 3×3 旋轉加 3×1 平移），但因旋轉矩陣的正交約束，實際只有 6 個獨立方程式，對應 6 個未知數 $\theta_1 \sim \theta_6$ 。方程式數與未知數數相等，看似「剛好可解」——但這些方程式充滿 $\sin$ 與 $\cos$ 的乘積與和，並非線性代數能直接處理。

10.1.2 三個本質差異

逆向運動學與正向運動學有三個本質差異，這正是它困難的根源。

差異一：解未必存在。當目標位姿落在工作空間之外，方程式無實數解。此外即使位置可達，某些姿態組合也可能不可達（靈巧工作空間小於可達工作空間，見第 4.10.1 節）。

差異二：解未必唯一。同一個末端位姿，往往對應多組關節角度。平面 2R 有兩組（手肘上 / 下）、六軸手臂典型有八組、冗餘手臂則有無限多組。控制器必須有一套規則來決定「選哪一組」。

差異三：未必有封閉解。正向運動學永遠可以直接算出來；逆向運動學只有在機構滿足特定條件時（如球形手腕）才有封閉公式，否則只能數值迭代。

表 10-2 正向與逆向運動學的比較

面向	正向運動學	逆向運動學
問題	已知 q 求 x	已知 x 求 q
數學本質	代入求值	解非線性方程組
解的存在性	永遠存在	可能無解（超出工作空間）
解的唯一性	唯一	常有多組解，冗餘時無限多
計算方式	矩陣連乘，直接得解	封閉解或數值迭代
計算成本	低且固定	封閉解低、數值解高且不定
實務角色	模擬、監控、碰撞檢測	路徑執行、示教、實際作業

10.1.3 兩大類解法

逆向運動學的解法分為兩大陣營，各有明確的適用場合。

封閉解（解析解）：用代數與幾何推導出直接的公式。優點是計算極快（微秒等級）、能求

出**全部**的解、且不會有收斂失敗的問題。缺點是必須針對每一種機構個別推導，且只有特定構型才存在。工業機器人控制器內部使用的都是封閉解。

數值解（迭代法）：由一個初始猜測出發，反覆修正直到誤差夠小。優點是**通用**——任何機構都適用，且容易加入額外約束（關節極限、避障、姿勢偏好）。缺點是計算較慢、只能找到一組解（取決於初始猜測）、在奇異點附近可能不穩定或不收斂。研究型機器人與冗餘手臂多採此法。

實務上兩者常結合使用：先用封閉解快速求出所有解、再用數值法做精細修正或處理額外約束。

10.2 解的存在性與工作空間

10.2.1 可達性判斷

在求解之前，應先判斷目標是否可達。最基本的檢查是距離判斷——以平面 2R 手臂為例，末端到基座的距離 r 必須滿足：

$$|L_1 - L_2| \leq r \leq L_1 + L_2$$

外側界限為完全伸直、內側界限為完全折疊。超出此範圍則無解。

對空間手臂，可達性的判斷較為複雜，但基本原則相同：先檢查位置是否在可達工作空間內，再檢查該位置能否達成所要求的姿態（靈巧工作空間）。實務上常用的作法是：先嘗試求解，若解出的角度含複數或超出關節極限，即判定為不可達。

10.2.2 無解時的處置

當目標不可達時，控制器絕不能把錯誤的計算結果直接送給馬達——這正是開頭故事中「第一支手臂撞爛」的原因。正確的處置方式有四種：

方式一：拒絕並報錯。回報「目標超出工作空間」，停止動作。這是最安全的作法，工業控制器的標準行為。

方式二：投影到最近可達點。若目標在工作空間外，取工作空間邊界上最接近的點。這在遙操作與視覺伺服中常用——操作員把目標拉到太遠時，手臂會伸到極限並停住，而非亂動。

方式三：放寬姿態要求。許多任務對姿態的要求並不嚴格（例如吸盤只需垂直向下，繞垂直

軸的角度無所謂)。放寬一個姿態自由度，往往就能讓原本不可達的點變為可達。

方式四：改變基座位置。若機器人裝在移動平台或行走軸上，可先移動基座再求解。這是行動操作機器人 (mobile manipulator) 的標準策略。

◎ 實務上最常見的「假不可達」

學生回報「手臂到不了這個點」時，有超過一半的情況並非真的超出工作空間，而是三個原因之一：一、**關節極限**——數學上有解，但某個角度超出機構允許範圍；二、**自我碰撞**——姿態可達但手臂會撞到自己；三、**姿態要求過嚴**——位置可達但要求的姿態在該處無法達成。診斷時應分別檢查這三項，而非直接認定超出工作空間。

10.3 平面 2R 手臂的封閉解

10.3.1 幾何解法：餘弦定理

平面 2R 手臂是理解逆運動學多解結構的最佳範例。給定目標位置 (x, y) ，求 θ_1 與 θ_2 。

第一步：求 θ_2 。觀察由基座、肘關節、末端所構成的三角形，其三邊分別為 L_1 、 L_2 與 $r = \sqrt{x^2 + y^2}$ 。由餘弦定理：

$$r^2 = L_1^2 + L_2^2 - 2L_1L_2 \cos(180^\circ - \theta_2) = L_1^2 + L_2^2 + 2L_1L_2 \cos \theta_2$$

注意三角形內角為 $180^\circ - \theta_2$ ，因為 θ_2 是連桿 2 相對於連桿 1 延長線的夾角。

整理後得到：

$$\cos \theta_2 = (x^2 + y^2 - L_1^2 - L_2^2) / (2 L_1 L_2)$$

求解 θ_2 的關鍵式。若右式的絕對值大於 1，代表目標不可達 (10.2.1 節)。

這裡出現第一個關鍵點：由 $\cos \theta_2$ 求 θ_2 時，會得到兩個解——正的與負的：

$$\theta_2 = \pm \arccos((x^2 + y^2 - L_1^2 - L_2^2) / (2 L_1 L_2))$$

正號對應手肘向下 (elbow-down)、負號對應手肘向上 (elbow-up)。這就是開頭故事中小妍的兩種拿杯姿勢。

第二步：求 θ_1 。已知 θ_2 後， θ_1 可由兩個角度相減求得：目標點的方位角，減去「第一節連桿與目標方向的夾角」：

$$\theta_1 = \text{atan2}(y, x) - \text{atan2}(L_2 \sin \theta_2, L_1 + L_2 \cos \theta_2)$$

第一項是目標的方位角、第二項是第一節連桿相對於目標方向的偏轉角。務必使用 atan2 以正確處理四個象

限。

由於 θ_2 有兩個值，代入後會得到兩個對應的 θ_1 ——因此總共兩組解。

10.3.2 兩組解的幾何意義

這兩組解在幾何上構成一個對稱：以「基座到末端的連線」為軸，兩種構型互為鏡像。手肘向上的解，肘關節位於連線上方；手肘向下的解，肘關節位於連線下方。

兩者的末端位置完全相同，但整條手臂佔據的空間完全不同。這在實務上極為重要：某一組解可能會撞到工件或治具，另一組則不會。控制器選擇構型的常見準則有三：

- **維持構型連續性**：整條軌跡都使用同一構型，避免中途切換（切換時手臂必須大幅擺動，既危險又耗時）。
- **最小關節移動量**：選擇離目前姿態最近的那一組解，減少運動時間與能耗。
- **避開障礙與關節極限**：檢查每一組解是否會發生碰撞或超出行程，選擇可行的那一組。

圖 10-1 平面 2R 手臂的兩組逆解

建議配圖：同一個目標點 (x, y) ，畫出兩種手臂構型——實線為手肘向上（肘關節在基座與末端連線的上方）、虛線為手肘向下（肘關節在下方），兩者以連線為軸呈鏡像對稱。標註 θ_1 、 θ_2 的定義角度，並在中間畫出由 L_1 、 L_2 、 r 構成的三角形與餘弦定理的關係。右側附兩組解的數值範例。

10.3.3 特殊情形

三種特殊情形需要在程式中特別處理。

情形一：目標在邊界 ($r = L_1 + L_2$)。此時 $\cos \theta_2 = 1$ ， $\theta_2 = 0$ ，兩組解合而為一（手臂完全伸直）。這是第 12 章要討論的**邊界奇異點**——在此處，末端已無法再沿徑向前進。

情形二：目標在內界 ($r = |L_1 - L_2|$)。此時 $\cos \theta_2 = -1$ ， $\theta_2 = 180^\circ$ ，手臂完全折疊。同樣是奇異點，且實體上通常會自我碰撞。

情形三：目標在原點且 $L_1 = L_2$ 。此時任何 θ_1 配上 $\theta_2 = 180^\circ$ 都是解——出現無限多解。這是一個退化的情形，數值上會導致 $\text{atan2}(0, 0)$ 的不定值，程式必須特別處理。

10.4 代數解法與三角恆等式技巧

10.4.1 代數解法的基本策略

幾何法直觀但難以推廣到空間機構。代數解法則系統性得多：把正向運動學方程式 ${}^0T_n(q)$

= $T_{\text{目標}}$ 的兩邊逐元素比對，得到一組方程式，再設法逐一解出各個關節變數。

關鍵技巧是設法讓某個未知數單獨出現。常用的手法是在方程式兩邊同乘某個變換的反矩陣，把已知的部分移到一邊。例如：

$${}^0T_1^{-1}(\theta_1) \cdot T_{\text{目標}} = {}^1T_2 \cdot {}^2T_3 \cdot \dots \cdot {}^{n-1}T_n$$

左邊只含 θ_1 與已知量，右邊不含 θ_1 。比對兩邊的某些元素，就能解出 θ_1 。

這個「逐步剝離」的策略是所有封閉解推導的核心。Craig 教科書中 PUMA 手臂的完整推導即採此法，過程冗長但邏輯清晰。

10.4.2 三種常見的三角方程式型式

推導過程中會反覆遇到三種型式的三角方程式，熟悉它們的解法能大幅加速推導。

型式一： $\sin \theta = a$ 、 $\cos \theta = b$ （兩者皆已知）。這是最理想的情形，直接用 atan2 得到唯一解：

$$\theta = \text{atan2}(a, b)$$

只要 a 與 b 都已知， θ 就唯一確定（不會有象限模糊）。這是為何要用 atan2 而非 $\arcsin$ 或 $\arccos$ 的根本原因。

型式二： $a \sin \theta + b \cos \theta = c$ 。這類方程式用「三角函數合成」處理。令 $\rho = \sqrt{a^2 + b^2}$ 、 $\varphi = \text{atan2}(b, a)$ ，則：

$$\rho \sin(\theta + \varphi) = c \Rightarrow \theta = \text{atan2}(c, \pm \sqrt{\rho^2 - c^2}) - \varphi$$

有兩組解（ $\pm$ 號），對應多解結構。當 $\rho < |c|$ 時無實數解，代表目標不可達。

型式三：兩式聯立，如 $a \sin \theta + b \cos \theta = c$ 與 $a \cos \theta - b \sin \theta = d$ 。此時可直接解出：

$$\sin \theta = (ac - bd)/(a^2 + b^2), \quad \cos \theta = (ad + bc)/(a^2 + b^2) \Rightarrow \theta = \text{atan2}(\dots)$$

化為型式一，得唯一解。

10.4.3 解的個數從何而來

觀察上述型式可以理解多解的來源：每一次開平方根或反餘弦，都會產生一個 $\pm$ 的分岔。平面 2R 有一次分岔（ θ_2 的正負），故有 2 組解；六軸手臂在推導過程中會遇到三次分岔，故有 $2^3 = 8$ 組解。

這三次分岔在機構上各有明確意義：肩部（前 / 後）、肘部（上 / 下）、腕部（翻轉 / 不翻

轉)。工業機器人的示教器上通常會顯示目前的構型代碼(如 "NUT" 或 "FUT")，正是在標示這三個二選一的狀態。程式設計師必須理解這些代碼，否則會遇到「同樣的座標，機器人卻用完全不同的姿勢過去」的困惑。

10.5 運動學解耦與球形手腕

10.5.1 Pieper 準則

並非所有六軸機構都有封閉解。1968 年 Pieper 提出一個充分條件 (Pieper 準則)：若六軸機器人的任意三個相鄰關節軸交會於一點、或三軸互相平行，則存在封閉解。

絕大多數工業機器人採用第一種情形：後三軸 (手腕) 交會於一點，構成第 9.8.1 節介紹的球形手腕。這不是巧合，而是刻意的設計決策——為了讓逆運動學可解。

10.5.2 解耦的核心洞見

球形手腕帶來的關鍵好處是：手腕中心點的位置，完全不受後三軸角度的影響。因為後三軸都繞著同一個點旋轉，無論它們怎麼轉，那個交會點都不會移動。

這使得問題可以拆成兩半。設末端工具的位姿為 (p, R) ，手腕中心到末端的距離為 d_6 (沿末端 Z 軸方向)，則手腕中心點的位置為：

$$p_{\text{wrist}} = p - d_6 \cdot \hat{a} \quad (\hat{a} \text{ 為末端座標系的 Z 軸方向，即 } R \text{ 的第三行})$$

由末端位姿直接算出手腕中心。這一步不需要任何關節角度資訊。

有了手腕中心的位置，問題就分解為：

- **子問題一 (位置)：**求 $\theta_1, \theta_2, \theta_3$ ，使前三軸把手腕中心送到 p_{wrist} 。這是一個三自由度的位置問題，可用類似平面 2R 的方法求解。
- **子問題二 (姿態)：**已知 $\theta_1, \theta_2, \theta_3$ 後，前三軸造成的旋轉 0R_3 為已知。剩下的旋轉必須由後三軸補足，因此求解 ${}^3R_6 = {}^0R_3^T \cdot R$ 目標，這是一個純姿態問題，等同於由旋轉矩陣求歐拉角 (第 8.10.1 節)。

兩個三自由度的問題，遠比一個六自由度的問題容易處理——這就是解耦的價值。

圖 10-2 運動學解耦示意

建議配圖：六軸手臂側視圖。以紅色圓點標出手腕中心 (後三軸交會處)，並用箭頭表示由末端位姿 p 沿 $-\hat{a}$ 方向退 d_6 得到 p_{wrist} 。左半部以藍色標示前三軸 ($\theta_1, \theta_2, \theta_3$) 負責把手腕中心送到位置；右半部以綠色標示

後三軸 (θ_4 、 θ_5 、 θ_6) 負責調整姿態。下方以兩個方框標註「子問題一：位置」與「子問題二：姿態」。

10.5.3 六軸手臂的八組解

依 10.4.3 節的分析，六軸手臂的解在推導過程中經歷三次二選一的分岔，因此共有 $2^3 = 8$ 組解：

表 10-3 六軸手臂的八組解結構

分岔	兩種選擇	機構意義	產生位置
肩部構型	前 / 後 (Front / Back)	手臂向前伸或向後反折跨過基座	求 θ_1 時的兩個相差 180° 的解
肘部構型	上 / 下 (Up / Down)	肘關節在肩腕連線的上方或下方	求 θ_3 時的 $\arccos$ 正負號
腕部構型	翻轉 / 不翻轉 (Flip / No-flip)	手腕以兩種等效姿態達成同一方向	求 θ_5 時的 $\arccos$ 正負號

這八組解在數學上完全等價（末端位姿相同），但在工程上差異巨大：某些解會超出關節極限、某些會發生自我碰撞、某些會撞到工件。實務上控制器的選解策略通常是：先過濾掉不可行的解（超限、碰撞），再從剩下的解中選擇離目前姿態最近的一組。

值得注意的是，當機器人接近奇異點時，某些解會互相靠攏甚至合併——例如手臂完全伸直時，肘上與肘下的解合而為一。這正是第 12 章要探討的現象。

10.6 數值逆運動學的原理

10.6.1 為什麼需要數值方法

封閉解雖然快速精確，卻有三個限制：必須逐一針對每種機構推導（工作量大且易錯）、只適用於滿足 Pieper 準則的構型、且難以納入額外約束（避障、關節極限、姿勢偏好）。

數值方法則完全通用。它的基本策略是：從一個初始猜測 q_0 出發，反覆問「目前差多遠？該往哪個方向調整？」直到誤差足夠小。這正是最佳化的思維，也是第 2 章提到「最佳化負責選擇」的具體實踐。

10.6.2 誤差與雅可比

設目前的關節角度為 q ，用正向運動學算出目前位姿 $f(q)$ ，與目標的差距（殘差）為：

$$\mathbf{e} = \mathbf{x}_{\text{目標}} - \mathbf{f}(\mathbf{q})$$

殘差向量。位置部分是簡單的向量相減；姿態部分則需用第 8.10.3 節的方法（由 $R_{\text{error}} = R_{\text{current}}^T \cdot R_{\text{target}}$ 取軸角）轉為三維向量。

接下來的關鍵問題是：關節該調整多少，才能減少這個誤差？答案來自一個線性近似——當關節有微小變化 $\Delta \mathbf{q}$ 時，末端位姿的變化為：

$$\Delta \mathbf{x} \approx \mathbf{J}(\mathbf{q}) \cdot \Delta \mathbf{q}$$

$\mathbf{J}$ 是雅可比矩陣 (Jacobian)，它把關節空間的微小變化映射到任務空間。這是第 11 章的主題，本章先接受這個關係並使用它。

把它反過來，就得到迭代的修正量：

$$\Delta \mathbf{q} = \mathbf{J}^{-1} \cdot \mathbf{e} \quad \mathbf{q}_{\text{new}} = \mathbf{q} + \alpha \cdot \Delta \mathbf{q}$$

α 是步長 ($0 < \alpha \leq 1$)，用於控制收斂速度與穩定性。這就是牛頓 - 拉夫森法在逆運動學上的應用。

10.6.3 迭代演算法

完整的數值逆運動學演算法如下：

1. **初始化**：給定初始猜測 $\mathbf{q}_0$ （通常用機器人目前的姿態，這是最好的猜測）。
2. **正向運動學**：計算 $\mathbf{f}(\mathbf{q})$ ，得到目前位姿。
3. **計算殘差**： $\mathbf{e} = \mathbf{x}_{\text{目標}} - \mathbf{f}(\mathbf{q})$ 。若 $\|\mathbf{e}\|$ 小於容差，則收斂成功、結束。
4. **計算雅可比**： $\mathbf{J}(\mathbf{q})$ （第 11 章會說明如何計算）。
5. **求修正量**： $\Delta \mathbf{q} = \mathbf{J}^+ \cdot \mathbf{e}$ （ $\mathbf{J}^+$ 為反矩陣或偽逆）。
6. **更新**： $\mathbf{q} \leftarrow \mathbf{q} + \alpha \cdot \Delta \mathbf{q}$ ，並將各關節限制在允許範圍內。
7. **檢查迭代次數**：若超過上限仍未收斂，回報失敗；否則回到步驟 2。

注意步驟 2 呼叫了正向運動學——這正是第 9.1.2 節所說「所有數值逆運動學內部都要反覆呼叫正向運動學」的具體體現。一次數值逆解通常需要 5 到 50 次迭代，每次都要算一遍正向運動學與雅可比矩陣，因此計算量遠高於封閉解。

◎ 初始猜測的重要性

數值法只會找到「離初始猜測最近」的那一組解。這帶來兩個實務結論：一、在連續軌跡上求解時，應以上一個點的解作為下一個點的初始猜測——這不只加速收斂，也自然維持了構型連續性（10.3.2 節）。二、若要找出所有的解，必須從多個不同的初始猜測分別出發，這稱為多重起始

(multi-start) 策略。

10.7 三種數值方法的比較

10.7.1 Jacobian 轉置法

最簡單的方法是直接用雅可比的**轉置**取代反矩陣：

$$\Delta \mathbf{q} = \alpha \cdot \mathbf{J}^T \cdot \mathbf{e}$$

轉置法。不需求反矩陣，計算極為簡單。

這個作法看似隨便，其實有堅實的理論基礎： $\mathbf{J}^T \cdot \mathbf{e}$ 恰好是誤差平方和 $\frac{1}{2} \|\mathbf{e}\|^2$ 對 $\mathbf{q}$ 的負梯度方向。因此這實際上是**梯度下降法**——沿著讓誤差下降最快的方向走。

優點是計算成本極低、在奇異點附近完全穩定（因為不需求逆）。缺點是收斂慢（可能需要數百次迭代），且步長 α 需要仔細調整——太大會震盪、太小則收斂太慢。它適合對即時性要求不高、或作為其他方法的備援。

10.7.2 偽逆法

當關節數 n 不等於任務維度 m 時， $\mathbf{J}$ 是 $m \times n$ 的非方陣，沒有一般意義的反矩陣，此時使用 **Moore-Penrose 偽逆**：

$$\mathbf{J}^+ = \mathbf{J}^T (\mathbf{J} \mathbf{J}^T)^{-1} \quad (\text{當 } n > m, \text{ 即冗餘情形})$$

偽逆給出的是「範數最小」的解，即在所有能達成目標的關節速度中，選擇最小的那一組。

偽逆法收斂快（二次收斂），是數值逆運動學的標準方法。但它有一個致命弱點：**在奇異點附近， $(\mathbf{J} \mathbf{J}^T)$ 接近奇異，其反矩陣的元素會趨於無窮大，導致算出的 $\Delta \mathbf{q}$ 極大——機器人會突然高速甩動。這在實體機器上極其危險。**

此外，冗餘機器人的偽逆解還有一個優雅的擴充。由於零空間的存在，可以在不影響末端運動的前提下加入次要目標：

$$\Delta \mathbf{q} = \mathbf{J}^+ \cdot \mathbf{e} + (\mathbf{I} - \mathbf{J}^+ \mathbf{J}) \cdot \mathbf{z}$$

第二項是零空間投影， $\mathbf{z}$ 可以是任何次要目標（如遠離關節極限、避開障礙、維持舒適姿勢）。這正是第 4.9.2 節所說冗餘四大好處的數學實現，第 12 章會詳細展開。

10.7.3 阻尼最小平方法 (DLS)

為了解決偽逆在奇異點附近爆炸的問題，Wampler 與 Nakamura 分別於 1986 年提出阻尼最小平方方法，在求逆時加入一個阻尼項：

$$\Delta \mathbf{q} = \mathbf{J}^T (\mathbf{J} \mathbf{J}^T + \lambda^2 \mathbf{I})^{-1} \cdot \mathbf{e}$$

λ 為阻尼係數。加入 $\lambda^2 \mathbf{I}$ 後，即使 $\mathbf{J} \mathbf{J}^T$ 奇異，整個矩陣仍然可逆，數值不會爆炸。

這個方法的物理意義是：它不再追求「完全消除誤差」，而是在「減少誤差」與「限制關節運動量」之間取得平衡。數學上，它最小化的目標函數是：

$$\min ||\mathbf{J} \cdot \Delta \mathbf{q} - \mathbf{e}||^2 + \lambda^2 ||\Delta \mathbf{q}||^2$$

第一項要求接近目標、第二項懲罰過大的關節運動。 λ 越大越保守（運動平緩但收斂慢）， λ 越小越積極（收斂快但接近奇異點時危險）。

實務上常用**可變阻尼**策略：平常用很小的 λ （接近偽逆的效能），偵測到接近奇異點時自動增大 λ （犧牲精度換取安全）。判斷是否接近奇異點的指標，通常是雅可比矩陣的最小奇異值（第 12 章）。

表 10-4 三種數值方法的比較

方法	修正公式	收斂速度	奇異點附近	計算成本	適用場合
Jacobian 轉置	$\alpha \cdot \mathbf{J}^T \cdot \mathbf{e}$	慢（線性）	穩定	極低	即時性要求低、教學示範
偽逆	$\mathbf{J}^+ \cdot \mathbf{e}$	快（二次）	爆炸（危險）	中	遠離奇異點的一般情形
阻尼最小平方（DLS）	$\mathbf{J}^T (\mathbf{J} \mathbf{J}^T + \lambda^2 \mathbf{I})^{-1} \cdot \mathbf{e}$	中～快	穩定（犧牲精度）	中	實務首選，尤其含奇異點的軌跡

◎ 為什麼 DLS 是實務首選

在真實機器人上，軌跡經過或接近奇異點是常態而非例外（例如手臂伸直去拿遠處的物件）。偽逆法在這種情況下會產生數百度每秒的關節速度指令，輕則觸發過速保護、重則損壞機構或傷人。

DLS 犧牲的只是那一小段路徑的精度（末端會稍微偏離理想路徑），換來的是全程的安全與穩定。這是典型的工程權衡——完美的數學解，未必是最好的工程解。

10.8 最佳化觀點與額外約束

10.8.1 把逆運動學寫成最佳化問題

數值逆運動學可以更一般地表述為一個最佳化問題：

$$\mathbf{q}^* = \operatorname{argmin}_{\mathbf{q}} \|\mathbf{f}(\mathbf{q}) - \mathbf{x}_{\text{目標}}\|_C^2 + \|\mathbf{q} - \mathbf{q}_0\|_W^2$$

第一項是任務誤差（以矩陣 C 加權，表示各方向的重要性）、第二項是正則項（懲罰偏離初始姿態 $\mathbf{q}_0$ ，以矩陣 W 加權）。

這個形式的威力在於它的**可擴充性**。任何額外的需求，都可以化為目標函數中的一項或一個約束條件：

- **關節極限**：加入不等式約束 $\mathbf{q}_{\min} \leq \mathbf{q} \leq \mathbf{q}_{\max}$ ，或在目標函數中加入接近極限時急遽上升的懲罰項。
- **避障**：加入「與障礙物距離大於安全值」的約束，或加入距離越近懲罰越大的項（這與第 26 章的人工勢場法相通）。
- **姿勢偏好**：第二項的 $\mathbf{q}_0$ 可設為「自然姿勢」，使解在不影響任務的前提下傾向於舒適姿態。這對人形與協作機器人特別重要。
- **多任務優先權**：不同的任務給予不同的權重 C ，重要的任務權重高、次要的任務權重低。第 12 章的分層逆運動學是其嚴格化版本。

10.8.2 關節極限的處理

關節極限是最常見也最必要的約束。三種處理方式各有優劣：

方式一：直接截斷（clamping）。每次迭代後把超限的關節拉回邊界。實作最簡單，但可能導致收斂停滯——關節被卡在邊界上，而演算法仍不斷推它往外。

方式二：加權迴避。當某關節接近極限時，增大其在 W 矩陣中的權重，使演算法傾向於少動它、多動其他關節。這是較優雅的作法，需要冗餘自由度才有效。

方式三：零空間投影。對冗餘機器人，利用 10.7.2 節的零空間項，把「遠離關節極限」設為次要目標。這樣主任務完全不受影響，次要目標在剩餘的自由度中被最佳化。

10.8.3 姿態誤差的表示

最後一個實務要點：位置誤差是簡單的向量相減，但**姿態誤差該怎麼表示**？這是初學者實作時最常卡住的地方。

錯誤的作法是把 RPY 角相減——這在角度跨越 $\pm 180^\circ$ 時會產生巨大的假誤差（ 179° 與

−179° 實際只差 2°，相減卻得到 358°)。

正確的作法有兩種。**軸角法**：計算 $R_{\text{error}} = R_{\text{current}}^T \cdot R_{\text{target}}$ ，再由此矩陣取出軸角表示（第 8.10.3 節），得到的 $\theta \cdot k$ 即為三維的姿態誤差向量，其大小恰為需要旋轉的角度。**四元數法**：計算 $q_{\text{error}} = q_{\text{current}} * \otimes q_{\text{target}}$ ，取其向量部（乘以 2）作為誤差向量，並注意處理雙重覆蓋（若純量部為負則取 $-q_{\text{error}}$ ）。

兩種方法都能給出「大小正確、方向正確」的姿態誤差向量，且在任何姿態下都不會有跳變問題。這也再次印證了第 8 章的結論——內部計算應使用四元數或旋轉矩陣，而非歐拉角。

10.9 封閉解與數值解的選擇

10.9.1 效能與特性的比較

兩類方法的差異，可以用一張表清楚呈現：

表 10-5 封閉解與數值解的全面比較

面向	封閉解	數值解
適用機構	僅滿足 Pieper 準則者	任意機構
求解時間	微秒等級，固定	毫秒等級，不固定
解的完整性	求出全部解（如 8 組）	只得一組（依初始猜測）
收斂保證	必定得解或判定無解	可能不收斂
奇異點附近	解合併，但數值穩定	需 DLS 才穩定
額外約束	難以納入	容易納入（最佳化框架）
冗餘機器人	無法處理（解無限多）	天然支援（零空間）
開發成本	高（需逐機構推導）	低（通用程式碼）
典型使用者	工業機器人控制器	研究、模擬、冗餘與行動操作

10.9.2 實務上的混合策略

成熟的機器人系統往往結合兩者。常見的混合策略有三種：

策略一：封閉解為主、數值解為輔。對標準六軸手臂使用封閉解求出八組解，再用簡單的規則篩選；只有在加裝了第七軸或外部行走軸（使系統變成冗餘）時，才切換到數值解。

策略二：封閉解提供初始猜測。先用簡化模型的封閉解得到一個近似答案，再用數值法對完

整模型（含工具偏移、標定修正）做精細收斂。這能大幅減少迭代次數。

策略三：分層處理。位置用封閉解（快），姿態與額外約束用數值最佳化（靈活）。這在視覺伺服與力控制中很常見。

10.9.3 現代趨勢：學習式逆運動學

近年出現第三條路線：用神經網路直接學習 $x \rightarrow q$ 的映射。訓練時用正向運動學大量產生 (q, x) 資料對（這是免費且無限的資料，因為正向運動學計算極快），再訓練網路做反向預測。

優點是推論極快（一次前向傳播）、天然處理冗餘、且能學到「人類覺得自然」的姿勢偏好（若訓練資料來自人類示範）。缺點是精度通常不如傳統方法（需要後接數值修正）、且缺乏收斂保證與可解釋性。目前多用於**提供高品質的初始猜測**，而非完全取代傳統方法。第 31 章與第 33 章會回到這個主題，討論學習如何與傳統運動學互補。

10.10 軌跡上的逆運動學

10.10.1 連續求解的三個要求

實務上很少只求解單一個點——通常是對一整條軌跡上的數百個點連續求解。這帶來三個額外要求。

要求一：構型連續性。整條軌跡應使用同一組構型分支。若中途由「肘上」切換到「肘下」，手臂必須做一次大幅度的翻轉動作——耗時、危險，且末端會偏離預定路徑。作法是：每個點都選擇離前一點最近的解。

要求二：關節速度連續。相鄰兩點的關節角度差應平順變化。若某處出現跳變，代表可能跨越了奇異點或切換了構型，應在執行前偵測並處理。

要求三：全程可達。應在執行前對整條軌跡預先求解一遍（離線驗證），確認每一點都有解且不超出關節極限。工業機器人的程式在下載到控制器時，通常會做這種預檢。

10.10.2 奇異點穿越問題

若軌跡必須經過或接近奇異點，會發生第 12 章要詳述的問題：某些關節需要以極高的速度轉動。處理方式有四種：

- **繞道：**修改路徑，讓它避開奇異點附近的區域。這是最安全的作法，但末端路徑會與原始規

劃有偏差。

- **減速**：接近奇異點時自動降低末端速度，使關節速度維持在限制內。這保持了路徑精度，但節拍會拉長。
- **DLS 阻尼**：使用 10.7.3 節的阻尼法，容許末端稍微偏離路徑以換取關節速度的可控。
- **改變構型或基座位置**：若機器人裝在行走軸上，可移動基座使目標落在較「舒服」的區域。

工業控制器通常內建這些機制，並在觸發時向操作員發出警告（如「接近奇異點，速度已降低」）。理解這些訊息的意義，是機器人程式設計師的基本素養。

10.11 由位置邁向速度

10.11.1 本章的成果與遺留問題

本章完成了機器人運動學的第二塊拼圖。你現在能夠：判斷目標是否可達、用幾何與代數法求出平面手臂的封閉解、理解六軸手臂的八組解結構與解耦原理、實作數值迭代求解、並用 DLS 處理奇異點附近的數值問題。

但本章刻意留下了一個未解的環節：**雅可比矩陣 J 究竟是什麼？怎麼算出來？**我們一直把它當成黑盒子使用，只知道它把關節速度映射到末端速度。這正是下一章的主題。

10.11.2 接下來：雅可比與奇異點

第 11 章：雅可比矩陣。我們將從頭推導 J 的每一個元素，理解它的兩種形式（幾何雅可比與解析雅可比），並發現一個美妙的對偶性——同一個矩陣不只連結速度，還連結**力與力矩**（ J^T 把末端受力映射到關節扭矩）。這使雅可比成為連接運動學與力學的橋樑，也是第 13 章動力學與第 20 章力控制的基礎。

第 12 章：奇異點與冗餘。本章多次提到「奇異點」——手臂完全伸直、兩軸對齊、解合併、數值爆炸。第 12 章將用雅可比矩陣的**秩與奇異值分解**，為這些現象給出統一的數學解釋，並說明冗餘機器人如何利用零空間繞開它們。

換句話說：第 9 章算出手在哪裡、第 10 章決定手該怎麼擺、第 11 章計算手能怎麼動、第 12 章解釋手何時動不了。四章合起來，才是完整的操作臂運動學。

完整計算例題

例題 10-1 平面 2R 手臂的完整逆解

題目：某平面 2R 手臂 $L_1 = 300 \text{ mm}$ 、 $L_2 = 200 \text{ mm}$ （與例題 9-1 相同）。目標末端位置為 $(311.57, 343.19) \text{ mm}$ 。（a）判斷是否可達。（b）求出所有解。（c）驗證其中一組解，並說明兩組解的機構意義。

解（a）：先求目標到基座的距離：

$$r = \sqrt{(311.57^2 + 343.19^2)} = \sqrt{(97076 + 117779)} = \sqrt{214855} \approx 463.52 \text{ mm}$$

目標距離。

可達範圍為 $|300 - 200| = 100 \leq r \leq 300 + 200 = 500$ 。463.52 落在此範圍內，目標可達。

解（b）：先求 θ_2 。代入 10.3.1 節的公式：

$$\begin{aligned} \cos \theta_2 &= (r^2 - L_1^2 - L_2^2) / (2 L_1 L_2) = (214855 - 90000 - 40000) / (2 \times 300 \times 200) \\ &= 84855 / 120000 = 0.70713 \end{aligned}$$

由於 $|0.70713| < 1$ ，確認有實數解。

$$\theta_2 = \pm \arccos(0.70713) = \pm 45.00^\circ$$

兩組解： $\theta_2 = +45^\circ$ （肘下）與 $\theta_2 = -45^\circ$ （肘上）。

接著求對應的 θ_1 。目標方位角為 $\text{atan2}(343.19, 311.57) = 47.76^\circ$ 。

解一（ $\theta_2 = +45^\circ$ ）：

$$\begin{aligned} \theta_1 &= 47.76^\circ - \text{atan2}(200 \sin 45^\circ, 300 + 200 \cos 45^\circ) \\ &= 47.76^\circ - \text{atan2}(141.42, 441.42) = 47.76^\circ - 17.76^\circ = 30.00^\circ \end{aligned}$$

第一組解： $(\theta_1, \theta_2) = (30^\circ, 45^\circ)$ 。

解二（ $\theta_2 = -45^\circ$ ）：

$$\begin{aligned} \theta_1 &= 47.76^\circ - \text{atan2}(200 \sin(-45^\circ), 300 + 200 \cos(-45^\circ)) \\ &= 47.76^\circ - \text{atan2}(-141.42, 441.42) = 47.76^\circ - (-17.76^\circ) = 65.52^\circ \end{aligned}$$

第二組解： $(\theta_1, \theta_2) = (65.52^\circ, -45^\circ)$ 。

解（c）：驗證解一。用第 9 章的正向運動學代回：

$$\begin{aligned} x &= 300 \cos 30^\circ + 200 \cos 75^\circ = 259.81 + 51.76 = 311.57 \checkmark \\ y &= 300 \sin 30^\circ + 200 \sin 75^\circ = 150.00 + 193.19 = 343.19 \checkmark \end{aligned}$$

與目標完全吻合，且正是例題 9-1 的輸入角度——正向與逆向互為反運算，驗證成功。

機構意義：解一（ $30^\circ, 45^\circ$ ）的肘關節位於基座與末端連線的下方；解二（ $65.52^\circ, -45^\circ$ ）

的肘關節位於上方。兩者末端位置相同、末端姿態不同 (φ 分別為 75° 與 20.52°) ——這說明平面 2R 手臂無法獨立控制位置與姿態 (只有兩個自由度卻有三個輸出量，見第 9.6.1 節)。

例題 10-2 工作空間邊界與無解判定

題目：同上手臂。(a) 目標 (480, 200) 是否可達？(b) 目標 (500, 0) 的解為何？有幾組？(c) 目標 (50, 50) 是否可達？

解 (a) : $r = \sqrt{(480^2 + 200^2)} = \sqrt{(230400 + 40000)} = \sqrt{270400} = 520.00 \text{ mm}$ 。超過最大可達距離 500 mm，不可達。

若強行代入公式： $\cos \theta_2 = (270400 - 130000)/120000 = 1.1700 > 1$ ， $\arccos$ 無實數解。程式必須在此處攔截並回報錯誤，而非讓 NaN 傳遞下去——這正是開頭故事中「手臂撞爛」的根源。

解 (b) : $r = 500 \text{ mm}$ ，恰為最大可達距離。

$$\cos \theta_2 = (250000 - 130000)/120000 = 1.0000 \Rightarrow \theta_2 = 0^\circ$$

只有一個解 ($\pm 0^\circ$ 相同)，兩組解合而為一。

$\theta_1 = \text{atan2}(0, 500) - \text{atan2}(0, 500) = 0^\circ$ 。因此唯一解為 $(0^\circ, 0^\circ)$ ，手臂完全伸直。這是邊界奇異點：在此處，手臂失去了沿徑向運動的能力 (無論關節怎麼轉，末端都無法再往外)，且逆解由兩組退化為一組。第 12 章會用雅可比矩陣的秩來精確描述這個現象。

解 (c) : $r = \sqrt{(2500 + 2500)} = \sqrt{5000} \approx 70.71 \text{ mm}$ 。小於最小可達距離 100 mm，不可達——這是工作空間中央的「空洞」(第 4.10.3 節)。 $\cos \theta_2 = (5000 - 130000)/120000 = -1.0417 < -1$ ，同樣無實數解。

工程結論：完整的可達性檢查必須同時檢驗上下界。實務程式應在計算 $\arccos$ 之前先檢查其引數是否落在 $[-1, 1]$ ，並依 10.2.2 節的四種方式處置。

例題 10-3 數值逆運動學的迭代過程

題目：以平面 2R 手臂 ($L_1 = 300$ 、 $L_2 = 200$) 為例，目標 (311.57, 343.19)，初始猜測 $q_0 = (0^\circ, 0^\circ)$ 。以偽逆法、步長 $\alpha = 1.0$ 進行迭代，說明前兩次迭代的計算過程。(平面 2R 的雅可比矩陣為下式，第 11 章將推導其來源。)

$$J = \begin{bmatrix} -L_1 s_1 & -L_2 s_{12} & -L_2 s_{12} \end{bmatrix}$$

$$\begin{bmatrix} L_1 c_1 + L_2 c_{12} & L_2 c_{12} \end{bmatrix}$$

平面 2R 手臂的雅可比矩陣 (僅位置部分, 2×2)。

迭代一： $q = (0^\circ, 0^\circ)$ ，正向運動學得 $f(q) = (500, 0)$ 。

$$e = (311.57, 343.19) - (500, 0) = (-188.43, 343.19)$$

殘差向量，其範數約 391.5 mm。

在 $q = (0, 0)$ 處， $s_1 = s_{12} = 0$ 、 $c_1 = c_{12} = 1$ ，故：

$$J = \begin{bmatrix} 0 & 0 \\ 500 & 200 \end{bmatrix}$$

此處 J 的第一列全為零——矩陣奇異！

這正是例題 10-2(b) 所揭示的邊界奇異點。 J 不可逆，偽逆法在此**無法計算**。這是數值方法的一個重要教訓：**初始猜測不可選在奇異點上**。改用 DLS ($\lambda = 10$) 或改變初始猜測即可解決。

改用初始猜測 $q_0 = (10^\circ, 10^\circ)$ ： $f(q) = 300\cos 10^\circ + 200\cos 20^\circ = 295.44 + 187.94 = 483.38$ ；
 $y = 300\sin 10^\circ + 200\sin 20^\circ = 52.09 + 68.40 = 120.49$ 。

$$e = (311.57 - 483.38, 343.19 - 120.49) = (-171.81, 222.70)$$

殘差範數約 281.3 mm。

計算 J 並求解 $\Delta q = J^{-1} \cdot e$ ，可得修正量 (此處省略數值細節)，更新後的 q 會朝正確方向移動。實際執行本程式可觀察到：約 5 至 8 次迭代後，殘差降至 0.01 mm 以下，收斂到解 $(30^\circ, 45^\circ)$ 或 $(65.52^\circ, -45^\circ)$ ——收斂到哪一組，取決於初始猜測落在哪一組解的「吸引域」中。

工程啟示：這個例題展示了數值法的三個實務要點——一、初始猜測極為關鍵；二、奇異點會使偽逆法失效，DLS 是必要的保險；三、數值法只給一組解，若需要全部解必須多重起始。

程式範例：封閉解與數值解的實作與比較

以下 Python 程式同時實作平面 2R 手臂的封閉解與三種數值解法，並比較其精度、迭代次數與在奇異點附近的行為。

```
import numpy as np

L1, L2 = 300.0, 200.0          # 連桿長度 (mm)

# ----- 正向運動學 -----
def fk(q):
```

```

t1, t2 = q
x = L1*np.cos(t1) + L2*np.cos(t1+t2)
y = L1*np.sin(t1) + L2*np.sin(t1+t2)
return np.array([x, y])

# ----- 雅可比矩陣 (第 11 章將推導) -----
def jacobian(q):
    t1, t2 = q
    s1, c1 = np.sin(t1), np.cos(t1)
    s12, c12 = np.sin(t1+t2), np.cos(t1+t2)
    return np.array([
        [-L1*s1 - L2*s12, -L2*s12],
        [ L1*c1 + L2*c12,  L2*c12]
    ])

# ----- 封閉解 (10.3 節) -----
def ik_closed_form(target):
    x, y = target
    r2 = x*x + y*y
    r = np.sqrt(r2)

    # 可達性檢查 (10.2.1 節)
    if r > L1 + L2 + 1e-9:
        return [], f"不可達：距離 {r:.2f} > 最大 {L1+L2:.2f} mm"
    if r < abs(L1 - L2) - 1e-9:
        return [], f"不可達：距離 {r:.2f} < 最小 {abs(L1-L2):.2f} mm"

    c2 = (r2 - L1*L1 - L2*L2) / (2*L1*L2)
    c2 = np.clip(c2, -1.0, 1.0) # 防止浮點誤差造成 arccos 失效
    solutions = []
    for sign in (+1, -1): # 肘下 / 肘上兩組解
        t2 = sign * np.arccos(c2)
        t1 = np.arctan2(y, x) - np.arctan2(L2*np.sin(t2), L1 + L2*np.cos(t2))
        solutions.append(np.array([t1, t2]))
        if abs(np.sin(t2)) < 1e-9: # 奇異點：兩解合一
            return [solutions[0]], "邊界奇異點：兩組解合而為一"
    return solutions, "OK"

# ----- 數值解：三種方法 -----
def ik_numerical(target, q0, method="dls", lam=10.0,
                  alpha=1.0, tol=1e-3, max_iter=200):
    q = np.array(q0, dtype=float)
    for k in range(max_iter):
        e = target - fk(q)
        if np.linalg.norm(e) < tol:
            return q, k, True
        J = jacobian(q)
        if method == "transpose":
            dq = J.T @ e * 1e-6 # 梯度下降需極小步長
        elif method == "pinv":
            dq = np.linalg.pinv(J) @ e
        else:
            # DLS (10.7.3 節)
            dq = J.T @ np.linalg.solve(J @ J.T + lam**2*np.eye(2), e)

```

```

    q = q + alpha * dq
    return q, max_iter, False

d, r2d = np.deg2rad, np.rad2deg

# ----- 驗證一：例題 10-1 -----
print("=== 例題 10-1：封閉解 ===")
target = np.array([311.57, 343.19])
sols, msg = ik_closed_form(target)
print(f"狀態: {msg}")
for i, s in enumerate(sols, 1):
    name = "肘下" if s[1] > 0 else "肘上"
    print(f" 解{i}({name}):  $\theta_1={r2d(s[0]):7.2f}^\circ$   $\theta_2={r2d(s[1]):7.2f}^\circ$ "
          f" 驗算  $fk={np.round(fk(s),2)}$ ")

# ----- 驗證二：例題 10-2 無解與奇異點 -----
print("\n=== 例題 10-2：可達性判定 ===")
for tgt in ([480, 200], [500, 0], [50, 50]):
    sols, msg = ik_closed_form(np.array(tgt, dtype=float))
    print(f" 目標 {tgt}: {msg} → 解的數目 = {len(sols)}")

# ----- 驗證三：三種數值方法比較 -----
print("\n=== 例題 10-3：數值方法比較 ( 初始猜測  $10^\circ, 10^\circ$  ) ===")
q0 = [d(10), d(10)]
for m in ("transpose", "pinv", "dls"):
    q, it, ok = ik_numerical(target, q0, method=m)
    err = np.linalg.norm(target - fk(q))
    print(f"  {m:10s}: 迭代 {it:3d} 次 收斂={ok} "
          f" 誤差={err:8.4f} mm 解=({r2d(q[0]):6.2f}°, {r2d(q[1]):6.2f}°)")

# ----- 驗證四：奇異點上的初始猜測 -----
print("\n=== 奇異點初始猜測 ( $0^\circ, 0^\circ$ ) 的行為 ===")
for m in ("pinv", "dls"):
    q, it, ok = ik_numerical(target, [0.0, 0.0], method=m)
    err = np.linalg.norm(target - fk(q))
    print(f"  {m:10s}: 迭代 {it:3d} 次 收斂={ok} 誤差={err:10.4f} mm")
print(" → 偽逆法在奇異點上失效，DLS 仍能脫離並收斂")

# ----- 應用：軌跡上的連續求解 ( 10.10 節 ) -----
print("\n=== 軌跡連續求解：以前一點的解作為初始猜測 ===")
q = np.array([d(30), d(45)])
for i, ang in enumerate(np.linspace(0, np.pi/2, 5)):
    tgt = np.array([400*np.cos(ang) + 50, 400*np.sin(ang) + 50])
    q, it, ok = ik_numerical(tgt, q, method="dls")
    print(f" 點{i}: 目標=({tgt[0]:6.1f},{tgt[1]:6.1f}) "
          f" 迭代={it:2d}  $\theta=({r2d(q[0]):6.2f}^\circ, {r2d(q[1]):6.2f}^\circ)$ ")
print(" → 迭代次數少且構型連續，未發生肘上/肘下切換")

```


請試著修改：（一）把 DLS 的 λ 由 10 改為 0.1 與 100，觀察收斂速度與穩定性的變化。（二）把軌跡連續求解改成每次都用固定初始猜測 $(0^\circ, 0^\circ)$ ，觀察是否會發生構型跳變。（三）在 `ik_closed_form` 中移除 `np.clip`，並輸入 $(480, 200)$ ，觀察 NaN 如何在系統中傳播——這正是實務上必須做可達性檢查的原因。

實作：手臂逆運動學驗證

本章實作是為實體手臂實作逆運動學並驗證其精度。建議使用附錄 C~E 的手臂，或第 9 章實作中已建立運動學模型的任一支手臂。

- 沿用第 9 章的模型：**取用你在第 9 章建立的 DH 參數表與正向運動學程式。
- 實作封閉解：**若手臂為平面型或三軸型，依 10.3 節推導其封閉解；若構型複雜，直接進入下一步。
- 實作數值解：**以本章程式為基礎，實作 DLS 數值逆運動學。雅可比矩陣可先用數值微分近似（對每個關節加一個微小擾動，觀察末端位置變化量除以擾動量），第 11 章學會解析法後再替換。
- 單點驗證：**選定一個工作空間內的目標點，求解後把角度送給手臂，用尺量測實際末端位置，記錄誤差。
- 多解觀察：**若手臂有多組解，分別執行不同構型的解，拍照記錄兩種姿勢，並觀察哪一組會發生碰撞或超出關節極限。
- 邊界測試：**逐步把目標點往外移，直到程式回報不可達。記錄實際的最大可達距離，並與理論值比較（差異來自關節極限與機構干涉）。
- 軌跡測試：**規劃一段直線或圓弧軌跡（取 20~50 個點），連續求解並執行。觀察是否發生構型跳變或速度突變，並記錄整段軌跡的最大誤差。
- 誤差來源分析：**將誤差分解為運動學模型誤差（第 9.9 節）、逆解收斂容差、以及機構背隙與剛性三部分，估算各自的貢獻。

表 10-6 逆運動學驗證記錄表（第一列為範例）

測試項	目標位置	求得關節角	實測位置	誤差	備註
單點（工作空間中央）	(250,	(28.4°,	(247,	4.2	肘下構型，正

測試項	目標位置	求得關節角	實測位置	誤差	備註
	150)	62.1°)	153)	mm	常
(自行填寫：多解、邊界、軌跡各項)					

國中小也看得懂：伸手拿東西的三個麻煩

伸手去拿桌上的橡皮擦——很簡單對吧？但你有沒有想過，你的手臂是怎麼「決定」肩膀要轉幾度、手肘要彎幾度的？你從來沒有算過，卻每次都能拿到。

機器人可沒這麼幸運，它必須用數學算出來。而且這個計算有三個麻煩的地方：

第一個麻煩：答案不只一個。 試試看——把手掌放在橡皮擦上不要動，然後把手肘抬高、再放低。手掌一直在同一個地方，手臂卻可以有好多種姿勢！機器人也一樣，它必須決定「要用哪一種姿勢過去」。有時候某一種姿勢會撞到桌子，就要換另一種。

第二個麻煩：有時候根本拿不到。 把橡皮擦推到桌子最遠的地方，你伸長手臂還是構不到。這時候人會說「我拿不到」然後放棄，但機器人如果沒寫好程式，它可能會硬要伸過去，結果就是撞壞自己。所以工程師一定要先幫機器人檢查：「這個地方我到得了嗎？」

第三個麻煩：算起來很難。 從角度算位置很簡單（第 9 章學的），但反過來從位置算角度，就像解一道很複雜的謎題。有些機器人的形狀設計得很聰明，可以直接套公式；有些就只能用「猜一猜、修一修、再猜一猜」的方法，一直逼近正確答案。你在附錄 J 做的機器狗，每走一步都要算四隻腳的角度，用的就是這一章的方法喔！

叫獸提醒與常見錯誤

- 誤區一：不做可達性檢查就求解。 $\arccos$ 的引數超出 $[-1, 1]$ 會產生 NaN，而 NaN 一旦傳到馬達指令就是災難。務必在計算前檢查並攔截。
- 誤區二：用 $\arcsin$ 或 $\arccos$ 求角度而非 $\operatorname{atan2}$ 。前兩者會遺失象限資訊，導致解在特定區域錯誤。只要 $\sin$ 與 $\cos$ 都已知，就該用 $\operatorname{atan2}$ 。
- 誤區三：忽略多解的存在。求出一組解就直接執行，可能得到會碰撞或超出關節極限的姿勢。應求出全部解再篩選。

- 誤區四：軌跡上每點都用固定初始猜測。這會導致構型跳變（肘上突然變肘下），手臂大幅甩動。應以前一點的解作為下一點的初始猜測。
- 誤區五：用偽逆法而不加阻尼。接近奇異點時會產生極大的關節速度指令，實體機器上極其危險。實務應使用 DLS。
- 誤區六：把姿態誤差用 RPY 角相減。角度跨越 $\pm 180^\circ$ 時會產生巨大的假誤差。應改用軸角或四元數表示姿態誤差。
- 誤區七：以為數值解一定會收斂。初始猜測不佳、目標不可達或落在奇異點上，都可能不收斂。必須設定迭代上限並處理失敗情形。
- 誤區八：忘記工具偏移。逆解求的通常是法蘭位姿，若程式給的是 TCP 目標，必須先扣除工具變換（第 8.9.1 節）。
- 誤區九：數值解的容差設得太嚴。要求 $1e-12$ 的精度會導致迭代次數暴增甚至無法收斂；實務上位置容差取 $0.01 \sim 0.1 \text{ mm}$ 即足夠。
- 誤區十：以為「數學上可達」就是「實際可達」。還須通過關節極限、自我碰撞與環境碰撞三重檢查。

課堂討論

1. 開頭故事中，小好用兩種姿勢拿到同一個杯子。若你是控制器的設計者，會用什麼準則來決定選哪一組解？請列出至少三個準則並排出優先順序。
2. 六軸手臂有八組解，但工業控制器通常只會執行其中一組。若操作員抱怨「機器人用了一個奇怪的姿勢過去」，你會如何診斷與處理？
3. DLS 犧牲末端精度來換取關節速度的可控。請設計一個情境說明這個取捨是值得的，再設計一個情境說明它可能造成問題。
4. 學習式逆運動學（用神經網路）目前主要用於提供初始猜測，而非完全取代傳統方法。請從精度、收斂保證、可解釋性與安全性四個角度討論原因。
5. 冗餘機器人的逆解有無限多組。這聽起來像是麻煩，但第 4.9.2 節說它是優點。請說明「無限多解」如何從問題轉變為工具，並舉出兩個實際應用。

章末摘要

- 逆向運動學由末端位姿求關節變數，是機器人實際作業時真正需要回答的問題。
- 與正向運動學的三個本質差異：解可能不存在、可能不唯一、可能沒有封閉解。
- 求解前必須做可達性檢查；平面 2R 的可達條件為 $|L_1 - L_2| \leq r \leq L_1 + L_2$ ，超出範圍時 $\arccos$ 的引數會超出 $[-1, 1]$ 。
- 無解時的四種處置：拒絕報錯、投影到最近可達點、放寬姿態要求、移動基座。
- 實務上「假不可達」的三大原因：關節極限、自我碰撞、姿態要求過嚴。
- 平面 2R 的封閉解由餘弦定理求 θ_2 ($\pm$ 兩解，對應肘上 / 肘下)，再由 atan2 求 θ_1 。
- 代數解法的核心策略是「逐步剝離」——兩邊同乘變換的反矩陣，使未知數單獨出現。
- 多解來自推導中的開方或反餘弦分岔；六軸手臂三次分岔產生 $2^3 = 8$ 組解，對應肩前後、肘上下、腕翻轉。
- Pieper 準則：三個相鄰軸交於一點或互相平行時，存在封閉解；球形手腕即為此設計。
- 運動學解耦把六自由度問題拆為位置（前三軸）與姿態（後三軸）兩個三自由度子問題。
- 數值法以 $\Delta q = J^+ \cdot e$ 迭代逼近，內部反覆呼叫正向運動學；初始猜測決定收斂到哪一組解。
- 三種數值方法：轉置法（穩定但慢）、偽逆法（快但奇異點爆炸）、DLS（實務首選，以阻尼換穩定）。
- 冗餘機器人可用零空間投影 $(I - J^+J) \cdot z$ 加入次要目標，不影響主任務。
- 姿態誤差不可用 RPY 相減，應以軸角或四元數表示，避免 $\pm 180^\circ$ 跳變。
- 軌跡連續求解須維持構型連續性、關節速度連續與全程可達；以前一點的解作為初始猜測是標準作法。
- 封閉解快速完整但機構受限，數值解通用靈活但較慢；實務常採混合策略。

觀念題與計算題

觀念題

1. 說明逆向運動學與正向運動學的三個本質差異，並各舉一個實例。
2. 為何逆向運動學可能無解？列出四種無解時的處置方式與各自適用的場合。
3. 說明實務上「假不可達」的三個常見原因，並各提出一種診斷方法。
4. 以平面 2R 手臂說明兩組解的幾何意義，並列出三種選解準則。

5. 為何求解角度時必須使用 atan2 而非 $\arcsin$ 或 $\arccos$?
6. 說明代數解法「逐步剝離」的策略，並寫出其關鍵的矩陣運算式。
7. 六軸手臂的八組解由哪三次分岔產生？各對應什麼機構意義？
8. 敘述 Pieper 準則，並說明球形手腕為何是刻意的設計決策。
9. 說明運動學解耦的核心洞見，以及手腕中心點如何由末端位姿求得。
10. 推導數值逆運動學的迭代公式，並說明雅可比矩陣在其中的角色。
11. 比較 Jacobian 轉置法、偽逆法與 DLS 的收斂速度、奇異點行為與適用場合。
12. 說明 DLS 中阻尼係數 λ 的物理意義，以及可變阻尼策略的作法。
13. 冗餘機器人的零空間投影項 $(I - J^+J) \cdot z$ 有什麼作用？舉兩個 z 的具體選擇。
14. 為何姿態誤差不可用 RPY 角相減？請說明正確的兩種表示方式。
15. 軌跡上連續求解有哪三個要求？為何應以前一點的解作為初始猜測？

計算題

1. 某平面 2R 手臂 $L_1 = 400 \text{ mm}$ 、 $L_2 = 250 \text{ mm}$ 。目標位置為 $(300, 400) \text{ mm}$ 。(a) 判斷可達性。(b) 求出全部解（角度以度為單位，取至小數點後兩位）。(c) 分別計算兩組解的末端姿態角 φ ，並說明為何不同。(d) 若目標改為 $(600, 300)$ ，是否可達？請說明。
2. 同上手臂。(a) 求工作空間的內外半徑。(b) 當目標恰在外邊界時，有幾組解？此時 θ_2 為何？(c) 當目標恰在內邊界時， θ_2 為何？此姿態在實體上可能有什麼問題？(d) 若把 L_2 改為 400 mm （兩連桿等長），內邊界半徑變為多少？這帶來什麼好處與代價？
3. 某六軸手臂的末端目標位姿為：位置 $p = (500, 200, 400) \text{ mm}$ ，姿態的旋轉矩陣第三行（接近向量 $\hat{a}$ ）為 $(0, 0, -1)^T$ 。已知手腕中心到末端的距離 $d_6 = 100 \text{ mm}$ 。(a) 求手腕中心點 p_{wrist} 的位置。(b) 說明接下來的求解步驟（不需實際計算）。(c) 若 d_6 量測錯誤（實際為 120 mm 卻用 100 mm 計算），估算末端位置的誤差量與方向。
4. 以平面 2R 手臂（ $L_1 = 300$ 、 $L_2 = 200$ ）進行數值逆運動學。目標 $(400, 100)$ ，初始猜測 $(20^\circ, 30^\circ)$ 。(a) 計算初始的正向運動學位置與殘差向量。(b) 計算此處的雅可比矩陣。(c) 以偽逆法求第一次迭代的 Δq （步長 $\alpha = 1.0$ ），並求更新後的關節角度。(d) 計算更新後的殘差範數，確認是否減小。

動手做與專題延伸

本章實作：逆運動學求解器

延續第 9 章的正向運動學程式，實作完整的逆運動學求解器。建議依下列順序完成，每一步都應通過測試再進入下一步。

- **階段一**：平面 2R 的封閉解，含完整的可達性檢查與兩組解的輸出。以第 9 章的正向運動學驗證（求解後代回，應得到原目標）。
- **階段二**：平面 3R 的封閉解。注意這是冗餘機構（三自由度、三個輸出但位置只需兩個），需先指定末端姿態角 φ ，才能求出唯一解。
- **階段三**：通用的數值 DLS 求解器，支援任意 DH 參數表。雅可比先用數值微分實作。
- **階段四**：加入關節極限處理與多重起始策略，使程式能找出多組解並篩選出可行者。

專題延伸

- **互動式逆運動學展示**：延續第 9 章的視覺化工具，改為滑鼠點選目標點，程式即時求解並繪出手臂姿態。同時顯示所有解（以不同顏色區分構型），並在目標不可達時以紅色標示工作空間邊界。
- **吸引域視覺化**：對平面 2R 手臂，在關節空間 (θ_1, θ_2) 中均勻取樣數千個初始猜測，記錄每個猜測最終收斂到哪一組解，並以兩種顏色繪出。你會看到一幅美麗的「吸引域」圖案，直觀理解為何初始猜測如此關鍵。
- **進階：三種數值方法的效能競賽**。設計一組測試案例（含遠離奇異點、接近奇異點、通過奇異點三類），對轉置法、偽逆法與不同 λ 的 DLS 分別測試，記錄迭代次數、最終誤差、最大關節速度與失敗率。將結果整理成圖表，並依此提出「在什麼情況下該用哪一種方法」的實務建議。

參考文獻與延伸閱讀

- Craig, J. J., Introduction to Robotics: Mechanics and Control, 4th ed., Pearson (第 4 章：逆向運動學，含 PUMA 的完整代數推導)。
- Pieper, D. L., "The Kinematics of Manipulators Under Computer Control," PhD Thesis, Stanford University, 1968 (Pieper 準則的原始出處)。
- Wampler, C. W., "Manipulator Inverse Kinematic Solutions Based on Vector Formulations and Damped Least-Squares Methods," IEEE Trans. SMC, 1986 (DLS 方

法)。

- Nakamura, Y. and Hanafusa, H., "Inverse Kinematic Solutions with Singularity Robustness," ASME J. Dynamic Systems, 1986 (奇異點強健的逆運動學)。
- Buss, S. R., "Introduction to Inverse Kinematics with Jacobian Transpose, Pseudoinverse and Damped Least Squares Methods," 2009 (三種數值方法的完整比較，極佳的入門講義)。
- Lynch, K. M. and Park, F. C., Modern Robotics, Cambridge University Press (第6章：逆運動學，含數值方法與冗餘處理)。
- Siciliano, B. et al., Robotics: Modelling, Planning and Control, Springer (第2、3章：逆運動學與冗餘)。
- Chiaverini, S., Oriolo, G. and Walker, I. D., "Kinematically Redundant Manipulators," in Springer Handbook of Robotics (冗餘機器人的逆運動學專章)。
- IKFast (OpenRAVE 專案) 與 TRAC-IK：兩套廣泛使用的開源逆運動學求解器，前者自動產生封閉解程式碼、後者結合數值最佳化。
- MoveIt 2 官方文件 (moveit.picknik.ai)：ROS 2 生態系中的運動規劃框架，內建多種逆運動學求解器可供比較。

第 11 章

雅可比矩陣、速度與靜力

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】一支六軸手臂末端沿桌面畫出一條等速直線（白色軌跡），軌跡上取三個位置，各疊印一個速度橢球——起點處橢球扁而傾斜、中段接近正圓、終點處被壓扁成細長狀。畫面右下角另疊一組力橢球（紅色），其形狀與同位置的速度橢球恰好「胖瘦相反」。左側示教器螢幕特寫，顯示末端速度固定為 100 mm/s，但關節速度欄位的數字劇烈跳動。標題文字：同一條直線，不斷改變的兌換率。

叫獸開講 「為什麼機器手臂在某些姿勢突然變得很笨重？」

實驗室的六軸手臂正在示範一段直線運動——末端沿著桌面平穩地由左向右滑過，速度固定在每秒 10 公分。叫獸請學生們看示教器上的關節速度顯示。

「注意第二軸與第三軸，」他說。螢幕上的數字不斷跳動：一開始第二軸每秒轉 8 度、第三軸每秒 -12 度；到了中段，變成 15 度與 -5 度；快到終點時，第二軸飆到 34 度而第三軸幾乎停住。

小翰皺眉：「末端明明是等速直線，為什麼關節速度變來變去？」

「因為它們之間不是固定的比例關係。」叫獸把手臂停在中段，「同樣是第二軸轉一度，在這個姿態下末端會移動 6 公釐；」他把手臂擺成接近伸直的姿態，「在這個姿態下，同樣轉一度，末端會移動 9 公釐。兩者的兌換率，隨姿態而變。」

他在白板上畫了一個方框：「這個兌換率，就是今天的主角——雅可比矩陣。它是一個會隨姿態改變的矩陣，把關節速度翻譯成末端速度。而它之所以重要，有三個理由。」

「第一，」他寫下數字 1，「上一章的數值逆運動學，我們一直把它當黑盒子用。今天要把它拆開。」

「第二，」他把手臂緩緩擺到完全伸直，示教器上某個關節的速度數字瞬間跳成一串驚人的大數，控制器隨即發出警報並停機。「在某些姿態下，這個兌換率會壞掉——末端只要動一點點，關節就得瘋狂旋轉。這就是奇異點，下一章的主題。」

「第三個理由最有趣。」叫獸走到手臂旁，用手推末端的夾爪。示教器上，各關節的電流讀數同步跳動。「我推的是末端，動的卻是關節的扭矩。同一個雅可比矩陣，不只連結速度，也連結力——而且是用它的轉置。這件事看起來像巧合，其實是一個深刻的對偶性，一百多年前的力學家早就發現了。」

他把筆放下：「所以雅可比不只是一個計算工具。它是機器人學裡運動學與力學之間的那座橋。過了這座橋，我們才能談動力學、談力控制、談協作機器人怎麼感覺到你碰它。今天，我們就來把這座橋一根一根搭起來。」

本章為什麼重要

雅可比矩陣是機器人學中使用頻率最高的一個矩陣。第 10 章的數值逆運動學需要它、第 12 章的奇異點分析靠它定義、第 13 章的動力學推導用到它、第 17 章的操作空間控制以它為核心、第 20 章的力控制與碰撞偵測更是直接建立在它的轉置之上。

它的價值來自一個關鍵性質：**線性化**。機器人的正向運動學是高度非線性的（充滿三角函數的乘積），難以直接分析與控制。但當我們只看「微小的變化」時，這個非線性關係就退化成一個線性映射——而這個線性映射的矩陣，就是雅可比。線性代數的所有工具（秩、奇異值、零空間、條件數）從此都能派上用場，這是機器人分析中最重要的一次降維打擊。

本章也將揭示一個貫穿全書的對偶關係：速度與力的對偶。 J 把關節速度映射到末端速度， J^T 則把末端力映射到關節扭矩。理解這個對偶，你會發現許多看似不相干的主題（運動、力覺、剛性、能量）其實都是同一枚硬幣的不同側面。

學習目標

- 能定義雅可比矩陣，並說明它作為「線性化的運動兌換率」的意義。
- 能由正向運動學以偏微分法推導平面手臂的雅可比矩陣。
- 能區分幾何雅可比與解析雅可比，並說明何時該用哪一種。
- 能以幾何法（軸向與位置向量的外積）直接寫出旋轉關節與移動關節的雅可比行。
- 能說明角速度為何不是姿態角的導數，以及這對雅可比的影響。
- 能運用虛功原理推導 $\tau = J^T \cdot F$ ，並說明速度與力的對偶性。
- 能說明雅可比在不同座標系之間的轉換方式。
- 能計算並解釋可操作度、速度橢球與力橢球的意義。
- 能說明雅可比在逆運動學、力控制與碰撞偵測中的應用。
- 能以 Python 實作解析與數值兩種雅可比計算並驗證其一致性。

先備知識

第 8 章的旋轉矩陣與齊次變換、第 9 章的正向運動學與 DH 參數、第 10 章對雅可比的初步使用。數學方面需要偏導數、向量外積、矩陣乘法與轉置的概念；11.8 節會用到特徵值與奇異值的基本觀念，但會以幾何直覺說明，不需要完整的線性代數理論。

第 5 章的扭力概念與第 8.4.2 節的反對稱矩陣（用於外積轉矩陣運算）在本章會再次出現。

核心名詞中英對照

表 11-1 本章核心名詞

中文	英文	說明
雅可比矩陣	Jacobian Matrix	關節速度到末端速度的線性映射矩陣
幾何雅可比	Geometric Jacobian	以線速度與角速度表示的雅可比
解析雅可比	Analytical Jacobian	以位置與姿態參數導數表示的雅可比
線速度	Linear Velocity	末端位置的時間導數
角速度	Angular Velocity	描述剛體轉動快慢與轉軸的向量
旋量 / 扭量	Twist	線速度與角速度合成的六維速度向量
扳手	Wrench	力與力矩合成的六維力向量
虛功原理	Principle of Virtual Work	由能量守恆推導力與運動關係的原理
對偶性	Duality	速度映射與力映射互為轉置的關係
可操作度	Manipulability	衡量機器人運動靈活程度的純量指標
速度橢球	Velocity Ellipsoid	描述各方向可達速度能力的橢球
力橢球	Force Ellipsoid	描述各方向可輸出力能力的橢球
奇異值分解	Singular Value Decomposition, SVD	將矩陣分解為旋轉—縮放—旋轉的方法
條件數	Condition Number	最大與最小奇異值之比，衡量數值狀況
數值微分	Numerical Differentiation	以有限差分近似導數的方法

11.1 雅可比矩陣的定義

11.1.1 由位置關係到速度關係

第 9 章建立了正向運動學：末端位姿是關節變數的函數。

$$\mathbf{x} = \mathbf{f}(\mathbf{q})$$

$\mathbf{x}$ 為末端位姿（六維：位置三維 + 姿態三維）， $\mathbf{q}$ 為關節變數（ n 維）。 $\mathbf{f}$ 是高度非線性的函數。

現在問一個新問題：若關節以某個速度轉動，末端會以多快的速度移動？把上式對時間微分，依鏈鎖律：

$$\dot{\mathbf{x}} = \partial \mathbf{f} / \partial \mathbf{q} \cdot \dot{\mathbf{q}} = \mathbf{J}(\mathbf{q}) \cdot \dot{\mathbf{q}}$$

這就是雅可比矩陣的定義。 $\mathbf{J}(\mathbf{q}) = \partial \mathbf{f} / \partial \mathbf{q}$ 是一個 $m \times n$ 的矩陣（ m 為任務維度、 n 為關節數），且它是 $\mathbf{q}$ 的函數——會隨姿態改變。

展開來看，雅可比矩陣的每一個元素都是一個偏導數：

$$\mathbf{J} = \begin{bmatrix} \partial f_1 / \partial q_1 & \partial f_1 / \partial q_2 & \cdots & \partial f_1 / \partial q_n \\ \partial f_2 / \partial q_1 & \partial f_2 / \partial q_2 & \cdots & \partial f_2 / \partial q_n \\ \vdots & \vdots & \ddots & \vdots \\ \partial f_m / \partial q_1 & \partial f_m / \partial q_2 & \cdots & \partial f_m / \partial q_n \end{bmatrix}$$

第 i 列第 j 行的元素，表示「第 j 個關節的微小變化，對第 i 個任務座標的影響」。

11.1.2 三種等價的理解方式

雅可比矩陣有三種讀法，能靈活切換是掌握本章的關鍵。

讀法一：速度的兌換率。給定關節速度 $\dot{\mathbf{q}}$ ， $\mathbf{J}$ 告訴你末端速度 $\dot{\mathbf{x}}$ 。這是開頭故事中示教器上顯示的關係——同樣的關節速度，在不同姿態下會產生不同的末端速度。

讀法二：微小位移的映射。把時間微分改寫成微小量： $\Delta \mathbf{x} \approx \mathbf{J} \cdot \Delta \mathbf{q}$ 。這是第 10 章數值逆運動學的基礎——「關節動一點點，末端會跑多遠」。

讀法三：非線性函數的局部線性化。 $\mathbf{J}$ 是 $\mathbf{f}$ 在某個 $\mathbf{q}$ 處的一階泰勒展開係數。非線性的 $\mathbf{f}$ 在局部被一個線性映射近似，這使得所有線性代數的工具都能派上用場。

這三種讀法在數學上完全相同，但在不同場合各有其直覺價值。討論運動控制時用讀法一、討論逆運動學迭代時用讀法二、討論奇異點與可操作度時用讀法三。

11.1.3 為什麼叫「雅可比」

這個名稱來自十九世紀德國數學家雅可比（Carl Gustav Jacob Jacobi）。在多變數微積分中，任何從 $\mathbb{R}^n$ 映射到 $\mathbb{R}^m$ 的函數，其一階偏導數所構成的矩陣就稱為雅可比矩陣。機器人學只是這個通用概念的一個應用——但因為它的物理意義（速度、力）如此豐富，反而成為最廣為人知的應用之一。

值得注意的是，在微積分中雅可比矩陣的行列式（Jacobian determinant）用於變數變換時的體積縮放。這個「體積」的概念在機器人學中同樣有意義——它與 11.8 節的可操作度直接相關。

11.2 偏微分法：由正向運動學直接推導

11.2.1 平面 2R 手臂的完整推導

最直接的推導方式是把正向運動學方程式逐項對每個關節變數偏微分。以平面 2R 手臂為例，第 9.6.1 節已得到：

$$\begin{aligned} x &= L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2) \\ y &= L_1 \sin \theta_1 + L_2 \sin(\theta_1 + \theta_2) \end{aligned}$$

平面 2R 的正向運動學。

對 θ_1 偏微分（注意 θ_1 同時出現在兩項中）：

$$\begin{aligned} \partial x / \partial \theta_1 &= -L_1 \sin \theta_1 - L_2 \sin(\theta_1 + \theta_2) \\ \partial y / \partial \theta_1 &= L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2) \end{aligned}$$

第一行的偏導數。

對 θ_2 偏微分（ θ_2 只出現在第二項中）：

$$\begin{aligned} \partial x / \partial \theta_2 &= -L_2 \sin(\theta_1 + \theta_2) \\ \partial y / \partial \theta_2 &= L_2 \cos(\theta_1 + \theta_2) \end{aligned}$$

第二行的偏導數。

把四個偏導數排成矩陣，得到平面 2R 手臂的雅可比：

$$J = \begin{bmatrix} -L_1 s_1 - L_2 s_{12} & -L_2 s_{12} \\ L_1 c_1 + L_2 c_{12} & L_2 c_{12} \end{bmatrix}$$

這正是第 10 章例題 10-3 中直接給出的矩陣，現在我們知道它從何而來。

11.2.2 解讀矩陣的每一行

這個矩陣的結構有清楚的物理意義。**第一行**（對 θ_1 的偏導）描述：只轉動第一關節時，末端如何移動。觀察其形式 $[-y_{\text{end}}, x_{\text{end}}]^T$ ——這正是「繞原點旋轉」的速度方向（垂直於半徑向量）。**第二行**（對 θ_2 的偏導）形式為 $[-(y_{\text{end}} - y_2), (x_{\text{end}} - x_2)]^T$ ，其中 (x_2, y_2) 是第二關節的位置——這是「繞第二關節旋轉」的速度方向。

這個觀察揭示了一個通則：**每一個旋轉關節對末端速度的貢獻，都是「繞該關節軸旋轉」所產生的速度**。這正是 11.4 節幾何法的核心思想，它讓我們不必做任何微分就能寫出雅可比。

11.2.3 偏微分法的優劣

偏微分法的優點是概念直接、對簡單機構容易操作、且能得到解析表達式（適合理論分析）。缺點是對六軸手臂而言，正向運動學的表達式極為龐大（第 9.8.3 節提到展開後可能數百項），

逐項微分不切實際。

因此實務上有三種替代方案：幾何法（11.4 節，用向量外積直接寫出，最常用）、符號運算軟體（用 SymPy 自動微分，適合理論推導）、以及數值微分（對每個關節加微小擾動，觀察末端變化，程式最容易實作但精度較低）。

11.3 線速度與角速度

11.3.1 六維速度：旋量

空間中的剛體有六個自由度，因此其速度也是六維的：三維的線速度 $\mathbf{v}$ 與三維的角速度 $\boldsymbol{\omega}$ 。兩者合起來稱為**旋量**（twist）：

$$\mathbf{v} = \begin{bmatrix} \mathbf{v} \\ \boldsymbol{\omega} \end{bmatrix} \in \mathbb{R}^6 \quad \text{完整的雅可比關係：} \mathbf{v} = \mathbf{J}(\mathbf{q}) \cdot \dot{\mathbf{q}}$$

上半部三列稱為位置雅可比 $\mathbf{J}_v$ （或 $\mathbf{J}_P$ ），下半部三列稱為姿態雅可比 $\mathbf{J}_\omega$ （或 $\mathbf{J}_O$ ）。

因此對六軸手臂而言， $\mathbf{J}$ 是一個 $6 \times n$ 的矩陣。若只關心位置（如點焊、取放），可只用上半的 $3 \times n$ ；若只關心姿態（如相機朝向控制），可只用下半的 $3 \times n$ 。

11.3.2 角速度的特殊性

線速度很直觀：它就是位置的時間導數， $\mathbf{v} = \dot{\mathbf{p}}$ 。但**角速度不是任何姿態表示法的導數**——這是本章最容易被忽略、卻最重要的觀念之一。

為什麼？回顧第 8 章：姿態可以用旋轉矩陣（九個數）、歐拉角（三個數）、四元數（四個數）表示。若把旋轉矩陣對時間微分，得到的是一個 3×3 矩陣 $\dot{\mathbf{R}}$ ，不是三維向量。事實上，角速度與旋轉矩陣的關係是：

$$\dot{\mathbf{R}} = [\boldsymbol{\omega}]_\times \cdot \mathbf{R} \quad \text{其中 } [\boldsymbol{\omega}]_\times \text{ 為 } \boldsymbol{\omega} \text{ 的反對稱矩陣 (第 8.4.2 節)}$$

角速度透過反對稱矩陣與旋轉矩陣的導數相關聯。這是角速度的嚴格定義。

角速度的物理意義很明確：它的方向是**瞬時轉軸**、長度是**轉動的角速率**。它是一個真正的物理量，可以直接用陀螺儀量測（第 6.3.1 節）。

相對地，歐拉角的導數（ θ, φ, ψ ）只是三個數字的變化率，**它們不構成一個向量**——不能直接相加、不能做外積、在萬向節鎖處會發散。這正是為什麼機器人學嚴格區分「幾何雅可比」與「解析雅可比」。

◎ 一個容易犯的錯

初學者常寫出「角速度 = RPY 角的導數」這樣的式子並直接使用。這在小角度時近似成立，但在一般姿態下是錯的——兩者之間相差一個依姿態而變的轉換矩陣（11.5.2 節）。若在控制迴路中混用，會導致末端姿態的運動方向偏離預期，而且在接近萬向節鎖時完全失控。判斷方法很簡單：若你的量是由陀螺儀量到的，那是角速度；若是由角度感測器的讀值相減得到的，那是角度導數。

11.3.3 為什麼角速度可以直接相加

角速度有一個極為有用的性質：**多個轉動的角速度可以直接向量相加**。若連桿 2 相對於連桿 1 以 ω_2 轉動、連桿 1 相對於基座以 ω_1 轉動，則連桿 2 相對於基座的角速度為：

$$\omega_{2/\text{基座}} = \omega_1 + \omega_2 \text{ (皆需先表示在同一座標系中)}$$

角速度的可加性。這與有限旋轉不可交換（第 8.7.2 節）形成有趣的對比。

為何有限旋轉不可交換，無窮小的角速度卻可以相加？直觀的解釋是：不可交換性來自二階項，而在無窮小的極限下二階項可以忽略。這正是「微分」的威力——它把複雜的非線性結構，在局部化簡為簡單的線性結構。

這個可加性是 11.4 節幾何法的理論基礎：每個關節的角速度貢獻可以直接相加，不需考慮順序。

11.4 幾何雅可比的直接構造法

11.4.1 核心思想

幾何法不需要任何微分。它的思想是：**分別考慮每一個關節的獨立貢獻，再把它們加起來**。具體而言，第 j 行的雅可比，就是「只讓第 j 個關節以單位速度運動、其餘關節鎖住」時，末端所產生的速度。

這樣做的合法性來自兩點：線速度可以相加（顯然）、角速度也可以相加（11.3.3 節）。因此總速度就是各關節貢獻的向量和，而這正是矩陣乘法 $J \cdot \dot{q}$ 所表達的意思。

11.4.2 旋轉關節的貢獻

設第 j 個關節為旋轉關節，其軸向的單位向量為 $\hat{z}_{j-1}$ （在基座座標系中表示），關節位置為 p_{j-1} ，末端位置為 p_n 。當此關節以角速度 $\dot{\theta}_j$ 轉動時：

角速度貢獻：整個下游連桿繞該軸旋轉，因此角速度就是沿軸向：

$$\omega_j = \dot{z}_{j-1} \cdot \theta_j$$

旋轉關節對末端角速度的貢獻。

線速度貢獻：末端繞著該軸做圓周運動。由剛體運動學，繞軸旋轉所產生的線速度為角速度與半徑向量的外積：

$$v_j = (\dot{z}_{j-1} \times (p_n - p_{j-1})) \cdot \theta_j$$

旋轉關節對末端線速度的貢獻。 $(p_n - p_{j-1})$ 是由關節指向末端的向量。

因此旋轉關節對應的雅可比行為：

$$J_j = \begin{bmatrix} \dot{z}_{j-1} \times (p_n - p_{j-1}) \\ \dot{z}_{j-1} \end{bmatrix} \quad \begin{matrix} \text{(上三列：線速度)} \\ \text{(下三列：角速度)} \end{matrix}$$

旋轉關節的雅可比行。這是本章最實用的一個公式。

11.4.3 移動關節的貢獻

移動關節的情形更簡單。當關節沿軸向 $\dot{z}_{j-1}$ 以速度 d_j 平移時，整個下游連桿隨之平移，但不產生任何轉動：

$$J_j = \begin{bmatrix} \dot{z}_{j-1} \\ 0 \end{bmatrix} \quad \begin{matrix} \text{(上三列：線速度)} \\ \text{(下三列：角速度為零)} \end{matrix}$$

移動關節的雅可比行。

11.4.4 構造雅可比的完整流程

有了上述兩個公式，構造雅可比只需三步：

1. **執行正向運動學：**計算 ${}^0T_1, {}^0T_2, \dots, {}^0T_n$ ，由各矩陣中取出每個關節的軸向 $\dot{z}_{j-1}$ (即 ${}^0R_{j-1}$ 的第三行) 與位置 p_{j-1} (即 ${}^0T_{j-1}$ 的平移欄)。
2. **逐行填入：**對每個關節，依其類型套用 11.4.2 或 11.4.3 節的公式，得到該行的六個元素。
3. **組合成矩陣：**把 n 個行並排，得到 $6 \times n$ 的雅可比矩陣。

這個方法的優雅之處在於：它完全重用了正向運動學的計算結果。既然執行控制時本來就要算正向運動學，雅可比幾乎是「順便」得到的，額外成本極低。這也是實務上一律採用幾何法的原因。

圖 11-1 旋轉關節對末端速度的貢獻

建議配圖：三維示意圖。畫出一支多關節手臂，特別標出第 j 個旋轉關節：其軸向單位向量 $\dot{z}_{j-1}$ (藍色箭頭)、

關節位置 p_{i-1} 與末端位置 p_n 。由關節指向末端的向量 $(p_n - p_{i-1})$ 以虛線表示（綠色）。末端處畫出兩個向量：角速度貢獻 $\hat{z}_{i-1}$ （沿軸向，藍色）與線速度貢獻 $\hat{z}_{i-1} \times (p_n - p_{i-1})$ （垂直於軸與半徑所張平面，紅色）。並以弧形箭頭表示末端繞軸的圓周運動。

11.4.5 以平面 2R 驗證

用幾何法重新計算平面 2R 手臂，驗證與 11.2.1 節的偏微分結果一致。兩個關節軸都沿 Z 方向（垂直紙面），即 $\hat{z}_0 = \hat{z}_1 = [0, 0, 1]^T$ 。

第一行： $p_0 = [0, 0, 0]^T$ （基座）、 $p_n = [x, y, 0]^T$ （末端）。

$$\hat{z}_0 \times (p_n - p_0) = [0, 0, 1] \times [x, y, 0] = [-y, x, 0]^T$$

取前兩個分量得 $[-y, x]^T = [-(L_1 s_1 + L_2 s_{12}), L_1 c_1 + L_2 c_{12}]^T$ ，與偏微分結果的第一行完全相同。

第二行： $p_1 = [L_1 c_1, L_1 s_1, 0]^T$ （第二關節位置）。

$$\hat{z}_1 \times (p_n - p_1) = [0, 0, 1] \times [L_2 c_{12}, L_2 s_{12}, 0] = [-L_2 s_{12}, L_2 c_{12}, 0]^T$$

與偏微分結果的第二行完全相同。

兩種方法得到相同答案，但幾何法完全不需要微分——只需要正向運動學算出的位置與軸向。對六軸手臂而言，這個差別是「幾行程式碼」與「數頁推導」的差別。

11.5 幾何雅可比與解析雅可比

11.5.1 兩者的差異

11.4 節構造的雅可比稱為幾何雅可比 J_g ，它的下半部給出真正的物理角速度 ω 。但在某些場合，我們想要的是姿態參數的變化率——例如控制器的目標是「RPY 角以每秒 5 度變化」，這時需要的是解析雅可比 J_a ：

$$\text{幾何: } \begin{bmatrix} \dot{v} \\ \omega \end{bmatrix} = J_g \cdot \dot{q} \quad \text{解析: } \begin{bmatrix} \dot{p} \\ \dot{\phi} \end{bmatrix} = J_a \cdot \dot{q}$$

q 為姿態參數（如 RPY 三角）， ϕ 是它們的時間導數，不等於角速度 ω 。

兩者的上半部（線速度）完全相同——位置就是位置，沒有歧義。差別只在下半部。

11.5.2 兩者的轉換關係

角速度與姿態參數導數之間存在一個轉換矩陣，其形式取決於所用的姿態表示法。以 Z-Y-X 的 RPY 角為例：

$$\omega = E(\phi) \cdot \dot{\phi} \Rightarrow J_g = \begin{bmatrix} I & 0 \\ 0 & E \end{bmatrix} \cdot J_a$$

$E(\phi)$ 是一個 3×3 的轉換矩陣，其元素為姿態角的三角函數。

對 RPY 角 (roll γ 、pitch β 、yaw α)，這個矩陣為：

$$E = \begin{bmatrix} 1 & -s\alpha & c\alpha \cdot c\beta \\ 0 & c\alpha & s\alpha \cdot c\beta \\ 0 & 0 & s\beta \end{bmatrix}$$

由角度導數映射到角速度的轉換矩陣。

關鍵觀察是：當 $\cos \beta = 0$ (即 pitch = $\pm 90^\circ$) 時， E 的行列式為零，矩陣不可逆。這正是第 8.3.3 節的萬向節鎖，在速度層面的表現——此時無法由角速度反推出唯一的角度導數。

11.5.3 實務上該用哪一種

用幾何雅可比的場合：力控制與碰撞偵測 (因為力矩對應的是物理角速度，見 11.6 節)、動力學推導 (第 13 章)、以及任何涉及物理量的計算。工業機器人控制器內部一律使用幾何雅可比。

用解析雅可比的場合：當任務直接以姿態參數描述時 (例如「保持 RPY 的 pitch 為 0」)、以及某些以歐拉角為介面的規劃演算法。但必須注意奇異點問題。

本書的預設：除非特別註明，本書後續所有的 J 皆指幾何雅可比。這與 Craig 教科書及工業界的慣例一致。

表 11-2 幾何雅可比與解析雅可比的比較

面向	幾何雅可比 J_g	解析雅可比 J_a
下半部輸出	角速度 ω (物理量)	姿態參數導數 $\dot{\phi}$
由何取得	陀螺儀可直接量測	由角度感測值微分
奇異點	只有機構奇異點	另有表示法奇異點 (萬向節鎖)
可否直接相加	可 (角速度可加)	不可
與力的對偶	成立 ($\tau = J_g^T \cdot F$)	不直接成立
典型用途	力控制、動力學、工業控制器	以歐拉角描述的任務、部分規劃演算法

11.6 虛功原理與速度—力對偶

11.6.1 由能量守恆推導

現在來處理開頭故事中最有趣的第三點：為什麼同一個雅可比矩陣也連結力？答案來自**虛功原理**——一個比機器人學古老得多的力學原理。

考慮機器人末端受到外力與力矩（合稱**扳手** $F = [f, n]^T$ ），同時各關節輸出扭矩 τ 。若系統處於靜力平衡且沒有能量損失，則關節做的功必須等於末端做的功：

$$\tau^T \cdot \delta q = F^T \cdot \delta x$$

虛功相等。 δq 為關節的虛位移、 δx 為末端的虛位移。

代入雅可比關係 $\delta x = J \cdot \delta q$ ：

$$\tau^T \cdot \delta q = F^T \cdot J \cdot \delta q$$

兩邊對任意的 δq 都成立。

由於此式對**任意**虛位移 δq 都必須成立，兩邊的係數必然相等，因此：

$$\tau = J^T \cdot F$$

靜力關係式。這是機器人學最優美的公式之一——同一個雅可比矩陣，用轉置就把末端的力映射成關節的扭矩。

11.6.2 對偶性的意義

這個結果揭示了一個深刻的對偶關係：

$$\text{速度：}\dot{x} = J \cdot \dot{q} \quad (\text{關節} \rightarrow \text{末端})$$

$$\text{力量：}\tau = J^T \cdot F \quad (\text{末端} \rightarrow \text{關節})$$

兩者的方向恰好相反，而矩陣互為轉置。這稱為運動—力對偶（kinematics-statics duality）。

方向相反這件事很值得玩味。速度是「由內往外傳」——馬達轉動，末端才移動；力則是「由外往內傳」——末端受力，關節才需要出力。而連結兩者的，竟是同一個矩陣的兩種形式。

這個對偶帶來三個重要推論。**推論一**：不需要額外的推導，就能由運動學直接得到靜力學。

推論二：雅可比的秩同時決定了「能往哪些方向運動」與「能抵抗哪些方向的力」（第 12 章）。

推論三：在奇異點處，機器人失去某方向的運動能力，但反過來說，它在該方向上可以抵抗無限大的力而不需任何關節扭矩——這解釋了為何手臂完全伸直時特別「有力」（也特別容易被卡住）。

11.6.3 三個實際應用

應用一：重力補償。已知各連桿的重量會在末端產生等效的力，用 $\tau = J^T \cdot F$ 即可算出各關節需要輸出多少扭矩來平衡它。這是第 17 章重力補償控制的基礎。

應用二：力控制。若要末端對工件施加特定的力（如研磨、裝配），可用此式反算出各關節該輸出的扭矩。這是第 20 章的核心。

應用三：碰撞偵測。反過來用：量測各關節的實際扭矩（由馬達電流估算，第 6.4.2 節），扣除重力與慣性所需的部分，剩餘的 τ_{ext} 就是外力造成的。再由 $F = (J^T)^+ \cdot \tau_{\text{ext}}$ 反推出末端受到的外力大小與方向。這正是協作機器人「感覺到你碰它」的原理——不需要任何力覺感測器，只靠電流與雅可比。

◎ 為什麼協作機器人的減速比不能太高

第 5 章提過高減速比會使關節無法反向驅動。現在可以從對偶性理解得更深：碰撞偵測依賴的是「外力造成的關節扭矩變化」。但減速比 N 會把外力對關節的影響縮小 N 倍，同時把摩擦力放大——當減速比達到 100 時，外力訊號可能已被摩擦的不確定性淹沒。這就是協作機器人採用中低減速比、或加裝關節力矩感測器的根本原因。

11.7 雅可比的座標系轉換

11.7.1 雅可比是相對於某個座標系的

一個常被忽略的重點是：雅可比矩陣的數值，取決於它是在**哪個座標系**中表達的。11.4 節構造的雅可比，其軸向 $\dot{z}$ 與位置 p 都是在基座座標系中表示的，因此得到的是「基座座標系中的雅可比」 0J 。

有時我們需要「末端座標系中的雅可比」 nJ ——例如力控制時，往往希望以工具自身的座標描述力的方向（「沿夾爪的接近方向施加 10 牛頓」）。兩者的轉換為：

$${}^nJ = \begin{bmatrix} {}^nR_0 & 0 \\ 0 & {}^nR_0 \end{bmatrix} \cdot {}^0J$$

上下兩塊各用旋轉矩陣轉換（線速度與角速度都是向量，都需要旋轉）。注意只有旋轉、沒有平移——因為速度向量的轉換不涉及原點位置。

11.7.2 參考點的改變

另一種常見的需求是改變**參考點**。11.4 節的雅可比是針對末端點 p_n 計算的；若裝上工具後，實際的作用點移到了 TCP（距離 p_n 有一個偏移 r ），雅可比該如何修正？

由剛體運動學，同一剛體上兩點的速度關係為：

$$\mathbf{v_TCP} = \mathbf{v_n} + \boldsymbol{\omega} \times \mathbf{r} \quad \boldsymbol{\omega_TCP} = \boldsymbol{\omega_n}$$

角速度處處相同（剛體不變形），但線速度會因為轉動而不同。

因此雅可比的修正為：

$$\mathbf{J_TCP} = \begin{bmatrix} \mathbf{I} & -[\mathbf{r}]_{\times} \\ \mathbf{0} & \mathbf{I} \end{bmatrix} \cdot \mathbf{J_n}$$

$[\mathbf{r}]_{\times}$ 為 $\mathbf{r}$ 的反對稱矩陣（第 8.4.2 節）。這個矩陣稱為伴隨變換（adjoint transformation）。

這個修正在實務上極為重要：換一支夾具，雅可比就變了。若程式中忘記更新 TCP 偏移，會導致末端在旋轉時產生預期外的平移——症狀是「機器人轉動夾爪時，工件的尖端會畫圓而非原地旋轉」。這是裝配與點膠作業中的常見故障。

11.8 可操作度、速度橢球與力橢球

11.8.1 各方向的能力並不相同

回到開頭故事的觀察：在不同姿態下，同樣的關節速度會產生不同的末端速度。更進一步，即使在同一個姿態下，往不同方向運動的能力也不相同。

要量化這件事，考慮所有「關節速度長度不超過 1」的組合，看看它們映射到末端後形成什麼形狀：

$$\{\dot{\mathbf{x}} = \mathbf{J} \cdot \mathbf{q} : \|\mathbf{q}\| \leq 1\} \Rightarrow \text{一個橢球}$$

這稱為**速度橢球**（velocity ellipsoid）。它的形狀完整描述了機器人在該姿態下各方向的運動能力。

橢球的主軸方向與長度，由雅可比的**奇異值分解**（SVD）給出：

$$\mathbf{J} = \mathbf{U} \boldsymbol{\Sigma} \mathbf{V}^T \quad \text{其中 } \boldsymbol{\Sigma} = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_m)$$

$\mathbf{U}$ 的行向量是橢球的主軸方向、 σ_i 是對應主軸的半徑長度。 σ_1 為最大奇異值、 σ_m 為最小奇異值。

直觀理解：最大奇異值的方向是「最容易動」的方向、最小奇異值的方向是「最難動」的方向。當某個奇異值趨近於零時，該方向的運動能力消失——這就是奇異點的嚴格定義，第 12 章將深入探討。

11.8.2 可操作度指標

Yoshikawa 於 1985 年提出一個純量指標來衡量整體的運動靈活度，稱為**可操作度**

(manipulability)：

$$w = \sqrt{(\det(J J^T))} = \sigma_1 \cdot \sigma_2 \cdot \dots \cdot \sigma_m$$

可操作度等於所有奇異值的乘積，也就是速度橢球的體積（相差一個常數係數）。

這個指標有三個實用價值。**價值一：奇異點偵測**。當 w 趨近於零時，代表至少有一個奇異值趨近於零，機器人接近奇異點。這可用來觸發第 10.7.3 節的 DLS 阻尼調整。**價值二：姿態最佳化**。冗餘機器人可利用零空間，在不影響末端任務的前提下最大化 w ，使機器人維持在「靈活的姿態」。**價值三：工作站佈局**。設計工作站時，應把主要作業區配置在可操作度較高的區域。

但可操作度也有其限制：它是所有方向的「平均」能力（體積），無法反映「某一個特定方向」的能力。若任務只在乎某方向（如垂直插入），應直接看該方向對應的奇異值，而非整體的 w 。

11.8.3 力橢球與對偶的幾何

由 11.6 節的對偶性，可以類似地定義**力橢球**：在關節扭矩長度不超過 1 的限制下，末端能輸出的力所形成的形狀。

由於力的映射用的是 $(J^T)^{-1}$ （或偽逆），力橢球的主軸方向與速度橢球**完全相同**，但半徑長度**互為倒數**：

速度橢球半徑： σ_i	力橢球半徑： $1/\sigma_i$
--------------------	---------------------

兩者互為倒數，形成一個優雅的對偶關係。

這個結果有非常直觀的物理意義：**容易動的方向，就是不容易出力的方向；反之亦然**。這正是槓桿原理的推廣——你用一把長槓桿可以輕鬆撬動重物（力大），但槓桿另一端要移動很長的距離（速度小）。

這個對偶對工程設計有直接的指導意義。以裝配作業為例：插入方向需要**大的力**（推入）與**精細的位置控制**（對準）——這兩個需求恰好衝突。解決方案有二：一是選擇讓插入方向對應到「力橢球長軸」的姿態（有力但精度差），再靠柔順性（第 20 章）補償對位；二是改用 SCARA 這類在垂直方向天生剛硬的機構（第 4.8.1 節的「選擇性順應」）——現在你能理解，那個設計正是在操控速度—力橢球的形狀。

圖 11-2 速度橢球與力橢球的對偶

建議配圖：兩格對照圖，各畫一支平面 $2R$ 手臂在同一姿態下的末端橢球。左格為速度橢球（藍色實線），長軸沿容易運動的方向（垂直於手臂伸展方向），標註半徑 σ_1 、 σ_2 。右格為力橢球（紅色實線），主軸方向相同但長短相反，長軸沿手臂伸展方向，標註半徑 $1/\sigma_2$ 、 $1/\sigma_1$ 。中央以雙向箭頭連接並標註「互為倒數」。下方另加一格：手臂接近伸直時，速度橢球被壓扁成一條線（ $\sigma_2 \rightarrow 0$ ）、力橢球則沿該方向無限延伸——標註「奇異點」。

11.9 雅可比的應用全景

11.9.1 運動速率控制

雅可比最直接的應用是**運動速率控制**（resolved motion rate control）：操作員或上層規劃給出末端的期望速度，控制器即時反算各關節該轉多快。

$$\mathbf{q}' = \mathbf{J}^{-1}(\mathbf{q}) \cdot \dot{\mathbf{x}}_{\text{desired}} \quad (\text{或以偽逆 / DLS 取代反矩陣})$$

這正是搖桿遙操作、視覺伺服與力控制的共同底層機制。

值得注意的是，這與第 10 章的數值逆運動學在數學上是同一件事——只是解釋不同。逆運動學是「反覆修正位置誤差直到收斂」，速率控制則是「持續把速度指令翻譯成關節速度」。前者是離散的迭代、後者是連續的控制迴路，但用的都是 $\mathbf{J}$ 的逆。這也是為什麼第 10 章的所有數值技巧（偽逆、DLS、零空間）在此完全適用。

11.9.2 六大應用領域

雅可比矩陣在機器人學中的應用範圍極廣，以下是它在本書後續章節出現的位置：

表 11-3 雅可比矩陣的應用一覽

應用	使用形式	說明	章節
數值逆運動學	$\mathbf{J}^+$ 或 DLS	迭代求解關節角度	第 10 章
奇異點分析	$\text{rank}(\mathbf{J})$ 、 $\sigma_{\min}$	判斷失去運動能力的位形	第 12 章
冗餘與零空間	$(\mathbf{I} - \mathbf{J}^+ \mathbf{J})$	在不影響任務下最佳化次要目標	第 12 章
運動速率控制	$\mathbf{J}^{-1}$	由末端速度指令求關節速度	第 11、17 章
動力學推導	$\mathbf{J}$ 、 $\dot{\mathbf{J}}$	計算慣性矩陣與科氏項	第 13、14 章
操作空間控制	$\mathbf{J}$ 、 $\mathbf{J}^T$ 、 $\Lambda = (\mathbf{J} \mathbf{M}^{-1} \mathbf{J}^T)^{-1}$	直接在任務空間設計控制律	第 17 章
重力補償	$\mathbf{J}^T \mathbf{F}_{\text{gravity}}$	計算平衡重力所需的關節扭矩	第 17 章
力控制	$\boldsymbol{\tau} = \mathbf{J}^T \mathbf{F}$	由期望的末端力求關節扭矩	第 20 章
碰撞偵測	$\mathbf{F} = (\mathbf{J}^T)^+ \cdot \boldsymbol{\tau}_{\text{ext}}$	由關節扭矩反推外力	第 20、35 章
視覺伺服	$\mathbf{J}_{\text{image}} \cdot \mathbf{J}_{\text{robot}}$	由影像誤差直接驅動關節	第 22 章

11.9.3 雅可比的時間導數

在動力學與加速度層級的控制中，還會用到雅可比的時間導數 $\dot{J}$ 。把速度關係再對時間微分一次：

$$\ddot{\mathbf{x}} = \mathbf{J}(\mathbf{q}) \cdot \dot{\mathbf{q}} + \dot{\mathbf{J}}(\mathbf{q}, \dot{\mathbf{q}}) \cdot \dot{\mathbf{q}}$$

末端加速度包含兩項：關節加速度的貢獻，以及因姿態改變導致雅可比本身變化的貢獻。

第二項 $\dot{\mathbf{J}} \cdot \dot{\mathbf{q}}$ 常被稱為「速度相關項」，它在物理上對應到科氏力與離心力——第 13 章動力學方程中的 $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) \cdot \dot{\mathbf{q}}$ 項正源於此。這說明了一件事：那些看似神秘的科氏力，其實只是「雅可比會隨姿態改變」這個事實的必然結果。

實務上 $\dot{J}$ 通常以數值微分近似 $(J(t) - J(t - \Delta t)) / \Delta t$ ，因為解析推導極為繁瑣且收益有限。

11.10 由速度邁向奇異點

11.10.1 本章的成果

本章完成了運動學的第三塊拼圖。你現在能夠：由偏微分或幾何法構造任意串聯手臂的雅可比矩陣、區分幾何與解析兩種形式、運用虛功原理連結速度與力、在不同座標系與參考點之間轉換雅可比、並用奇異值與可操作度量化機器人在各方向的運動與出力能力。

更重要的是，你獲得了一個統一的視角：**雅可比是機器人學的中央樞紐**。運動學在它的左邊、力學在它的右邊、控制在它的上方、而奇異點在它的內部。本書後續二十多章中，將有超過一半會直接或間接用到它。

11.10.2 接下來：當雅可比壞掉的時候

本章多次提到一個現象：在某些姿態下，雅可比會失去滿秩，導致某個奇異值趨近於零。此時速度橢球被壓扁、可操作度歸零、逆矩陣不存在、數值計算爆炸。

第 12 章將專門處理這個現象。我們將回答：奇異點在哪裡？如何預先偵測？為什麼機構設計上無法完全避免？以及——如果機器人有冗餘自由度，能否用零空間繞開它？屆時，第 4 章的冗餘、第 8 章的萬向節鎖、第 9 章的手臂伸直、第 10 章的解合併與數值爆炸，這些散落各章的線索，將被雅可比矩陣的秩統一串起來。

完整計算例題

例題 11-1 平面 2R 手臂的雅可比與速度計算

題目：平面 2R 手臂 $L_1 = 300 \text{ mm}$ 、 $L_2 = 200 \text{ mm}$ ，目前姿態 $\theta_1 = 30^\circ$ 、 $\theta_2 = 45^\circ$ 。(a) 求此姿態下的雅可比矩陣。(b) 若兩關節皆以每秒 10 度的角速度轉動，求末端的線速度。(c) 若要末端以 100 mm/s 沿 +X 方向運動，求所需的關節角速度。

解 (a)：代入 11.2.1 節的公式。 $\theta_1 = 30^\circ$ 、 $\theta_1 + \theta_2 = 75^\circ$ 。

$$s_1 = 0.5000, c_1 = 0.8660, s_{12} = 0.9659, c_{12} = 0.2588$$

先算出四個三角函數值。

$$J = \begin{bmatrix} -300(0.5) - 200(0.9659) & -200(0.9659) \\ 300(0.866) + 200(0.2588) & 200(0.2588) \end{bmatrix} = \begin{bmatrix} -343.19 & -193.19 \\ 311.57 & 51.76 \end{bmatrix}$$

單位為 mm/rad。

驗證技巧：J 的第一行應等於 $[-y, x]^T$ (11.2.2 節)。由例題 9-1 知末端位置為 (311.57, 343.19)，故 $[-y, x] = [-343.19, 311.57]$ ✓ 與計算結果相符。

解 (b)：關節角速度需先轉為弧度： $10^\circ/\text{s} = 0.1745 \text{ rad/s}$ 。

$$\dot{x} = J \cdot \dot{q} = \begin{bmatrix} -343.19 & -193.19 \\ 311.57 & 51.76 \end{bmatrix} \begin{bmatrix} 0.1745 \\ 0.1745 \end{bmatrix} = \begin{bmatrix} -93.60 \\ 63.42 \end{bmatrix} \text{ mm/s}$$

末端線速度。

速度大小為 $\sqrt{93.60^2 + 63.42^2} \approx 113.1 \text{ mm/s}$ ，方向為 $\text{atan2}(63.42, -93.60) \approx 145.9^\circ$ (指向上方)。注意兩個關節都以相同角速度正轉，末端卻往左上移動——這正是雅可比「兌換率隨姿態而變」的具體體現。

解 (c)：需求解 $J \cdot \dot{q} = [100, 0]^T$ 。先求 J 的行列式：

$$\det(J) = (-343.19)(51.76) - (-193.19)(311.57) = -17763 + 60190 = 42427$$

行列式不為零，J 可逆，此姿態非奇異點。

$$J^{-1} = (1/42427) \begin{bmatrix} 51.76 & 193.19 \\ -311.57 & -343.19 \end{bmatrix}$$

反矩陣 (二階矩陣的反矩陣公式：主對角互換、副對角變號，再除以行列式)。

$$\dot{q} = J^{-1} \cdot \begin{bmatrix} 100 \\ 0 \end{bmatrix} = (1/42427) \begin{bmatrix} 5176 \\ -31157 \end{bmatrix} = \begin{bmatrix} 0.1220 \\ -0.7344 \end{bmatrix} \text{ rad/s} = \begin{bmatrix} 6.99 \\ -42.08 \end{bmatrix} \text{ }^\circ/\text{s}$$

所需的關節角速度。

工程觀察：要讓末端以 100 mm/s 沿 X 方向移動，第二關節需以 42 度/秒反轉——遠快於第一關節的 7 度/秒。若目標速度提高到 500 mm/s，第二關節就需要 210 度/秒，可能已超過馬達能力。這說明了一件事：**末端速度的限制，往往不是由馬達的最高轉速直接決定，而是由當前姿態下的雅可比決定。**第 15 章的軌跡規劃必須考慮這一點。

例題 11-2 靜力分析與碰撞偵測

題目：同上手臂與姿態。(a) 若末端受到一個沿 -X 方向、大小 20 N 的外力，求各關節需輸出多少扭矩才能維持靜止。(b) 反過來，若量測到關節扭矩為 $\tau = [-5.0, -2.5]^T$ N·m (已扣除重力與摩擦)，求末端受到的外力。

解 (a)：使用 $\tau = J^T \cdot F$ 。外力 $F = [-20, 0]^T$ N。注意 J 的單位是 mm/rad，需轉換為 m/rad (除以 1000)。

$$J^T = \begin{bmatrix} -0.34319 & 0.31157 \\ -0.19319 & 0.05176 \end{bmatrix} \quad (\text{單位：m/rad})$$

雅可比的轉置。

$$\tau = J^T \cdot F = \begin{bmatrix} -0.34319(-20) + 0.31157(0) \\ -0.19319(-20) + 0.05176(0) \end{bmatrix} = \begin{bmatrix} 6.864 \\ 3.864 \end{bmatrix} \text{ N}\cdot\text{m}$$

各關節所需扭矩。

因此第一關節需輸出 6.86 N·m、第二關節需 3.86 N·m。注意第一關節的負擔較重——這符合直覺，因為它承受了整條手臂的力矩。

解 (b)：這是碰撞偵測的計算方向。由 $\tau = J^T \cdot F$ 反解 $F = (J^T)^{-1} \cdot \tau$ 。

$$(J^T)^{-1} = (J^{-1})^T = (1/0.042427) \begin{bmatrix} 0.05176 & -0.31157 \\ 0.19319 & -0.34319 \end{bmatrix}$$

轉置的反矩陣等於反矩陣的轉置。

$$\begin{aligned} F &= (J^T)^{-1} \cdot \begin{bmatrix} -5.0 \\ -2.5 \end{bmatrix} = \begin{bmatrix} (0.05176)(-5.0) + (-0.31157)(-2.5) \\ (0.19319)(-5.0) + (-0.34319)(-2.5) \end{bmatrix} / 0.042427 \\ &= \begin{bmatrix} 0.5202 \\ -0.1080 \end{bmatrix} / 0.042427 = \begin{bmatrix} 12.26 \\ -2.55 \end{bmatrix} \text{ N} \end{aligned}$$

推得的末端外力。

因此末端受到一個約 12.5 N、方向偏向 +X 稍微向下的力。這正是協作機器人判斷「有人碰到我了、力來自哪個方向」的計算過程——完全不需要力覺感測器，只需要馬達電流與雅可比矩陣。

實務提醒：此計算的精度受兩個因素限制。一是關節扭矩的估算誤差（摩擦力的不確定性，第 6.4.2 節）；二是接近奇異點時 $(J^T)^{-1}$ 會放大誤差。因此高階協作機器人仍會加裝關節力矩感測器以提升可靠度。

例題 11-3 可操作度與速度橢球

題目：同上手臂。(a) 計算目前姿態的可操作度。(b) 計算 $\theta_2 = 5^\circ$ 時的可操作度並比較。(c) 求 θ_2 為何值時可操作度最大？

解 (a)：對方陣而言， $w = \sqrt{(\det(J^T J))} = |\det(J)|$ 。由例題 11-1， $\det(J) = 42427 \text{ mm}^2/\text{rad}^2$ 。

$$w = |\det(J)| = 42427 \text{ mm}^2/\text{rad}^2$$

目前姿態的可操作度。

解 (b)：對平面 2R 手臂，行列式有一個極為簡潔的封閉形式。展開 $\det(J)$ 可得：

$$\det(J) = L_1 L_2 \sin \theta_2$$

平面 2R 手臂的可操作度只取決於 θ_2 ，與 θ_1 完全無關。

驗證： $L_1 L_2 \sin 45^\circ = 300 \times 200 \times 0.7071 = 42426 \checkmark$ 與 (a) 一致。

當 $\theta_2 = 5^\circ$ ： $w = 300 \times 200 \times \sin 5^\circ = 60000 \times 0.0872 = 5229 \text{ mm}^2/\text{rad}^2$ 。僅為原本的 12.3%——手臂接近伸直時，運動能力大幅下降。

解 (c)：由 $w = L_1 L_2 |\sin \theta_2|$ 可知，當 $\theta_2 = \pm 90^\circ$ 時 w 達到最大值 $60000 \text{ mm}^2/\text{rad}^2$ 。此時前臂與上臂互相垂直——這是平面 2R 手臂最靈活的姿態。

工程啟示：這個結果有直接的實務意義。第一，工作站佈局時應讓主要作業區落在「手肘約彎 90 度」的位置，而非讓手臂接近伸直或折疊。第二，當 $\theta_2 \rightarrow 0$ 或 180° 時 $w \rightarrow 0$ ，即為奇異點（第 12 章）。第三，可操作度與 θ_1 無關，代表可以自由旋轉基座而不影響靈活度——這是設計轉台或行走軸時的有用資訊。

程式範例：雅可比的三種計算法與應用

以下 Python 程式實作雅可比的解析法、幾何法與數值微分法，驗證三者一致，並示範速度控制、靜力分析與可操作度計算。

```

import numpy as np

L1, L2 = 0.300, 0.200          # 連桿長度 (m)

def fk(q):
    """正向運動學：回傳末端位置 (x, y)"""
    t1, t2 = q
    return np.array([L1*np.cos(t1) + L2*np.cos(t1+t2),
                     L1*np.sin(t1) + L2*np.sin(t1+t2)])

def joint_positions(q):
    """回傳各關節與末端的位置 (供幾何法使用)"""
    t1, t2 = q
    p0 = np.array([0.0, 0.0])
    p1 = np.array([L1*np.cos(t1), L1*np.sin(t1)])
    p2 = fk(q)
    return p0, p1, p2

# ----- 方法一：解析法 (11.2 節偏微分) -----
def jacobian_analytic(q):
    t1, t2 = q
    s1, c1 = np.sin(t1), np.cos(t1)
    s12, c12 = np.sin(t1+t2), np.cos(t1+t2)
    return np.array([[-L1*s1 - L2*s12, -L2*s12],
                     [ L1*c1 + L2*c12,  L2*c12]])

# ----- 方法二：幾何法 (11.4 節：z × (p_n - p_j)) -----
def jacobian_geometric(q):
    p0, p1, p2 = joint_positions(q)
    z = np.array([0.0, 0.0, 1.0])          # 兩軸皆沿 z
    cols = []
    for pj in (p0, p1):                  # 各關節位置
        r = np.append(p2 - pj, 0.0)       # 由關節指向末端
        v = np.cross(z, r)                # 線速度貢獻
        cols.append(v[:2])                # 平面問題只取 x, y
    return np.column_stack(cols)

# ----- 方法三：數值微分 -----
def jacobian_numeric(q, eps=1e-7):
    q = np.asarray(q, dtype=float)
    J = np.zeros((2, len(q)))
    for j in range(len(q)):
        dq = np.zeros_like(q); dq[j] = eps
        J[:, j] = (fk(q + dq) - fk(q - dq)) / (2*eps)  # 中央差分
    return J

d, r2d = np.deg2rad, np.rad2deg
q = np.array([d(30), d(45)])

# ----- 驗證：三種方法一致性 -----

```

```

print("=== 三種雅可比計算法的比較 (單位 m/rad) ===")
Ja, Jg, Jn = jacobian_analytic(q), jacobian_geometric(q), jacobian_numeric(q)
print(f"解析法:\n{np.round(Ja, 5)}")
print(f"幾何法與解析法一致: {np.allclose(Ja, Jg)}")
print(f"數值法與解析法一致: {np.allclose(Ja, Jn, atol=1e-6)}")
print(f"驗證第一行 = [-y, x]: {np.allclose(Ja[:,0], [-fk(q)[1], fk(q)[0]])}")

# ----- 應用一：速度正解 (例題 11-1b) -----
print("\n=== 例題 11-1(b)：關節速度 → 末端速度 ===")
qd = np.array([d(10), d(10)])
xd = Ja @ qd
print(f"末端速度: ({xd[0]*1000:.2f}, {xd[1]*1000:.2f}) mm/s")
print(f"速度大小: {np.linalg.norm(xd)*1000:.1f} mm/s "
      f"方向: {r2d(np.arctan2(xd[1], xd[0])):.1f}°")

# ----- 應用二：速度逆解 (例題 11-1c) -----
print("\n=== 例題 11-1(c)：末端速度 → 關節速度 ===")
x_des = np.array([0.100, 0.0]) # 100 mm/s 沿 +X
qd_req = np.linalg.solve(Ja, x_des)
print(f"所需關節角速度: ({r2d(qd_req[0]):.2f}, {r2d(qd_req[1]):.2f}) °/s")

# ----- 應用三：靜力對偶 (例題 11-2) -----
print("\n=== 例題 11-2：速度-力對偶 ===")
F = np.array([-20.0, 0.0]) # 末端外力 (N)
tau = Ja.T @ F
print(f"(a) 外力 {F} N → 關節扭矩 ({tau[0]:.3f}, {tau[1]:.3f}) N·m")
tau_meas = np.array([-5.0, -2.5])
F_est = np.linalg.solve(Ja.T, tau_meas)
print(f"(b) 扭矩 {tau_meas} N·m → 推得外力 ({F_est[0]:.2f}, {F_est[1]:.2f}) N")

# ----- 應用四：可操作度 (例題 11-3) -----
print("\n=== 例題 11-3：可操作度分析 ===")
def manipulability(q):
    J = jacobian_analytic(q)
    return np.sqrt(np.linalg.det(J @ J.T))

for t2_deg in (5, 30, 45, 90, 135, 175):
    w = manipulability([d(30), d(t2_deg)])
    theory = L1*L2*abs(np.sin(d(t2_deg)))
    bar = "■" * int(w/L1/L2*40)
    print(f"θ2={t2_deg:4d}° w={w:.6f} 理論值={theory:.6f} {bar}")
print("→ θ2 = 90° 時最靈活；接近 0° 或 180° 時趨近奇異")

# ----- 應用五：速度橢球 (奇異值分解) -----
print("\n=== 速度橢球的主軸 (SVD) ===")
for t2_deg in (90, 20):

```

```
J = jacobian_analytic([d(30), d(t2_deg)])
U, S, Vt = np.linalg.svd(J)
print(f"  θ2={t2_deg:3d}°  奇異值 σ=({S[0]:.4f}, {S[1]:.4f})  "
      f"條件數={S[0]/S[1]:7.2f}")
print(f"          最易運動方向: {r2d(np.arctan2(U[1,0], U[0,0])):7.1f}°  "
      f"最難運動方向: {r2d(np.arctan2(U[1,1], U[0,1])):7.1f}°")
print("  → 條件數越大代表各方向能力差異越懸殊，越接近奇異點")
```

請試著修改：（一）把 θ_2 設為 0.5 度，觀察條件數如何暴增、以及速度逆解算出的關節角速度有多大。（二）比較數值微分在 $\text{eps} = 1\text{e-}3$ 與 $1\text{e-}12$ 時的誤差——你會發現太大與太小都不好（前者截斷誤差大、後者浮點捨入誤差大）。（三）繪出可操作度隨 θ_2 變化的曲線，驗證 $w = L_1 L_2 |\sin \theta_2|$ 。

實作：雅可比的實驗驗證

本章實作是在實體手臂上驗證雅可比矩陣的兩個面向：速度映射與力映射。建議使用第 9、10 章已建立模型的手臂。

第一部分：速度映射驗證

1. **選定姿態**：把手臂擺到一個遠離奇異點的姿態（如平面手臂的 $\theta_2 \approx 90^\circ$ ），記錄關節角度。
2. **計算理論雅可比**：用本章程式算出該姿態的 J 。
3. **單關節測試**：只讓第一關節以固定的低速轉動一小段（如 5 度），用方格紙或尺量測末端移動的距離與方向。與 J 的第一行預測值比較。
4. **重複第二關節**：同樣測試第二關節，與 J 的第二行比較。
5. **合成測試**：兩關節同時以不同速度轉動，驗證末端速度是否等於兩者貢獻的向量和。
6. **變換姿態重測**：把手臂擺到接近伸直的姿態（ $\theta_2 \approx 10^\circ$ ），重複上述測試。你會發現：同樣的關節轉動量，末端移動的距離與方向都改變了——親眼驗證「兌換率隨姿態而變」。

第二部分：力映射驗證（需可量測關節電流的平台）

1. **基準量測**：在無外力的情況下，記錄各關節維持姿態所需的電流（此為重力與摩擦的貢獻）。
2. **施加已知外力**：在末端掛上已知重量的砝碼（例如 500 g，即約 4.9 N 向下的力），記錄各

關節電流的變化量。

3. **換算扭矩**：用第 5 章的 $\tau = Kt \cdot I$ 把電流變化換算為扭矩變化。
4. **理論比對**：用 $\tau = J \cdot F$ 計算理論值，與實測比較。誤差主要來自摩擦力的不確定性。
5. **反向推算**：改以實測扭矩用 $F = (J \tau)^{-1} \cdot \tau$ 反推外力，與已知的砝碼重量比較，評估「無感測器碰撞偵測」的實際精度。
6. **姿態影響**：在接近奇異點的姿態重複實驗，觀察反推精度如何劣化。

表 11-4 雅可比驗證記錄表（第一列為範例）

測試項	姿態 (θ_1, θ_2)	輸入	理論預測	實測值	誤差	說明
單關節速度	(30°, 90°)	θ_1 轉 5°	末端移 26.2 mm，方向 120°	25.1 mm，118°	4.2%	誤差來自背隙
(自行填寫其餘測試)						

國中小也看得懂：機器手臂也有變速器

你有沒有騎過腳踏車的變速器？換到「輕齒輪」時，腳踩一圈車子跑得比較少，但踩起來很輕鬆；換到「重齒輪」時，踩一圈跑很遠，但要用力踩。踩踏圈數和前進距離之間的比例，會隨著齒輪而改變——這就是「兌換率」。

機器手臂也有這種兌換率，而且更神奇：它會隨著手臂的姿勢改變。試試看：把你的手臂彎成 90 度（像舉重一樣），然後手肘轉一點點，看看手掌移動多少；再把手臂伸直，同樣手肘轉一點點，這次手掌移動的距離就不一樣了。同樣的關節轉動，手移動的距離卻不同——這就是這一章在算的東西。

還有一件超酷的事。這個兌換率不只管「動得多快」，還管「用得出多少力」。而且它們是相反的：容易動的方向，出力就小；不容易動的方向，反而力氣大。想想看用剪刀剪厚紙板——你會把紙板放在剪刀最靠近軸的地方，因為那裡刀刃動得慢，但力氣最大！機器手臂也一樣，工程師必須算清楚：現在這個姿勢，往哪個方向比較有力、往哪個方向比較靈活。

叫獸提醒與常見錯誤

- 誤區一：以為雅可比是固定的矩陣。它是關節角度的函數，每一次姿態改變都必須重算。
- 誤區二：把角速度當成歐拉角的導數。兩者相差一個依姿態而變的轉換矩陣，混用會導致姿態運動方向錯誤。
- 誤區三：混用幾何與解析雅可比。力的對偶關係 $\tau = J^T \cdot F$ 只對幾何雅可比成立。
- 誤區四：忘記單位一致性。雅可比的線速度部分單位是「長度/弧度」，角度必須用弧度而非度數。
- 誤區五：忘記轉換座標系。雅可比是相對於特定座標系表達的，用錯座標系會使力或速度的方向完全偏離。
- 誤區六：換工具後不更新 TCP 偏移。雅可比的參考點改變時必須用伴隨變換修正，否則旋轉時末端會產生非預期的平移。
- 誤區七：直接用 J^{-1} 而不檢查條件數。接近奇異點時反矩陣的元素會暴增，應改用 DLS (第 10.7.3 節)。
- 誤區八：以為可操作度高就是好。它是各方向的平均能力；若任務只在乎某一方向，應直接檢視該方向對應的奇異值。
- 誤區九：數值微分的步長選擇不當。太大則截斷誤差大、太小則浮點捨入誤差大，建議使用中央差分且 ϵ 取 $1e-6 \sim 1e-8$ 。
- 誤區十：忽略末端速度受姿態限制。同樣的馬達能力，在不同姿態下能達到的末端速度差異可達數倍，軌跡規劃時必須考量。

課堂討論

1. 開頭故事中，末端等速直線運動而關節速度不斷變化。請解釋為什麼，並討論這對軌跡規劃 (第 15 章) 帶來什麼限制。
2. 速度橢球與力橢球的主軸方向相同、半徑互為倒數。請以「用螺絲起子鎖螺絲」為例，說明這個對偶對操作策略的影響。
3. 協作機器人靠 $F = (J^T)^+ \cdot \tau_{ext}$ 偵測碰撞，不需力覺感測器。請討論這個方法的三個限制，以及各自的改善方向。
4. 可操作度 $w = L_1 L_2 |\sin \theta_2|$ 與 θ_1 完全無關。這個結果對工作站佈局有什麼指導意義？若手臂裝在一個可旋轉的轉台上，這個結論還成立嗎？

5. 第 8 章的萬向節鎖與本章解析雅可比的奇異點是同一回事嗎？請比較「表示法的奇異點」與「機構的奇異點」，並說明改用四元數能否解決後者。

章末摘要

- 雅可比矩陣 $J(q) = \partial f / \partial q$ 把關節速度映射到末端速度： $\dot{x} = J \cdot \dot{q}$ ，它是正向運動學的局部線性化。
- 三種等價讀法：速度的兌換率、微小位移的映射、非線性函數的局部線性化。
- 偏微分法概念直接但對六軸手臂不切實際；幾何法完全重用正向運動學結果，是實務標準作法。
- 旋轉關節的雅可比行為 $[\dot{z} \times (p_n - p_j); \dot{z}]$ ，移動關節為 $[\dot{z}; 0]$ 。
- 角速度不是任何姿態表示法的導數；它由 $\dot{R} = [\omega] \times \cdot R$ 定義，可用陀螺儀直接量測，且多個角速度可直接相加。
- 幾何雅可比輸出物理角速度、解析雅可比輸出姿態參數導數，兩者相差一個轉換矩陣 $E(\varphi)$ ，後者在萬向節鎖處奇異。
- 由虛功原理得 $\tau = J^T \cdot F$ ：同一矩陣的轉置把末端力映射為關節扭矩，這是速度—力對偶性。
- 對偶性的三大應用：重力補償、力控制、以及無感測器的碰撞偵測（ $F = (J^T)^+ \cdot \tau_{\text{ext}}$ ）。
- 雅可比的座標系轉換只需旋轉（上下兩塊各乘 R ）；參考點改變則需伴隨變換（含 $-[r] \times$ 項）。
- 速度橢球由 SVD 給出：主軸方向為 U 的行、半徑為奇異值 σ_i ；最大奇異值方向最易運動、最小奇異值方向最難。
- 可操作度 $w = \sqrt{(\det(J J^T))}$ 等於所有奇異值的乘積，可用於奇異點偵測、姿態最佳化與工作站佈局。
- 力橢球與速度橢球主軸相同但半徑互為倒數：容易動的方向不容易出力，反之亦然。
- 平面 2R 手臂的可操作度為 $w = L_1 L_2 |\sin \theta_2|$ ，僅取決於肘關節角， $\theta_2 = \pm 90^\circ$ 時最靈活。
- $J \cdot \dot{q}$ 項對應動力學中的科氏力與離心力，說明這些力源於「雅可比隨姿態改變」這一事實。
- 末端可達速度受當前姿態的雅可比限制，而非僅由馬達最高轉速決定。

觀念題與計算題

觀念題

1. 定義雅可比矩陣，並說明它的三種等價讀法。
2. 說明偏微分法與幾何法的差異，並解釋為何實務上採用後者。
3. 寫出旋轉關節與移動關節的雅可比行公式，並說明各項的物理意義。
4. 為何說「角速度不是姿態角的導數」？請說明角速度的嚴格定義。
5. 有限旋轉不可交換，但角速度可以直接相加。請解釋這個看似矛盾的現象。
6. 比較幾何雅可比與解析雅可比的差異，並各舉一個適用場合。
7. 由虛功原理推導 $\tau = J^T \cdot F$ ，並說明速度—力對偶的三個推論。
8. 說明協作機器人如何在沒有力覺感測器的情況下偵測碰撞，並列出此方法的兩個限制。
9. 為何雅可比需要做座標系轉換？換工具後為何必須用伴隨變換修正？
10. 說明速度橢球的意義，以及奇異值分解中 U 與 Σ 各代表什麼。
11. 定義可操作度並說明其三個實用價值與一個限制。
12. 說明力橢球與速度橢球的關係，並以槓桿原理類比說明其物理意義。
13. 說明 $J \cdot q$ 項的物理意義，以及它與動力學中哪一項對應。
14. 為何末端可達速度不是由馬達最高轉速直接決定？這對軌跡規劃有何影響？

計算題

1. 平面 2R 手臂 $L_1 = 400 \text{ mm}$ 、 $L_2 = 300 \text{ mm}$ ，姿態 $\theta_1 = 20^\circ$ 、 $\theta_2 = 60^\circ$ 。(a) 求雅可比矩陣。(b) 驗證第一行是否等於 $[-y, x]^T$ 。(c) 若兩關節分別以 $15^\circ/\text{s}$ 與 $-20^\circ/\text{s}$ 轉動，求末端速度的大小與方向。(d) 求此姿態的可操作度。
2. 同上手臂與姿態。(a) 若要末端以 200 mm/s 沿 $+Y$ 方向運動，求所需的關節角速度。(b) 若馬達的最大關節速度為 $60^\circ/\text{s}$ ，此速度是否可達成？(c) 在此姿態下，末端沿 $+Y$ 方向的最大可達速度為多少？
3. 同上手臂與姿態。(a) 末端受到 $F = [0, -30]^T \text{ N}$ 的外力（模擬掛上約 3 kg 的負載），求各關節所需扭矩。(b) 若量測到的關節扭矩為 $\tau = [8.0, 3.0]^T \text{ N}\cdot\text{m}$ ，反推末端外力。(c) 若第一關節的扭矩量測有 $\pm 0.5 \text{ N}\cdot\text{m}$ 的誤差，估算對推得外力的影響。
4. 某平面 2R 手臂 $L_1 = L_2 = 250 \text{ mm}$ 。(a) 寫出可操作度隨 θ_2 變化的表達式。(b) 求 $\theta_2 = 15^\circ$ 、 45° 、 90° 、 150° 時的可操作度。(c) 若要求可操作度不低於最大值的 50% ， θ_2 的

允許範圍為何？(d) 由此結果，建議工作站的主要作業區應配置在距離基座多遠的環狀範圍內？

動手做與專題延伸

本章實作：雅可比計算模組

延續第 9、10 章的程式，實作通用的雅可比計算模組。建議依下列階段完成：

- **階段一**：以幾何法 (11.4.4 節) 實作通用的 $6 \times n$ 雅可比計算，支援任意 DH 參數表與 R/P 關節類型。
- **階段二**：實作數值微分版本作為驗證基準，確認兩者在多組隨機姿態下的一致性 (誤差應小於 $1e-6$)。
- **階段三**：加入座標系轉換與 TCP 偏移的伴隨變換功能。
- **階段四**：實作可操作度、奇異值分解與條件數的計算，並整合為一個「姿態健康度」報告函式。
- **階段五**：回頭替換第 10 章數值逆運動學中的數值微分雅可比，比較計算速度的改善幅度。

專題延伸

- **速度橢球視覺化**：延續第 9、10 章的視覺化工具，在末端位置繪出速度橢球 (以 SVD 的主軸方向與奇異值為半徑)。加上滑桿控制關節角度，即時觀察橢球如何隨姿態變形——當手臂接近伸直時，你會看到橢球被壓扁成一條線。
- **可操作度地圖**：對工作空間中的每一個點，計算其可操作度 (若有多組逆解則取較大者)，以熱力圖繪出。這張圖能直觀顯示「機器人在哪些區域最靈活」，是工作站佈局的實用工具。
- **進階：力橢球與任務適配分析**。針對一個具體任務 (例如「垂直向下插入，需 50 N 推力與 0.1 mm 的水平定位精度」)，在工作空間中找出所有能同時滿足「該方向力橢球半徑足夠大」與「水平方向速度橢球半徑足夠小 (即精度高)」的區域。將結果與可操作度地圖比較，討論為何「最靈活的位置」未必是「最適合這個任務的位置」。

參考文獻與延伸閱讀

- Craig, J. J., Introduction to Robotics: Mechanics and Control, 4th ed., Pearson (第 5

章：雅可比——速度與靜力，本章主要參考）。

- Yoshikawa, T., "Manipulability of Robotic Mechanisms," International Journal of Robotics Research, 1985 (可操作度概念的原始論文)。
- Lynch, K. M. and Park, F. C., Modern Robotics, Cambridge University Press (第 5 章：速度運動學與靜力，含旋量與螺旋理論觀點)。
- Siciliano, B. et al., Robotics: Modelling, Planning and Control, Springer (第 3 章：微分運動學與靜力)。
- Murray, R. M., Li, Z. and Sastry, S. S., A Mathematical Introduction to Robotic Manipulation, CRC Press (旋量與伴隨變換的嚴謹處理)。
- Salisbury, J. K. and Craig, J. J., "Articulated Hands: Force Control and Kinematic Issues," IJRR, 1982 (雅可比在多指手力控制中的應用)。
- Chiacchio, P. et al., "Global Task Space Manipulability Ellipsoids for Multiple-Arm Systems," IEEE Trans. Robotics, 1991 (可操作度橢球的延伸)。
- De Luca, A. and Mattone, R., "Sensorless Robot Collision Detection and Hybrid Force/Motion Control," ICRA 2005 (無感測器碰撞偵測的經典論文)。
- Corke, P., Robotics, Vision and Control, 3rd ed., Springer (含雅可比與可操作度的 MATLAB / Python 實作範例)。
- Pinocchio 與 KDL 函式庫技術文件：高效的雅可比計算實作，含伴隨變換與座標系轉換功能。

第 12 章

奇異點、冗餘與多任務運動

編著：李世淵（ 機器人叫獸 ）

助教：李天宇、李宇晴

【章首情境圖建議】畫面左半為一支六軸手臂逐漸伸直的四格連續動作，末端速度橢球（藍色）隨之被壓扁，最後退化成一條線段；橢球下方標註條件數由 $3 \rightarrow 12 \rightarrow 58 \rightarrow \infty$ 的變化。右半為一支七軸手臂，末端夾爪固定在同一位置（綠色圓圈標註「末端不動」），手肘卻沿一條圓弧畫出三重殘影，旁註「零空間自運動」。中央以一條紅色分隔線區隔，上方文字：奇異點——關上一扇門；下方文字：冗餘——開啟另一扇窗。

叫獸開講 「手臂伸得太直，反而最沒有力？」

六軸手臂正在執行一段簡單的直線軌跡：由左而右，橫越工作台，速度每秒 15 公分。前半段一切順利。到了中段，手臂逐漸伸直，控制器突然發出刺耳的警報，螢幕上跳出一行紅字——**接近奇異點，第五軸速度超限**——然後手臂僵在原地不動。

小翰盯著螢幕：「奇怪，目標點明明在工作範圍內啊，程式也沒有語法錯誤。」

叫獸把手臂手動移回起點，指著第五軸：「你看這個關節。當手腕的第四軸與第六軸轉到互相平行時，會發生什麼事？」他慢慢轉動第四軸，接著轉第六軸——兩者產生了**完全相同**的末端轉動。

小紓先反應過來：「那不就是.....第 8 章的萬向節鎖？」

「幾乎一模一樣，但這次是**機構**的，不是數學表示法的。」叫獸在白板上寫下一行字：**兩個關節做同一件事，等於少了一個關節**。「你的手臂號稱有六個自由度，但在這個姿態下，它實際上只剩五個。少掉的那一個方向，你再怎麼轉馬達都到不了。」

他接著問了一個更尖銳的問題：「但為什麼控制器不是直接說『到不了』，而是說『速度超限』？」教室安靜下來。

叫獸畫了一個橢圓，然後把它壓扁成一條線：「因為問題不在『到不了』，而在『代價』。上一章的速度橢球，在這裡被壓扁了。末端想在那個消失的方向移動 0.1 公釐，關節就得轉幾百度——控制器算出了這個數字，發現超過馬達極限，只好停機保護。」他停了一下，「如果控制器沒有這個保護，會發生什麼事？手臂會用它最快的速度甩過去。這就是為什麼工業機器人的說明書上，奇異點永遠寫在安全章節裡。」

下課前，他在白板上並排寫下四件事：**第 8 章的萬向節鎖、第 9 章的手臂完全伸直、第 10 章的兩組解合而為一、第 11 章的可操作度歸零**。「這四件事，你們可能以為是四個不相干的現象。今天我要證明給你們看——它們是同一件事。而串起它們的，是一個線性代數裡最基本的概念：**矩陣的秩**。」

本章為什麼重要

奇異點是機器人學中最重要的「壞消息」。它無法用更好的控制演算法消除、無法用更強的

馬達克服，因為它是**機構幾何本身的性質**。任何機器手臂都有奇異點，這是數學的必然。

理解奇異點有三層價值。**第一層是安全**：奇異點附近的關節速度會暴增，是工業現場最常見的異常停機與機構損傷原因之一。**第二層是設計**：知道奇異點在哪裡，才能規劃工作站佈局與軌跡，讓機器人在「舒服」的區域作業。**第三層是理論**：奇異點的分析統一了前面四章的多個現象，並自然引出冗餘、零空間與多任務控制——這些正是現代協作機器人與人形機器人的核心技術。

本章也是第二篇的收尾。讀完之後，操作臂運動學的四塊拼圖（正向、逆向、速度、奇異）全部就位，你將具備完整分析一支機械手臂運動能力的工具。

學習目標

- 能以雅可比矩陣的秩定義奇異點，並說明其物理意義。
- 能區分邊界奇異點與內部奇異點，並各舉實例。
- 能運用行列式或最小奇異值判斷是否接近奇異點。
- 能說明奇異點處的三個現象：運動能力喪失、關節速度暴增、逆解退化。
- 能運用奇異值分解找出失去的運動方向與對應的關節速度方向。
- 能說明條件數的意義並用它評估姿態的數值品質。
- 能列舉六軸手臂的三類典型奇異點及其發生條件。
- 能說明五種處理奇異點的實務策略及其取捨。
- 能定義冗餘自由度並說明零空間的幾何意義。
- 能運用零空間投影實現次要目標最佳化。
- 能說明多任務優先權控制的原理與分層逆運動學的作法。

先備知識

第 11 章的雅可比矩陣（尤其是幾何構造法、速度橢球與可操作度）、第 10 章的數值逆運動學與偽逆、第 9 章的正向運動學。線性代數方面需要矩陣的秩、線性相依、零空間與奇異值分解的概念——本章會以幾何直覺重新說明這些概念，不需要事先精通。

核心名詞中英對照

表 12-1 本章核心名詞

中文	英文	說明
奇異點 / 奇異位形	Singularity	雅可比矩陣失去滿秩的機器人位形
秩	Rank	矩陣中線性獨立的行 (或列) 的數目
滿秩	Full Rank	秩等於行數與列數中較小者
邊界奇異點	Boundary Singularity	發生於工作空間邊界的奇異點
內部奇異點	Interior Singularity	發生於工作空間內部的奇異點
腕部奇異點	Wrist Singularity	手腕兩軸共線造成的奇異點
肩部奇異點	Shoulder Singularity	腕心落在第一軸線上造成的奇異點
肘部奇異點	Elbow Singularity	手臂完全伸直或折疊造成的奇異點
奇異值	Singular Value	SVD 分解中對角矩陣的元素
條件數	Condition Number	最大與最小奇異值之比
零空間	Null Space	使 $J \cdot \dot{q} = 0$ 的關節速度所張的空間
自運動	Self-motion	末端不動但關節持續變化的運動
冗餘度	Degree of Redundancy	關節數減去任務維度
零空間投影	Null Space Projection	把次要目標投影到不影響主任務的方向
任務優先權	Task Priority	多任務衝突時的執行順序安排
分層逆運動學	Hierarchical IK	依優先權逐層求解多任務的方法

12.1 奇異點的定義

12.1.1 由秩出發

第 11 章建立了速度關係 $\dot{x} = J(q) \cdot \dot{q}$ 。現在問一個問題：這個映射能覆蓋所有的末端速度方向嗎？

答案取決於雅可比矩陣的秩。矩陣 J 的秩，等於它的行向量中線性獨立的個數，也等於映射的像空間 (image) 的維度。若 J 是 6×6 的方陣：

$\text{rank}(J) = 6 \Rightarrow$ 末端可往六個方向自由運動 (非奇異)

$\text{rank}(J) < 6 \Rightarrow$ 至少有一個方向到不了 (奇異)

奇異點的嚴格定義：雅可比矩陣失去滿秩的位形。

秩下降的直觀意義是：某兩行 (或多行) 變成線性相依。換句話說，某兩個關節產生了「重

複」的運動效果——就像開頭故事中手腕的第四軸與第六軸轉出同樣的結果。兩個關節做同一件事，等於少了一個關節，於是有一個方向永遠到不了。

12.1.2 三種等價的判準

判斷是否奇異，有三種數學上等價但用途不同的方法。

判準一：行列式為零（僅適用於方陣）。

$$\det(J) = 0 \Leftrightarrow \text{奇異}$$

計算最快，但只能給出「是或否」，無法量化「有多接近」，且非方陣（冗餘機器人）無法使用。

判準二：可操作度為零（第 11.8.2 節）。

$$w = \sqrt{\det(J J^T)} = 0 \Leftrightarrow \text{奇異}$$

適用於任意形狀的矩陣，且能量化接近程度。但它是所有方向的乘積，某一方向的能力嚴重下降時， w 會被其他方向的大值稀釋。

判準三：最小奇異值為零（最推薦）。

$$\sigma_{\min} = 0 \Leftrightarrow \text{奇異}$$

直接反映「最弱的那個方向」，是實務上最可靠的指標。 $\sigma_{\min}$ 的值本身就代表該方向的運動能力大小。

實務上的標準作法是監控 $\sigma_{\min}$ ：當它低於某個門檻（例如最大奇異值的 5%）時，就啟動第 12.5 節的防護措施。

12.1.3 奇異點的三個症狀

在奇異點處，會同時出現三個彼此相關的現象——這三個症狀分別對應到前面三章遺留的謎題。

症狀一：運動能力喪失。末端在某個特定方向上完全無法移動，無論關節如何轉動。這是第 9 章例題 9-1(b) 中「手臂完全伸直後無法再沿徑向前進」的數學解釋。

症狀二：關節速度暴增。要在接近該方向（而非恰好在該方向）移動時，所需的關節速度趨於無窮大。這是開頭故事中控制器警報的原因，也是第 10 章偽逆法數值爆炸的來源。

$$\dot{q} = J^{-1} \cdot \dot{x}, \quad \text{當 } \sigma_{\min} \rightarrow 0 \text{ 時, } J^{-1} \text{ 的元素 } \rightarrow \infty$$

反矩陣的元素與奇異值的倒數成正比，因此最小奇異值趨近於零時，關節速度指令會趨於無窮大。

症狀三：逆解退化。原本分離的多組逆解，在奇異點處合併。這是第 10 章例題 10-2(b) 中「手臂完全伸直時兩組解合而為一」的現象。

◎ 為什麼症狀二比症狀一更危險

症狀一（到不了）看起來嚴重，實際上是安全的——控制器算出無解，直接報錯停止。真正危險的是症狀二：在奇異點**附近**（而非恰好在上面），數學上仍然有解，控制器會忠實地執行它算出的指令——而那個指令可能是「第五軸以每秒 800 度轉動」。若沒有速度限制保護，手臂會以最高速度暴衝。這就是為什麼實務上要監控的不是「是否恰好在奇異點」，而是「距離奇異點多近」。

12.2 邊界奇異點與內部奇異點

12.2.1 邊界奇異點

邊界奇異點發生在工作空間的邊界上——手臂完全伸直（外邊界）或完全折疊（內邊界）。此時末端已到達可達範圍的極限，無法再沿徑向往外（或往內）移動。

以平面 2R 手臂為例，第 11 章例題 11-3 已導出其可操作度為 $w = L_1 L_2 |\sin \theta_2|$ 。因此：

$$\theta_2 = 0^\circ \Rightarrow w = 0 \quad (\text{完全伸直，外邊界奇異點})$$

$$\theta_2 = 180^\circ \Rightarrow w = 0 \quad (\text{完全折疊，內邊界奇異點})$$

平面 2R 手臂的兩個奇異位形，皆只與肘關節角度有關。

邊界奇異點的一個重要特性是：**它在實務上相對容易避開**。只要規劃時保留工作空間的餘裕（第 4.10.3 節建議至少 10%），就不會接近它。此外，操作員的直覺也能幫忙——「手臂快伸直了，該把工件移近一點」是很自然的判斷。

12.2.2 內部奇異點

內部奇異點則麻煩得多。它發生在工作空間**內部**，位置往往不直觀，操作員在示教時完全感覺不到危險，直到執行軌跡時突然停機。

內部奇異點的成因是**兩個或多個關節軸線排成特殊的幾何關係**——通常是共線或平行。此時它們產生重複的運動效果，秩下降。開頭故事中手腕第四、六軸平行的情形，正是典型的內部奇異點。

這類奇異點的三個特性使它特別危險：一、位置分散在工作空間內部，難以憑直覺避開；二、可能恰好落在常用的作業區域；三、通過它時末端位置變化不大，但關節可能需要大幅翻轉。

圖 12-1 邊界奇異點與內部奇異點

建議配圖：左圖為平面 2R 手臂的工作空間（環狀區域），以紅色粗線標出外圓（完全伸直）與內圓（完全折疊）兩個邊界奇異點，並在旁畫出兩種手臂姿態的示意。右圖為六軸手臂的側視圖，以紅色標出三處內部奇異：手腕兩軸共線（腕部）、腕心通過第一軸線（肩部）、手臂完全伸直（肘部）。下方以顏色深淺表示可操作度分布，深色區為高可操作度、白色為奇異區。

12.3 六軸手臂的三類典型奇異點

12.3.1 腕部奇異點

發生條件：第四軸與第六軸共線（通常是第五軸轉到 0° 或 180° 時）。

物理現象：第四軸與第六軸的轉動效果完全相同，等於少了一個自由度。末端無法繞垂直於這條共線軸的某個方向轉動。

為何常見：許多常用姿態（如夾爪垂直向下、手腕保持水平）恰好會讓第五軸接近 0° 。這是工業現場最頻繁遇到的奇異點，也是開頭故事的情境。

典型症狀：末端只需要轉一個很小的角度，第四軸與第六軸卻同時以極高速度反向旋轉（一個 $+400^\circ/\text{s}$ 、另一個 $-400^\circ/\text{s}$ ），淨效果卻只有幾度的末端轉動。這在示教器上看起來就像手腕「發瘋似地扭動」。

12.3.2 肩部奇異點

發生條件：手腕中心點落在第一軸（腰部旋轉軸）的軸線上。

物理現象：此時無論第一軸如何轉動，手腕中心都不會移動（因為它就在轉軸上）。末端失去沿某個切線方向運動的能力。

典型症狀：當軌跡經過機器人正上方或正下方（基座軸線附近）時，第一軸需要瞬間旋轉 180° 。這是「機器人突然整個轉半圈」這類詭異行為的常見原因。

實務對策：工作站佈局時，避免讓作業路徑通過基座的正上方區域。這也是為什麼許多工業機器人的說明書會標出「基座上方的禁止作業圓柱區」。

12.3.3 肘部奇異點

發生條件：手臂完全伸直（或完全折疊），即上臂與前臂共線。

物理現象：這是邊界奇異點在空間手臂上的版本。末端無法再沿徑向（由肩部指向腕心的方

向) 移動。

典型症狀：要求末端繼續往外移動時，第二軸與第三軸的速度暴增，控制器報「超出工作空間」或「速度超限」。

實務對策：規劃時保留工作半徑的餘裕。多數控制器會在接近伸直極限時自動降速。

表 12-2 六軸手臂三類奇異點的比較

類型	發生條件	失去的運動方向	典型症狀	實務對策
腕部奇異	第 5 軸 $\approx 0^\circ$ 或 180° (第 4、6 軸共線)	繞某軸的轉動	第 4、6 軸高速反向旋轉	調整工具姿態、加入姿態餘隙
肩部奇異	腕心落在第 1 軸線上	沿切線方向的平移	第 1 軸瞬間翻轉 180°	避開基座正上方區域
肘部奇異	手臂完全伸直或折疊	沿徑向的平移	第 2、3 軸速度暴增	保留工作半徑 10% 以上餘裕

◎ 一個容易忽略的事實

奇異點的位置與機構尺寸無關，只與構型有關。無論你買的是臂展 500 mm 還是 3000 mm 的六軸手臂，只要都是球形手腕的 PUMA 構型，它們的奇異位形 (以關節角度描述) 完全相同。這意味著：在小型手臂上學到的奇異點知識，可以直接套用到大型手臂上；而在模擬器中驗證過的軌跡，也不會因為換成不同尺寸的實機而突然遇到新的奇異點。

12.4 奇異值分解的幾何解讀

12.4.1 SVD 告訴我們什麼

第 11.8.1 節已介紹奇異值分解 $J = U \Sigma V^T$ 。現在從奇異點的角度重新解讀它的三個矩陣：

U (左奇異向量)：其行向量是**任務空間**中的一組正交方向。第 i 行對應第 i 個奇異值，代表末端運動的一個主軸方向。

Σ (奇異值)：對角線上的 $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_m \geq 0$ ，代表各方向的「運動增益」。 σ_i 的意義是：關節以單位速度沿第 i 個關節方向運動時，末端在第 i 個任務方向上獲得的速度。

V (右奇異向量)：其行向量是**關節空間**中的一組正交方向。第 i 行是產生第 i 個末端運動方向所需的關節速度組合。

因此 SVD 給出一組完整的「對應關係」：沿 $\mathbf{v}_i$ 方向轉動關節，末端會沿 $\mathbf{u}_i$ 方向移動，增益為 σ_i 。

12.4.2 奇異點的精確描述

有了 SVD，奇異點的描述變得極為精確：

$$\sigma_k = 0 \Rightarrow \text{末端無法沿 } \mathbf{u}_k \text{ 方向運動} \\ \Rightarrow \text{沿 } \mathbf{v}_k \text{ 方向轉動關節，末端完全不動（自運動）}$$

第 k 個奇異值歸零時， $\mathbf{u}_k$ 是「失去的任務方向」、 $\mathbf{v}_k$ 是「無效的關節運動方向」。

這個結果同時給出了兩個重要資訊。第一，失去的是哪個方向——這對決定如何繞道極為有用。第二，哪組關節運動變成徒勞——這正是零空間的來源（12.6 節）。

以開頭故事的腕部奇異為例：SVD 會告訴你 $\mathbf{u}_k$ 是「繞某個特定軸的轉動」（末端到不了的方向），而 $\mathbf{v}_k$ 是 $[0, 0, 0, +1, 0, -1]^T$ （第四軸與第六軸等量反向轉動——確實，這樣轉的話末端完全不動）。

12.4.3 條件數：數值品質的指標

條件數定義為最大與最小奇異值之比：

$$\kappa = \sigma_{\max} / \sigma_{\min}$$

條件數。 $\kappa = 1$ 代表各方向能力完全相同（等向性，最理想）； $\kappa \rightarrow \infty$ 代表接近奇異。

條件數的實用價值在於它是無量綱的相對指標，不受機器人尺寸影響。這使得不同機型之間可以直接比較。實務上的經驗參考值為：

表 12-3 條件數的實務判讀

條件數 κ	狀態	說明	建議措施
1 ~ 5	優良	各方向能力接近，數值穩定	理想作業區，可全速運行
5 ~ 20	可接受	有明顯的方向差異但仍可控	正常作業，注意速度規劃
20 ~ 100	警戒	接近奇異，關節速度開始放大	啟動 DLS 阻尼、降低速度
> 100	危險	極接近奇異，數值不可靠	拒絕執行或大幅降速繞道

此外還有一個相關指標：等向性（isotropy），定義為 $1/\kappa$ 。它衡量速度橢球接近正球的程度。設計工作站時，讓主要作業區落在等向性高的區域，能同時獲得良好的運動性能與數值穩定。

性。

12.5 奇異點的處理策略

12.5.1 五種策略

面對奇異點，工程上有五種處理方式。它們並非互斥，實務系統通常同時採用多種。

策略一：軌跡繞道（規劃層）。在路徑規劃階段就避開奇異區域。作法是把可操作度或條件數納入路徑的代價函數，讓規劃器自動偏好「舒服」的區域。優點是根本解決、不需即時處理；缺點是路徑會偏離最短路徑，且需要事先知道奇異點分布。

策略二：降速（執行層）。偵測到 $\sigma_{\min}$ 下降時，按比例降低末端速度，使關節速度維持在限制內。優點是保持路徑精度不變；缺點是節拍時間拉長，且在真正的奇異點上仍然無解（速度必須降為零）。

$$v_{\max_allowed} = \sigma_{\min} \cdot \dot{q}_{\max}$$

可容許的末端速度與最小奇異值成正比。這是降速策略的核心公式。

策略三：阻尼最小平方（數值層）。即第 10.7.3 節的 DLS。以 λ^2 阻尼項確保矩陣可逆，容許末端稍微偏離目標路徑以換取關節速度的可控。

$$\dot{q} = J^T(JJ^T + \lambda^2 I)^{-1} \cdot \dot{x}$$

可變阻尼的常見設計： $\lambda^2 = \lambda_0^2(1 - \sigma_{\min}/\sigma_{\text{門檻}})^2$ （ $\sigma_{\min}$ 低於門檻時才啟動阻尼）。

策略四：改變構型（規劃層）。利用第 10 章的多解結構，切換到另一組不會經過奇異點的解。缺點是切換過程本身需要大幅運動，只能在軌跡的起訖點進行。

策略五：增加自由度（機構層）。加裝第七軸或把手臂裝在行走軸上，使系統成為冗餘機構，即可用零空間繞開奇異點（12.6、12.7 節）。這是最根本的解決方案，也是協作機器人多採七軸設計的原因之一。

表 12-4 五種奇異點處理策略的比較

策略	層級	路徑精度	節拍影響	實作難度	適用場合
軌跡繞道	規劃	路徑改變	略增	中	離線編程、已知環境
降速	執行	完全保持	顯著增加	低	通用，工業控制器標配
DLS 阻尼	數值	略有偏差	幾乎不變	低	即時控制、遙操作

策略	層級	路徑精度	節拍影響	實作難度	適用場合
改變構型	規劃	完全保持	切換時大幅增加	中	軌跡起訖點
增加自由度	機構	完全保持	不變	高 (需硬體)	新系統設計、協作機器人

12.5.2 實務上的複合策略

成熟的工業控制器通常這樣組合：離線編程時用**策略一**檢查並修正路徑；執行時持續監控 $\sigma_{\min}$ ，接近門檻時啟動**策略二**降速；若仍不足則啟動**策略三**的 DLS 阻尼；若條件數超過危險門檻則直接拒絕執行並報錯。這是一個由粗到細的多層防護。

對協作機器人與研究平台，則多採用**策略五**——直接用七軸機構，把問題在設計階段就解決掉大半。

◎ 奇異點無法被「消除」

常有學生問：能不能設計一支沒有奇異點的機械手臂？答案是**不能**。這是一個拓撲學上的結論：任何以旋轉關節構成的串聯手臂，其工作空間中必定存在奇異位形。這與第 8 章「任何用三個數描述三維旋轉的方法必有奇異點」是同一類的數學必然。工程上能做的，只有把奇異點「移到」不常用的區域、或增加自由度讓機器人有繞道的餘裕——而不是消滅它們。

12.6 冗餘與零空間

12.6.1 冗餘的定義

第 4.9 節已介紹冗餘的概念：當機器人的自由度多於任務所需時，稱為冗餘。現在可以用雅可比精確定義：

$$\text{冗餘度 } r = n - m \quad (n \text{ 為關節數、} m \text{ 為任務維度})$$

當 $n > m$ 時， J 是「寬矩陣」（行多於列），方程式 $J \cdot \dot{q} = \dot{x}$ 的解不唯一。

最典型的例子是七軸手臂執行六維任務（ $r = 1$ ），以及人類手臂（七自由度）執行「把手掌放到指定位姿」（六維任務）——正是第 4 章開頭那個「手掌貼牆卻能抬起手肘」的實驗。

12.6.2 零空間的幾何意義

冗餘帶來的關鍵結構是**零空間**（null space）：

$$N(J) = \{ \dot{q} : J \cdot \dot{q} = 0 \}$$

零空間是所有「不產生任何末端運動」的關節速度所構成的集合。

這個定義的物理意義極為優美：沿零空間方向轉動關節，末端完全不動，但手臂的形狀改變了。這種運動稱為**自運動**（self-motion）。人類手臂那個「手掌固定、手肘上下擺動」的動作，就是沿零空間的自運動。

零空間的維度等於冗餘度 r （在非奇異的情況下）。七軸手臂執行六維任務時，零空間是一維的——因此自運動只有一個方向（手肘繞著「肩腕連線」這條軸擺動），這正是為什麼人類手臂的自運動看起來像手肘畫一個圓弧。

圖 12-2 冗餘手臂的零空間自運動

建議配圖：一支七軸手臂（或人類手臂）的三張疊影，末端夾爪位置與姿態完全相同（以綠色圓圈標出並註明「末端固定」），但手肘位置不同，沿一個圓弧分布（以藍色虛線畫出手肘的移動軌跡，並標註「自運動流形」）。旁邊以小圖標出「肩—腕連線」這條自運動的旋轉軸。下方文字：末端不動，形狀可變——這就是零空間。

12.6.3 零空間出現在奇異點

這裡有一個容易混淆但極為重要的區別。零空間有**兩種**來源：

來源一：冗餘造成的零空間（好的）。當 $n > m$ 時，即使 J 滿秩，零空間仍然存在且維度為 $n - m$ 。這是可用的資源，讓我們能在完成任務的同時追求次要目標。

來源二：奇異造成的零空間（壞的）。當 J 失去滿秩時（無論是否冗餘），會出現額外的零空間維度。這對應到 12.4.2 節的 v_k 方向——沿該方向轉動關節是徒勞的，因為末端不動，但我們也失去了對應的 u_k 任務方向。

兩者的差別在於：冗餘的零空間**不伴隨任務方向的損失**（機器人仍能到達所有六個方向），而奇異的零空間**伴隨著任務方向的損失**。因此判斷方法是：檢查 J 的秩是否等於 $\min(m, n)$ ——是則為冗餘零空間，否則為奇異零空間。

12.7 零空間投影與次要目標

12.7.1 通解的結構

對冗餘機器人，方程式 $J \cdot \dot{q} = \dot{x}$ 有無限多解。線性代數告訴我們，其通解的結構為「特解 + 齊次解」：

$$\dot{q} = J^+ \cdot \dot{x} + (I - J^+ J) \cdot z$$

第一項是偽逆給出的特解（範數最小的解）、第二項是零空間中的任意元素。 z 可以是任何 n 維向量。

矩陣 $P = (I - J^+J)$ 稱為**零空間投影算子**。它有兩個關鍵性質：

$$J \cdot P = 0 \quad (\text{投影後的結果必在零空間中})$$

$$P \cdot P = P \quad (\text{冪等性，投影兩次等於投影一次})$$

這保證了第二項無論 z 取什麼值，都不會影響末端運動——主任務永遠被完美執行。

這個結構的工程意義極為重大：它讓我們可以「免費」追求次要目標。主任務（末端軌跡）由第一項保證，而第二項提供了一個完全不干擾主任務的通道，可以用來做任何我們想做的事。

12.7.2 四種常見的次要目標

次要目標透過選擇 z 來指定。標準作法是把 z 取為某個目標函數 $H(q)$ 的梯度：

$$z = \pm k \cdot \nabla H(q)$$

取正號代表最大化 H 、負號代表最小化。 k 為增益，控制次要目標的積極程度。

目標一：遠離關節極限。定義 H 為各關節與其行程中點的距離平方和，最小化它可使機器人傾向於保持在關節行程的中央：

$$H(q) = \sum_i (q_i - q_{\text{mid},i})^2 / (q_{\text{max},i} - q_{\text{min},i})^2$$

常用的關節極限迴避函數。這是第 10.8.2 節提到的第三種處理方式。

目標二：最大化可操作度（避開奇異點）。取 $H(q) = w(q)$ 並最大化它。這使機器人在執行任務的同時，自動維持在靈活的姿態——這是策略五（12.5.1 節）的具體實現。

目標三：避開障礙物。定義 H 為手臂各點與障礙物的最小距離，最大化它。這讓手臂能在末端沿既定軌跡運動的同時，自動「閃避」環境中的障礙——正是第 4 章開頭那個六軸手臂撞到治具的問題的解方。

目標四：維持自然或舒適姿勢。定義 H 為與某個參考姿態 q_{ref} 的距離，最小化它。這對人形機器人與協作機器人特別重要——讓機器人的動作看起來自然，能顯著提升人類的接受度（第 35 章）。

表 12-5 四種次要目標的設計

次要目標	目標函數 $H(q)$	最佳化方向	典型應用
遠離關節極限	與行程中點的正規化距離平方和	最小化	長時間連續作業、避免中途停機

次要目標	目標函數 $H(q)$	最佳化方向	典型應用
最大化可操作度	$w = \sqrt{\det(J J^T)}$	最大化	避開奇異點、維持靈活姿態
避開障礙物	手臂各點與障礙的最小距離	最大化	狹窄空間作業、人機共處
維持自然姿勢	與參考姿態的加權距離	最小化	人形機器人、協作機器人

12.7.3 實作要點與陷阱

要點一：增益不能太大。次要目標的增益 k 過大時，雖然理論上不影響主任務，但數值誤差會累積，且關節可能以很高的速度做自運動。實務上應設定自運動的速度上限。

要點二：次要目標之間可能衝突。同時要求「遠離關節極限」與「最大化可操作度」時，兩個梯度方向可能相反。此時需要加權，或改用 12.8 節的分層架構。

要點三：投影算子的計算成本。 $P = (I - J^+J)$ 需要計算偽逆並做矩陣乘法，對高自由度系統成本不低。實務上常用 SVD 一次算出所有需要的量，或採用遞迴的分層算法。

要點四：零空間在奇異點會「變質」。如 12.6.3 節所述，接近奇異時零空間會混入「奇異造成的方向」，此時沿零空間運動可能無法真正達成次要目標。因此應同時監控 $\sigma_{\min}$ 。

12.8 多任務與分層逆運動學

12.8.1 多任務的需求

實際的機器人往往必須同時滿足多個任務。以一台雙臂協作機器為例，它可能同時需要：右手跟隨軌跡（6 維）、左手保持不動（6 維）、頭部相機朝向工件（2 維）、身體保持平衡（2 維）、以及所有關節遠離極限。若機器人只有 14 個自由度，這些任務的總維度已經超過——必然有些任務無法完全滿足。

此時需要一套機制來決定：哪些任務優先、哪些可以妥協。這就是任務優先權（task priority）的概念。

12.8.2 合併為單一大任務

最簡單的作法是把所有任務堆疊成一個「大任務」，各自加權：

$$\Phi(q) = \begin{bmatrix} \sqrt{\rho_1} (\phi_1(q) - y_1^*) \\ \sqrt{\rho_2} (\phi_2(q) - y_2^*) \\ \vdots \end{bmatrix}$$

每個任務 i 有自己的運動圖 φ_i 、目標值 y_i^* 與權重 ρ_i 。權重越大代表越重要。

對應的大雅可比矩陣就是各任務雅可比的垂直堆疊。求解方式與單一任務相同，只是矩陣變高了。

這個作法的優點是實作簡單、所有任務同時被考慮。缺點是**任務之間會互相妥協**——即使把某個任務的權重設得很高，它仍可能因為其他任務而產生誤差。對於「絕對不能違反」的任務（如關節極限、避免碰撞），這是不可接受的。

12.8.3 分層逆運動學

嚴格的作法是**分層**：先完全滿足最高優先權的任務，再在它的零空間中滿足次高優先權的任務，依此類推。

以兩層為例，主任務為 $\dot{x}_1$ （雅可比 J_1 ）、次任務為 $\dot{x}_2$ （雅可比 J_2 ）：

$$\dot{q}' = J_1^+ \dot{x}_1 + (J_2 P_1)^+ \cdot (\dot{x}_2 - J_2 J_1^+ \dot{x}_1) \quad , \text{ 其中 } P_1 = I - J_1^+ J_1$$

第一項完全滿足主任務；第二項在主任務的零空間中盡量滿足次任務。括號中的修正項扣除了主任務運動對次任務造成的「副作用」。

這個公式可以遞迴推廣到任意多層。它保證了**嚴格的優先權**：低優先權的任務絕不會干擾高優先權的任務，即使前者完全無法達成。

代價是計算成本較高（每一層都要做一次偽逆），且層數越多、可用的自由度越少——到某一層時零空間可能已經耗盡，該層以下的任務完全無法執行。

12.8.4 實務上的折衷

Toussaint 等學者指出一個實務上的觀察：**嚴格的分層在數值上未必最穩健**。因為每一層的投影都會使矩陣的條件數惡化，多層之後數值品質可能很差。

因此實務上常採取折衷：對真正不可違反的約束（碰撞、關節極限）使用嚴格分層或硬約束；對其餘任務則使用加權合併，並以權重的數量級差異（例如相差 100 倍）來近似優先權。這樣既保留了優先權的效果，又維持了數值穩定性。

這個主題在第 17 章（操作空間控制）與第 28 章（人形機器人全身控制）會再次出現，屆時還要加入動力學的考量。

12.9 第二篇的總結

12.9.1 四塊拼圖

走完第 8 至 12 章，操作臂運動學的完整體系已經建立。回顧這五章：

第 8 章建立了描述空間的語言——座標系、旋轉矩陣、四元數、齊次變換。這是所有幾何運算的字母表。

第 9 章 (正向運動學) 回答「手在哪裡」：把關節角度代入 DH 參數，連乘變換矩陣即得末端位姿。解必定存在且唯一。

第 10 章 (逆向運動學) 回答「手該怎麼擺」：由目標位姿反求關節角度。可能無解、多解或無限多解，需用封閉解或數值迭代。

第 11 章 (雅可比) 回答「手能怎麼動」：把關節速度映射到末端速度，並透過轉置連結力與扭矩。

第 12 章 (奇異點與冗餘) 回答「手何時動不了、以及多餘的自由度怎麼用」：由雅可比的秩統一解釋所有的退化現象，並用零空間開發冗餘的價值。

12.9.2 串起來的線索

本章開頭承諾要證明四個現象是同一件事，現在可以完整回答：

表 12-6 四個現象的統一解釋

現象	出現章節	表面症狀	統一的數學解釋
萬向節鎖	第 8.3.3 節	兩軸對齊，失去一個旋轉自由度	解析雅可比中的 $E(\varphi)$ 矩陣奇異
手臂完全伸直	第 9 章例題 9-1	無法再沿徑向前進	幾何雅可比失去滿秩 (肘部奇異)
兩組逆解合併	第 10 章例題 10-2	肘上與肘下的解重合	解空間在奇異點處退化
可操作度歸零	第 11 章例題 11-3	$w = L_1 L_2 \sin \theta_2 \rightarrow 0$	最小奇異值 $\sigma_{\min} \rightarrow 0$

四者的共同根源是：**雅可比矩陣失去滿秩**。這正是為什麼線性代數中一個看似抽象的概念——矩陣的秩——會成為機器人學的核心工具。

12.9.3 接下來：讓運動學動起來

到目前為止，我們的分析都是**運動學的** (kinematic) ——只談位置、速度與幾何，完全沒有考慮質量、慣性與力的動態效應。這隱含了一個假設：關節可以瞬間達到任何我們指定的速度。

這個假設在低速時尚可接受，但在高速運動、重負載或需要精確力控制時就會失效。手臂有質量、有慣性、會因重力下垂、加速時需要額外的力矩、旋轉時會產生科氏力——這些都是**動力學** (dynamic) 的範疇。

第三篇 (第 13 ~ 21 章) 將處理這些問題：第 13、14 章建立動力學方程 (Lagrange 法與 Newton-Euler 法)、第 15 章規劃考慮動態限制的軌跡、第 16 ~ 19 章設計控制器、第 20 章處理力控制、第 21 章討論抓取。屆時你會發現，本篇建立的雅可比矩陣仍然無所不在——它將出現在慣性矩陣、科氏項、以及操作空間控制的每一個公式中。

換句話說：第二篇教你機器人**能不能到那裡**，第三篇教你**該用多大的力氣到那裡**。

完整計算例題

例題 12-1 平面 2R 手臂的奇異點分析

題目：平面 2R 手臂 $L_1 = 300 \text{ mm}$ 、 $L_2 = 200 \text{ mm}$ 。(a) 求所有奇異位形。(b) 在 $\theta_2 = 5^\circ$ 時計算雅可比、行列式、奇異值與條件數。(c) 若末端要以 100 mm/s 沿最難運動的方向移動，所需的關節速度為多少？(d) 評估此姿態是否安全。

解 (a)：由第 11 章例題 11-3， $\det(J) = L_1 L_2 \sin \theta_2$ 。令其為零：

$$L_1 L_2 \sin \theta_2 = 0 \Rightarrow \theta_2 = 0^\circ \text{ 或 } 180^\circ$$

奇異位形只取決於 θ_2 ：完全伸直 (0°) 或完全折疊 (180°)，與 θ_1 完全無關。

解 (b)：取 $\theta_1 = 30^\circ$ 、 $\theta_2 = 5^\circ$ 。則 $\theta_1 + \theta_2 = 35^\circ$ 。

$$J = \begin{bmatrix} -300 \sin 30^\circ - 200 \sin 35^\circ & -200 \sin 35^\circ \\ 300 \cos 30^\circ + 200 \cos 35^\circ & 200 \cos 35^\circ \end{bmatrix} = \begin{bmatrix} -264.72 & -114.72 \\ 423.61 & 163.83 \end{bmatrix}$$

單位 mm/rad 。

$$\det(J) = L_1 L_2 \sin 5^\circ = 300 \times 200 \times 0.08716 = 5229.6 \text{ mm}^2/\text{rad}^2$$

行列式，遠小於最大值 60000。

計算奇異值 (此處直接給出 SVD 結果)： $\sigma_1 \approx 528.4$ 、 $\sigma_2 \approx 9.90 \text{ mm/rad}$ 。

$$\kappa = \sigma_1 / \sigma_2 = 528.4 / 9.90 \approx 53.4$$

條件數約 53，依表 12-3 落在「警戒」區間。

解 (c)：最難運動的方向對應最小奇異值 $\sigma_2 = 9.90 \text{ mm/rad}$ 。所需的關節速度大小為：

$$\| \dot{\mathbf{q}} \| = \| \dot{\mathbf{x}} \| / \sigma_{\min} = 100 / 9.90 \approx 10.1 \text{ rad/s} \approx 579^\circ/\text{s}$$

關節需以每秒近 580 度轉動——遠超過一般伺服馬達的能力（典型上限約 180~360 °/s）。

作為對比，若沿最容易的方向（ $\sigma_1 = 528.4$ ）移動同樣的 100 mm/s，只需 $100/528.4 \approx 0.19 \text{ rad/s} \approx 10.8^\circ/\text{s}$ 。同樣的末端速度，兩個方向所需的關節速度相差 53 倍——這正是條件數的物理意義。

解 (d)：不安全。條件數 53 已進入警戒區，且沿某方向的關節速度需求已超過馬達能力。建議措施：一、若軌跡允許，把 θ_2 增大到 30° 以上（此時 κ 降至約 10 以下）；二、若必須經過此姿態，啟動 DLS 阻尼並降低末端速度至約 30 mm/s 以下；三、規劃時把此區域標記為避開區。

例題 12-2 冗餘手臂的零空間自運動

題目：某平面 3R 手臂 $L_1 = L_2 = L_3 = 200 \text{ mm}$ ，執行「末端到達指定位置 (x, y)」的任務（僅位置，2 維）。目前姿態為 $\theta = (30^\circ, 45^\circ, -30^\circ)$ 。（a）求冗餘度。（b）求雅可比與零空間的維度。（c）求零空間的基底向量，並解釋其物理意義。（d）若要沿零空間運動使第二關節角度減小，該如何選擇 $\mathbf{z}$ ？

解 (a)：關節數 $n = 3$ 、任務維度 $m = 2$ （只控制位置，不控制姿態）。

$$\text{冗餘度 } r = n - m = 3 - 2 = 1$$

有一個冗餘自由度。

解 (b)：位置雅可比為 2×3 矩陣。各關節的累積角度為 $\theta_1 = 30^\circ$ 、 $\theta_{12} = 75^\circ$ 、 $\theta_{123} = 45^\circ$ 。

$$\mathbf{J} = \begin{bmatrix} -L_1 s_1 - L_2 s_{12} - L_3 s_{123} & -L_2 s_{12} - L_3 s_{123} & -L_3 s_{123} \\ L_1 c_1 + L_2 c_{12} + L_3 c_{123} & L_2 c_{12} + L_3 c_{123} & L_3 c_{123} \end{bmatrix}$$

2×3 的位置雅可比。

代入數值（ $s_1=0.5$, $c_1=0.866$, $s_{12}=0.966$, $c_{12}=0.259$, $s_{123}=c_{123}=0.707$ ）：

$$\mathbf{J} = \begin{bmatrix} -434.5 & -334.5 & -141.4 \\ 346.9 & 193.2 & 141.4 \end{bmatrix}$$

單位 mm/rad。

由於 J 為 2×3 且滿秩 ($\text{rank} = 2$)，零空間維度為 $n - \text{rank}(J) = 3 - 2 = 1$ 維。

解 (c)：零空間的基底向量 v 滿足 $J \cdot v = 0$ 。解此齊次方程組可得 (取單位化後)：

$$v \approx [0.412, -0.833, 0.370]^T$$

零空間基底。沿此方向轉動關節，末端位置完全不變。

物理意義：三個關節以此比例同時轉動時，手臂的形狀改變 (中間的關節位置移動)，但末端維持在原處。這正是「手掌固定、手肘擺動」的平面版本。可驗證 $J \cdot v \approx [0, 0]^T$ 。

解 (d)：要使 θ_2 減小，次要目標可定義為 $H(q) = -\theta_2$ (最大化 $-\theta_2$ 即最小化 θ_2)，其梯度為 $\nabla H = [0, -1, 0]^T$ 。代入零空間投影：

$$q'_{\text{null}} = (I - J^+J) \cdot k \cdot [0, -1, 0]^T$$

投影後的結果必定平行於 v (因為零空間是一維的)，方向取決於 $[0, -1, 0]$ 與 v 的內積符號。

由於 v 的第二個分量为 -0.833 (負值)，而目標梯度的第二個分量也為負，兩者內積為正，因此投影結果會沿 $+v$ 方向——即 $q' \approx k \cdot [0.412, -0.833, 0.370]^T$ ，確實使 θ_2 減小，同時 θ_1 與 θ_3 自動調整以維持末端不動。

例題 12-3 六軸手臂的腕部奇異判定

題目：某六軸手臂目前關節角度為 $\theta = (20^\circ, -40^\circ, 60^\circ, 15^\circ, 2^\circ, -35^\circ)$ 。(a) 判斷是否接近腕部奇異點。(b) 若第五軸持續減小到 0.5° ，估算條件數的變化趨勢。(c) 提出兩種處理方式。

解 (a)：腕部奇異的條件是第五軸接近 0° 或 180° (第 4、6 軸共線)。目前 $\theta_5 = 2^\circ$ ，極為接近奇異點。

對球形手腕，其姿態部分的雅可比行列式正比於 $\sin \theta_5$ ：

$$\det(J_{\text{wrist}}) \propto \sin \theta_5 = \sin 2^\circ = 0.0349$$

僅為最大值 ($\theta_5 = 90^\circ$ 時) 的 3.5%。

解 (b)：條件數大致與 $1/\sin \theta_5$ 成正比。當 θ_5 由 2° 降至 0.5° ：

$$\kappa \text{ 的比值} \approx \sin 2^\circ / \sin 0.5^\circ = 0.0349 / 0.00873 \approx 4.0$$

條件數會惡化約 4 倍。若原本 $\kappa \approx 60$ (警戒)，將升至約 240 (危險)。

這說明了一個重要特性：接近奇異點時，狀況惡化的速度是非線性的。從 $\theta_5 = 10^\circ$ 降到 5°

只惡化 2 倍，但從 2° 降到 0.5° 卻惡化 4 倍，從 0.5° 降到 0.1° 則惡化 5 倍。這正是為何必須設定門檻並提早介入，而非等到真正逼近才處理。

解 (c)：處理方式一（調整工具姿態）：多數任務對工具繞其自身軸的旋轉角度並不敏感（例如吸盤、圓形夾爪、點焊槍）。若把工具的目標姿態繞末端 Z 軸旋轉一個角度，往往能讓 θ_5 遠離 0° ，完全避開此奇異點。這是成本最低、效果最好的作法。

處理方式二（DLS 加降速）：若姿態不可調整，則啟動阻尼並降低末端角速度。依 12.5.1 節，可容許的末端速度應與 $\sigma_{\min}$ 成正比—— $\theta_5 = 2^\circ$ 時，末端角速度應降至正常值的 3.5% 以下。這會嚴重影響節拍，因此僅作為備案。

程式範例：奇異點分析與零空間控制

以下 Python 程式實作奇異點的偵測與量化、SVD 幾何解讀、DLS 阻尼比較，以及冗餘手臂的零空間自運動。

```
import numpy as np

# ----- 平面手臂通用模型 -----
class PlanarArm:
    def __init__(self, links):
        self.L = np.array(links, dtype=float)
        self.n = len(links)

    def fk(self, q):
        """回傳末端位置 (x, y)"""
        ang = np.cumsum(q)
        return np.array([np.sum(self.L * np.cos(ang)),
                        np.sum(self.L * np.sin(ang))])

    def jacobian(self, q):
        """2xn 位置雅可比"""
        ang = np.cumsum(q)
        J = np.zeros((2, self.n))
        for j in range(self.n):
            J[0, j] = -np.sum(self.L[j:] * np.sin(ang[j:]))
            J[1, j] = np.sum(self.L[j:] * np.cos(ang[j:]))
        return J

    def analyze(self, q):
        """回傳姿態的「健康度報告」"""
        J = self.jacobian(q)
        U, S, Vt = np.linalg.svd(J)
        w = np.sqrt(np.linalg.det(J @ J.T))      # 可操作度
        kappa = S[0] / S[-1] if S[-1] > 1e-12 else np.inf
```



```

        return dict(J=J, U=U, S=S, Vt=Vt, w=w, kappa=kappa)

d, r2d = np.deg2rad, np.rad2deg

# ----- 例題 12-1：平面 2R 奇異點分析 -----
print("=== 例題 12-1：平面 2R 奇異點掃描 ===")
arm2 = PlanarArm([300.0, 200.0])
print(f"{'θ2':>6} {'det(J)':>10} {'σ_min':>9} {'κ':>9} 狀態")
for t2 in (0.5, 5, 15, 30, 60, 90, 120, 179):
    r = arm2.analyze([d(30), d(t2)])
    det = np.linalg.det(r['J'])
    if r['kappa'] < 5: st = "優良"
    elif r['kappa'] < 20: st = "可接受"
    elif r['kappa'] < 100: st = "警戒"
    else: st = "危險"
    print(f"{t2:6.1f} {det:10.1f} {r['S'][-1]:9.2f} {r['kappa']:9.1f} {st}")
print("→ 驗證 det(J) = L1·L2·sin(θ2) · 奇異點在 θ2 = 0° 與 180°")

# ----- SVD 幾何解讀 (12.4 節) -----
print("\n=== SVD 解讀：θ2 = 5° (接近奇異) ===")
r = arm2.analyze([d(30), d(5)])
print(f"奇異值 σ = ({r['S'][0]:.2f}, {r['S'][1]:.2f}) mm/rad")
print(f"最易運動方向 u1: 角度 {r2d(np.arctan2(r['U'][1,0], r['U'][0,0])):7.1f}°"
      f" 增益 {r['S'][0]:.1f}")
print(f"最難運動方向 u2: 角度 {r2d(np.arctan2(r['U'][1,1], r['U'][0,1])):7.1f}°"
      f" 增益 {r['S'][1]:.1f}")
for i, name in enumerate(("最易", "最難")):
    need = 100.0 / r['S'][i]
    print(f"沿{name}方向以 100 mm/s 運動 → 關節速度 {r2d(need):7.1f} °/s")

# ----- 偽逆 vs DLS 在奇異點附近 (12.5 節) -----
print("\n=== 偽逆 vs DLS：末端 100 mm/s 沿最難方向 ===")
x_dot = r['U'][:, 1] * 100.0 # 沿最難方向
J = r['J']
qd_pinv = np.linalg.pinv(J) @ x_dot
for lam in (0.0, 5.0, 20.0):
    if lam == 0.0:
        qd, tag = qd_pinv, "偽逆(λ=0)"
    else:
        qd = J.T @ np.linalg.solve(J @ J.T + lam**2*np.eye(2), x_dot)
        tag = f"DLS(λ={lam:.0f})"
    achieved = J @ qd
    err = np.linalg.norm(x_dot - achieved)
    print(f" {tag:12s} 關節速度 max={r2d(np.abs(qd).max()):8.1f} °/s "
          f"末端誤差={err:6.2f} mm/s")
print(" → DLS 大幅降低關節速度，代價是末端略微偏離目標")

```

```

# ----- 例題 12-2：冗餘手臂的零空間 -----
print("\n=== 例題 12-2：平面 3R 的零空間自運動 ===")
arm3 = PlanarArm([200.0, 200.0, 200.0])
q = np.array([d(30), d(45), d(-30)])
J3 = arm3.jacobian(q)
print(f"雅可比 2x3 · 秩 = {np.linalg.matrix_rank(J3)} · "
      f"零空間維度 = {3 - np.linalg.matrix_rank(J3)}")

# 求零空間基底 (SVD 的最後一個右奇異向量)
U, S, Vt = np.linalg.svd(J3)
v_null = Vt[-1]
print(f"零空間基底 v = {np.round(v_null, 4)}")
print(f"驗證 J·v = {np.round(J3 @ v_null, 10)} (應為零向量)")

# 沿零空間運動：末端不動 · 形狀改變
print("\n 沿零空間自運動 ( 每步 5° ) : ")
P_null = np.eye(3) - np.linalg.pinv(J3) @ J3      # 零空間投影算子
q_sm = q.copy()
for step in range(4):
    pos = arm3.fk(q_sm)
    print(f" 步{step}: θ=({r2d(q_sm[0]):6.1f},{r2d(q_sm[1]):6.1f}, "
          f"{r2d(q_sm[2]):6.1f})° 末端=({pos[0]:7.2f},{pos[1]:7.2f})")
    q_sm = q_sm + P_null @ (d(5) * np.sign(v_null))
print(" → 三個關節角度都在變 · 但末端位置幾乎不動")

# ----- 次要目標：最大化可操作度 -----
print("\n=== 零空間次要目標：最大化可操作度 ===")
def manip(arm, q):
    J = arm.jacobian(q)
    return np.sqrt(np.linalg.det(J @ J.T))

q_opt = q.copy()
print(f"起始: w = {manip(arm3, q_opt):.1f}, 末端 = {np.round(arm3.fk(q_opt),2)}")
for it in range(30):
    J3 = arm3.jacobian(q_opt)
    Pn = np.eye(3) - np.linalg.pinv(J3) @ J3
    grad = np.zeros(3)      # 數值梯度
    for j in range(3):
        dq = np.zeros(3); dq[j] = 1e-5
        grad[j] = (manip(arm3, q_opt+dq) - manip(arm3, q_opt-dq)) / 2e-5
    q_opt = q_opt + 2e-6 * (Pn @ grad)      # 只沿零空間爬升
print(f"最佳化後: w = {manip(arm3, q_opt):.1f}, "
      f"末端 = {np.round(arm3.fk(q_opt),2)}")
print(f"關節角度: ({r2d(q_opt[0]):.1f}, {r2d(q_opt[1]):.1f}, "
      f"{r2d(q_opt[2]):.1f})°")
print("→ 可操作度提升 · 末端位置維持不變——這就是冗餘的價值")

```

請試著修改：（一）把 arm2 的掃描細分到 $\theta_2 = 0.1^\circ$ ，觀察條件數如何爆炸。（二）在零空間自運動中改用「遠離關節極限」作為次要目標（梯度指向各關節行程的中點），觀察姿態如何變化。（三）把平面 3R 改為控制位置與姿態（3 維任務），此時冗餘度為 0，驗證零空間會消失。

實作：奇異點的實驗觀察

本章實作是在實體手臂上親眼觀察奇異點的三個症狀。**安全提醒：本實驗必須在低速下進行，並確保緊急停止隨手可及（第 7.9 節）。**

- 繪製可操作度地圖：**以第 11 章的程式計算工作空間中各點的可操作度，繪成熱力圖。標出可操作度低於最大值 10% 的「危險區」。
- 邊界奇異點觀察：**讓末端沿一條徑向直線由內往外緩慢移動（速度設為額定的 10%），記錄各關節的速度變化。接近工作空間邊界時，觀察哪個關節的速度開始上升，以及控制器是否發出警告。
- 條件數即時監控：**撰寫程式即時計算並顯示條件數。手動操作手臂經過不同區域，觀察數值變化，找出條件數超過 20 的所有姿態。
- 內部奇異點搜尋：**若使用六軸手臂，把第五軸緩慢轉向 0° ，觀察第四軸與第六軸的行為。嘗試讓末端繞某個軸小幅轉動，記錄兩軸的速度指令。
- 三個症狀的驗證：**在接近奇異的姿態下，分別驗證：（a）某方向無法運動（嘗試沿該方向移動，觀察是否報錯）；（b）關節速度暴增（記錄速度指令）；（c）逆解退化（用第 10 章的程式求逆解，觀察多組解是否合併）。
- DLS 效果比較：**以第 10 章的程式，分別用偽逆與 DLS 執行同一段經過奇異區的軌跡（僅在模擬中執行），比較關節速度峰值與末端路徑誤差。
- 繞道策略驗證：**修改軌跡使其避開危險區，比較繞道前後的執行時間、最大關節速度與是否觸發警報。

表 12-7 奇異點觀察記錄表（第一列為範例）

測試項	姿態	條件數	最大關節速度	末端速度	控制器反應
邊界奇異	$\theta_2 = 8^\circ$	42	$312^\circ/\text{s}$	50 mm/s	警告：接近奇異點
（自行填寫其餘測試）					

國中小也看得懂：有一個方向你就是去不了

伸直你的手臂，指尖碰到牆壁。現在，試著讓指尖**再往前一點點**——做不到對不對？不管你怎麼用力、怎麼轉肩膀，手臂已經伸到極限了。這就是機器人學裡說的「奇異點」：有一個方向，你就是去不了。

再試一個更神奇的實驗。把手臂伸直（但不要碰牆），然後試著讓指尖往前移動**一公分**。你會發現：肩膀和手肘必須轉好大一個角度，指尖才會往前一點點。**越接近伸直，就要轉越多才能前進一點點**。機器人也一樣——這時候它的馬達會拚命高速旋轉，控制器就會發出警告：「太危險了，停下來！」

那怎麼辦呢？聰明的辦法是**多一個關節**。你的手臂其實有七個可以轉的地方（肩膀三個、手肘一個、手腕三個），比「把手放到某個位置」需要的六個還多一個。多出來的那一個，讓你可以在手不動的情況下擺動手肘——繞開那些「去不了」的姿勢。

所以工程師設計機器人時，會刻意讓某些機器人多一隻「手肘」。這樣它就能像你一樣，遇到卡住的時候換個姿勢再試一次。你在附錄 E、J 做的機器人，關節越多就越靈活，原因就在這裡喔！

叫獸提醒與常見錯誤

- 誤區一：以為奇異點可以靠好的演算法消除。它是機構幾何的性質，任何串聯手臂都必定存在，只能避開或繞道。
- 誤區二：只檢查「是否恰好在奇異點」。真正危險的是奇異點**附近**——此時數學上有解，控制器會忠實執行一個極大的速度指令。應監控 $\sigma_{\min}$ 或條件數。
- 誤區三：用行列式判斷冗餘機器人的奇異。非方陣沒有行列式，應改用最小奇異值或 $\sqrt{\det(JJ^T)}$ 。
- 誤區四：只看可操作度 w 。它是所有奇異值的乘積，某一方向嚴重退化時可能被其他方向的大值掩蓋。應同時檢視 $\sigma_{\min}$ 。
- 誤區五：混淆兩種零空間。冗餘造成的零空間是可用資源；奇異造成的零空間則伴隨任務方向的損失。判斷方法是檢查 J 的秩。

- 誤區六：次要目標的增益設得太大。雖然理論上不影響主任務，但數值誤差累積與過快的自運動都可能造成問題。
- 誤區七：以為分層越多越好。每一層投影都會惡化條件數，層數過多時數值品質會嚴重下降。
- 誤區八：忽略腕部奇異的普遍性。夾爪垂直向下這類常用姿態很容易落在第五軸接近 0° 的位置，是工業現場最頻繁的異常來源。
- 誤區九：規劃時只檢查起訖點。奇異點可能出現在路徑中間，必須對整條軌跡逐點檢查條件數。
- 誤區十：以為換一台更大的機器人就能避開。奇異位形只與構型有關、與尺寸無關，同構型的大小手臂奇異點完全相同。

課堂討論

1. 開頭故事中控制器報的是「速度超限」而非「無法到達」。請解釋兩者的差異，並討論為何後者反而比較安全。
2. 第 12.5 節列出五種處理奇異點的策略。請為以下三個情境各選擇最合適的策略並說明理由：（a）離線編程的點焊軌跡；（b）操作員用搖桿遙控的遠端作業；（c）新產線的機器人選型。
3. 協作機器人多採七軸設計。除了本章提到的繞開奇異點，冗餘還帶來哪些好處？代價又是什麼？請結合第 4.9.3 節討論。
4. 嚴格分層逆運動學保證了優先權，但數值品質可能較差。請設計一個「混合式」方案：哪些任務用嚴格分層、哪些用加權合併？判準是什麼？
5. 本章證明了萬向節鎖、手臂伸直、逆解合併、可操作度歸零是同一件事。請再找出本書前面章節中至少一個現象，說明它是否也能用雅可比的秩來解釋。

章末摘要

- 奇異點的定義是雅可比矩陣失去滿秩的位形；物理意義是某兩個以上的關節產生重複的運動效果，等於少了一個自由度。
- 三種判準： $\det(J) = 0$ （僅方陣）、可操作度 $w = 0$ 、最小奇異值 $\sigma_{\min} = 0$ （最推薦）。

- 奇異點的三個症狀：某方向運動能力喪失、關節速度暴增、多組逆解合併。其中第二個最危險，因為數學上仍有解。
- 邊界奇異點發生於工作空間邊界（手臂伸直或折疊），容易避開；內部奇異點位置分散且不直觀，更為危險。
- 六軸手臂的三類典型奇異：腕部（第 5 軸 $\approx 0^\circ$ ，最常見）、肩部（腕心落在第 1 軸線上）、肘部（手臂完全伸直）。
- 奇異位形只與構型有關、與機構尺寸無關，同構型的大小手臂奇異點完全相同。
- SVD 給出完整的幾何解讀： u_k 是失去的任務方向、 v_k 是徒勞的關節運動方向、 σ_k 是該方向的運動增益。
- 條件數 $\kappa = \sigma_{\max}/\sigma_{\min}$ 是無量綱指標； $\kappa < 5$ 優良、 $5 \sim 20$ 可接受、 $20 \sim 100$ 警戒、 >100 危險。
- 五種處理策略：軌跡繞道、降速、DLS 阻尼、改變構型、增加自由度；實務常採多層複合防護。
- 任何串聯旋轉關節手臂必定存在奇異點，這是數學必然，工程上只能避開而非消除。
- 冗餘度 $r = n - m$ ；零空間 $N(J)$ 是所有不產生末端運動的關節速度所構成的集合，沿其運動稱為自運動。
- 零空間有兩種來源：冗餘造成的（可用資源，不損失任務方向）與奇異造成的（伴隨任務方向損失）。
- 通解結構 $\dot{q} = J^+ \dot{x} + (I - J^+ J) \dot{z}$ ，第二項提供不干擾主任務的通道，可用於次要目標。
- 四種常見次要目標：遠離關節極限、最大化可操作度、避開障礙物、維持自然姿勢。
- 多任務可用加權合併（簡單但會互相妥協）或分層逆運動學（嚴格優先權但數值品質較差），實務常採折衷。
- 第二篇的四塊拼圖：正向（手在哪裡）、逆向（手該怎麼擺）、雅可比（手能怎麼動）、奇異與冗餘（手何時動不了、多餘自由度怎麼用）。

觀念題與計算題

觀念題

1. 以雅可比矩陣的秩定義奇異點，並說明「兩個關節做同一件事」的物理意義。

2. 比較三種奇異點判準的適用範圍與優缺點，並說明為何推薦使用最小奇異值。
3. 說明奇異點的三個症狀，並解釋為何「關節速度暴增」比「運動能力喪失」更危險。
4. 區分邊界奇異點與內部奇異點，並說明後者為何更難處理。
5. 說明六軸手臂三類典型奇異點的發生條件與典型症狀。
6. 為何說「奇異位形與機構尺寸無關」？這個結論對模擬驗證有什麼意義？
7. 說明 SVD 中 U 、 Σ 、 V 三個矩陣在奇異點分析中各代表什麼。
8. 定義條件數並說明其實務判讀標準；為何說它是「無量綱的相對指標」？
9. 列舉五種處理奇異點的策略，並比較各自在路徑精度與節拍上的影響。
10. 為何說奇異點無法被消除？請以拓撲學的必然性與第 8 章的類比說明。
11. 定義冗餘度與零空間，並說明自運動的物理意義。
12. 區分「冗餘造成的零空間」與「奇異造成的零空間」，並說明判斷方法。
13. 寫出冗餘機器人速度逆解的通解結構，並說明零空間投影算子的兩個性質。
14. 列舉四種常見的次要目標及其目標函數設計。
15. 比較加權合併與分層逆運動學的優缺點，並說明實務上的折衷作法。

計算題

1. 平面 2R 手臂 $L_1 = 400 \text{ mm}$ 、 $L_2 = 250 \text{ mm}$ 。(a) 求所有奇異位形。(b) 在 $\theta_1 = 45^\circ$ 、 $\theta_2 = 10^\circ$ 時求雅可比與行列式。(c) 求可操作度，並與最大可能值比較。(d) 若最小奇異值為 17.4 mm/rad ，求沿最難方向以 80 mm/s 運動所需的關節速度 (度/秒)，並判斷一般伺服馬達 (上限 $300^\circ/\text{s}$) 是否足夠。
2. 同上手臂。(a) 若要求條件數不超過 20，試由 $\det(J) = L_1 L_2 \sin \theta_2$ 的關係，估算 θ_2 的允許範圍 (提示：可先用程式掃描)。(b) 由此結果，建議末端的安全作業半徑範圍。(c) 若把 L_2 改為 400 mm (兩連桿等長)，內邊界奇異會發生什麼變化？
3. 某平面 3R 手臂 $L_1 = L_2 = L_3 = 150 \text{ mm}$ ，僅控制末端位置 (2 維任務)。目前姿態 $\theta = (20^\circ, 50^\circ, -40^\circ)$ 。(a) 求冗餘度與零空間維度。(b) 求 2×3 的雅可比矩陣。(c) 驗證雅可比是否滿秩。(d) 若次要目標為「使第三關節角度趨近於 0° 」，寫出對應的目標函數與梯度。
4. 某六軸手臂在某姿態下的六個奇異值為 $\sigma = (520, 380, 295, 88, 41, 6.2)$ (單位已正規化)。

(a) 求可操作度與條件數。(b) 依表 12-3 判斷其狀態。(c) 若末端各方向的速度需求皆為 100 單位，求所需關節速度的最大值。(d) 若採用 DLS 且 $\lambda = 15$ ，估算最小奇異值方向的有效增益變化 (提示：DLS 的等效增益為 $\sigma/(\sigma^2 + \lambda^2)$)。

動手做與專題延伸

本章實作：奇異點分析工具箱

延續第 9~11 章的程式，完成一套完整的運動學分析工具。這將是第二篇的總成果，也是後續章節的基礎。

- **階段一**：整合前三章的模組（正向運動學、逆運動學、雅可比），並加入本章的奇異點分析（ $\sigma_{\min}$ 、條件數、可操作度）。
- **階段二**：實作「姿態健康度報告」函式：輸入關節角度，輸出完整診斷——可操作度、條件數、最小奇異值、失去的運動方向、與最近奇異點的估計距離、以及建議措施。
- **階段三**：實作軌跡的預先檢查功能：輸入一串目標點，逐點求逆解並檢查條件數，回報所有超過警戒門檻的路段。
- **階段四**：對冗餘機器人實作零空間投影與次要目標最佳化，支援至少兩種次要目標。

專題延伸

- **可操作度地圖**：對平面手臂繪製工作空間的可操作度熱力圖，並以等高線標出條件數 20 與 100 的邊界。加入關節極限的限制，觀察可用區域如何縮小。這張圖可直接作為工作站佈局的參考。
- **奇異點穿越的動態比較**：規劃一條刻意穿越奇異點的直線軌跡，分別以偽逆、DLS（三種不同 λ ）與降速策略執行（模擬中），繪出關節速度隨時間的曲線與末端路徑誤差。整理成圖表，量化「路徑精度」與「關節速度」之間的取舍關係。
- **進階：七軸手臂的自運動視覺化**。為七軸手臂（可用公開的 URDF，如 Franka Emika 或 KUKA iiwa）實作零空間自運動：固定末端位姿，沿零空間連續運動並繪製手肘位置的軌跡。你會看到手肘畫出一個完整的圓弧——這就是「自運動流形」。進一步，在此流形上分別找出可操作度最大、最遠離關節極限、以及最不易碰撞的三個姿態，並討論它們是否重合。

參考文獻與延伸閱讀

- Craig, J. J., Introduction to Robotics: Mechanics and Control, 4th ed., Pearson (第 5 章：奇異點與雅可比) 。
- Nakamura, Y., Advanced Robotics: Redundancy and Optimization, Addison-Wesley (冗餘機器人與零空間的經典專著) 。
- Yoshikawa, T., "Manipulability of Robotic Mechanisms," IJRR, 1985 (可操作度與奇異點的量化指標) 。
- Chiaverini, S., "Singularity-Robust Task-Priority Redundancy Resolution," IEEE Trans. Robotics and Automation, 1997 (分層逆運動學與奇異點強健性) 。
- Siciliano, B. and Slotine, J.-J., "A General Framework for Managing Multiple Tasks in Highly Redundant Robotic Systems," ICAR 1991 (多任務優先權框架的奠基論文) 。
- Maciejewski, A. A. and Klein, C. A., "Obstacle Avoidance for Kinematically Redundant Manipulators," IJRR, 1985 (零空間避障的經典應用) 。
- Toussaint, M., Introduction to Robotics 課程講義, TU Berlin (多任務、分層逆運動學與正則化的實務討論) 。
- Lynch, K. M. and Park, F. C., Modern Robotics, Cambridge University Press (第 5 章：奇異點的旋量理論觀點) 。
- Escande, A., Mansard, N. and Wieber, P.-B., "Hierarchical Quadratic Programming," IJRR, 2014 (現代的分層最佳化求解法) 。
- 各機器人廠商的技術手冊中「奇異點」與「特異姿態」章節，含實機的奇異區域圖與規避建議。